Threshold Signatures Reloaded
ML-DSA and Enhanced Raccoon with Identifiable Aborts

Giacomo Borin[1], Sofia Celi[2], Rafael del Pino[3], Thomas Espitau[3], Guilhem Niot[3,4], Thomas Prest[3]

[1]IBM Research Europe & University of Zurich
[2]Brave Research & University of Bristol
[3]PQShield
[4]Univ Rennes, CNRS, IRISA

Abstract

Threshold signatures enable multiple participants to collaboratively produce a digital signature, ensuring both fault tolerance and decentralization. As we transition to the post-quantum era, lattice-based threshold constructions have emerged as promising candidates. However, existing approaches often struggle to scale efficiently, lack robustness guarantees, or are incompatible with standard schemes — most notably, the NIST-standard ML-DSA.

In this work, we explore the design space of Fiat-Shamir-based lattice threshold signatures and introduce the two most practical schemes to date. First, we present an enhanced TRaccoon-based [DKM+24] construction that supports up to 64 participants with identifiable aborts, leveraging novel short secret-sharing techniques to achieve greater scalability than previous state-of-the-art methods. Second — and most importantly — we propose the first practical ML-DSA-compatible threshold signature scheme, supporting up to 6 users.

We provide full implementations and benchmarks of our schemes, demonstrating their practicality and efficiency for real-world deployment as protocol messages are computed in at most a few milliseconds, and communication cost ranges from 10.5 kB to 525 kB depending on the threshold.

1 Introduction

Threshold signature schemes (TSS) are an essential cryptographic primitive that allows a group of users to generate signatures collaboratively. In a threshold signature scheme, a private key is distributed among N participants in such a way that any subset of at least T participants (where $T \leqslant N$) can jointly produce a valid signature, while any subset of $T-1$ or fewer participants cannot.

The practical applications of threshold signatures span various domains that require security, decentralization, and fault tolerance. Financial institutions use threshold signatures to safeguard high-value transactions, requiring multiple approvals before transferring funds. Blockchain networks employ these schemes to secure multi-signature wallets and facilitate decentralized consensus mechanisms without single points of failure, typically in the $N = 3, T = 2$ setting. Critical infrastructure systems use them to ensure that sensitive operations require approval from multiple authorized parties, thereby mitigating external attacks and insider threats.

While numerous candidates have been proposed in the classical setting and are competitive in terms of communication cost and efficiency, the situation becomes significantly more complex in the post-quantum setting. Recently, the cryptographic community has focused on lattice-based threshold constructions. As research in this area progresses, *Fiat-Shamir-based signatures* combined with noise flooding have emerged as the baseline for the most promising solutions. Various trade-offs involving the number of users, robustness guarantees, and the capacity to identify corrupted parties during the signing operations have been identified.

As the research landscape evolves, a notable gap persists: the absence of threshold signature schemes that are interoperable with the NIST standard ML-DSA. By interoperability, we mean the resulting threshold signature should be verifiable using the standard ML-DSA signature scheme. This compatibility is crucial for real-world adoption because it allows threshold signatures to be seamlessly integrated into existing cryptographic infrastructures without necessitating modifications to verification procedures.

Our Contributions

In this paper, we address this gap and more broadly explore the theoretical and practical limits of *lattice-based threshold signatures built on the Fiat-Shamir paradigm*. We introduce several optimizations to existing solutions to accommodate large and small threshold regimes. In particular, we propose two approaches.

Enhanced Raccoon-based Scheme: T-Raccoon up to 64 parties. We design a scalable threshold signature scheme with *identifiable aborts* and a Distributed Key Generation (DKG), building on Raccoon. Our construction improves the short secret-sharing approach from prior works, enabling efficient operation with up to *64 parties*–a fourfold increase over the previous state-of-the-art [dPENP25], which supported only 16. It substantially improves the practicality of threshold signatures with *identifiable aborts* in large-scale distributed systems. Our scheme produces signatures of size 10.7 kB (NIST Level I) and 17.8 kB (Level V), which are *smaller* than those of the original TRaccoon scheme [DKM+24], while offering identifiable aborts.

ML-DSA-Compatible Threshold Scheme: Threshold ML-DSA up to 6 parties. We introduce the *first practical threshold variant of ML-DSA*, based on the Finally! signature scheme [dPN25]. Our scheme supports up to *6 parties* and ensures compatibility with ML-DSA via a tailored *ad hoc* replicated sharing technique and partial signing.
Our construction leverages recent techniques from [dPN25] and performs a per-party signature rejection sampling before aggregating them into a ML-DSA signature. In order to balance the degradation of requiring several parties to succeed signing simultaneously, we introduce critical optimizations to support 6 parties. This includes using compact distributions based on unbalanced hyperballs (c.f. parameters r, r', ν in Table 1) to remain under the parameters of ML-DSA, an optimized secret sharing reconstruction (c.f. Alg. 21), a tailored correctness analysis for ML-DSA's hint mechanism (Theorem 4.1 and Heuristic 1), the introduction of parallel protocol repetitions.

Both approaches rely on the concept of *short secret-sharing scheme* [dPENP25]s for key distribution among participants. In particular, we introduce a new short secret-sharing scheme compatible with a DKG and that scales up to 64 parties, particularly relevant for noise-flooded schemes. Our work further demonstrates the versatility of this class of secret sharing, reinforcing its value and applicability in real-world threshold protocols.

We implement and benchmark both schemes under realistic deployment scenarios in LAN and WAN environments, demonstrating their efficiency and suitability for practical use. Our approach is very efficient, with protocol messages computed in at most a few milliseconds, and a communication per party ranging from 10.5 kB to 525 kB depending on the threshold.

Related Work

Threshold signature schemes

There exists numerous TSS based on RSA [GHKR08], Schnorr signatures [KG20, CKM23] and ECDSA [CGG+20, CCL+23]. The design of practical schemes in the post-quantum setting however appears more challenging, with most of the literature proposing schemes incompatible with NIST standards or candidates. Proposals leveraged assumptions on lattices, isogenies [DM20], or hash functions [KCLM22]. The recent work [CEN24] proposed a threshold variant of the multivariate standard candidates MAYO and UOV using generic MPC and may achieve reasonable effiency.

Lattice-based Fiat-Shamir threshold schemes

[CS19, DKLS25] proposed to apply generic MPC to thresholdize Dilithium. However, the former is inefficient in computation time and communication, and the latter requires a trusted third party to produce correlated pseudorandomness, deviating from the stronger active threat model. In contrast, TSS based on plain Fiat-Shamir (without rejection sampling) have been proposed recently, where the scheme either relies on threshold FHE [ASY22, GKS24] or on standard tools [DKM+24, EKT24, KRT24, CATZ24] applied on (variants of) the Raccoon scheme [dEK+23], reaching better practicality at the cost of larger signature sizes. Advanced security properties of lattice threshold signatures have also been considered, such as adaptive security [KRT24], robustness [ENP24], identifiable aborts [dPKN+25], and Beyond UnForgeability Features (BUFF) [FMT25].

Improved Secret Sharing

For lattice-based threshold schemes, the secret sharing technique's choice dictates the resulting scheme's characteristics. Recent advances suggest that short secret sharing techniques [dPENP25] enable simpler and more secure constructions [BS23, CATZ24], are naturally compatible with rejection sampling [dPN25], and support identifiable aborts at no additional cost [dPENP25]. Consequently, a recent line of work has focused on identifying the most effective short secret sharings. For the parameters we consider, our *Vandermonde-based* secret sharing outperforms previously adopted constructions—see Section 3.1.

2 Preliminaries

2.1 General notations

In all this work, we denote by κ the security parameter.

Sets, functions, and distributions. For an integer $N > 0$, we denote $[N] = \{0, \dots, N-1\}$. To denote the *assign* operation, we use $y := f(x)$ when f is deterministic and $y \leftarrow f(x)$ when randomized. We recall that the union $S \cup T$ of two sets, written $S \sqcup T$ when disjoint.

Dictionaries. We recall that a dictionary, or associative array, is a collection of ordered (key, value) pairs with unique keys. We denote $\{:\}$ the empty dictionary, and $\mathsf{Dict}[k]$ the value of Dict corresponding to the key k. We note $\mathsf{Dict}_1 \cup \mathsf{Dict}_2$ the union of two dictionaries. When a key k is present in both Dict_1 and Dict_2 and $\mathsf{Dict}_1[k], \mathsf{Dict}_2[k]$ are either both sets or both dictionaries, the new value $(\mathsf{Dict}_1 \cup \mathsf{Dict}_2)[k]$ sets as $\mathsf{Dict}_1[k] \cup \mathsf{Dict}_2[k]$.

Distributions. We also introduce useful distributions for this work. For a set S that is finite or with a finite measure, we denote $\mathcal{U}(S)$ the *uniform distribution* over S, and shorthand $x \xleftarrow{\$} S$ for $x \leftarrow \mathcal{U}(S)$. Given a real $0 \leqslant p \leqslant 1$, we define the *Bernoulli distribution* $\mathsf{Ber}(p)$ that returns 1 with probability p, and 0 otherwise. The *Binomial distribution* $\mathsf{Bin}(m, p)$ is the sum of m Bernoulli distributions $\mathsf{Ber}(p)$.

Gaussians. Given $\sigma > 0$, $\mathbf{x}, \mathbf{c} \in \mathbb{R}^m$, let $\rho_{\sigma,\mathbf{c}}(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{c}\|^2}{2\sigma^2}\right)$. We define the *discrete Gaussian distribution* $D_{\sigma,\mathbf{c}}$ over $\mathbb{Z}^m$ by its probability distribution function $D_{\sigma,\mathbf{c}}(\mathbf{x}) = \frac{\rho_{\sigma,\mathbf{c}}(\mathbf{x})}{\sum_{\mathbf{y}\in\mathbb{Z}^m} \rho_{\sigma,\mathbf{c}}(\mathbf{y})}$. We omit $\mathbf{c}$ when it is $\mathbf{0}$.

Linear algebra. Throughout the work, for a fixed power-of-two n, we denote $\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$ the cyclotomic ring and $\mathcal{R}_q = \mathcal{R}/(q\mathcal{R})$. We also let $\mathcal{R}_\mathbb{R} = \mathbb{R}[x]/(x^n+1)$ be a polynomial ring over real numbers. Given $\mathbf{x} \in \mathcal{R}_\mathbb{R}^\ell$, we denote $\|\mathbf{x}\|_2$ the Euclidean norm (resp. $\|\mathbf{x}\|_\infty$ the infinite norm) of the $(n\,\ell)$-dimensional vector of the coefficients of $\mathbf{x}$. Unless specified otherwise, vectors are *column* vectors. We extend Gaussian distributions over $\mathcal{R}_\mathbb{R}$ using vector coefficient embeddings.

Lattices. We define a lattice as a discrete subgroup of $\mathbb{R}^n$. We recall the definition of the smoothing parameter. The dual of a lattice L, noted $L^\vee$, is the set $L^\vee = \{\mathbf{x} \in \mathsf{Span}_{\mathbb{R}}(L) \mid \forall \mathbf{v} \in L, \langle x, v\rangle \in \mathbb{Z}\}$. For any lattice L, and any real $\varepsilon > 0$, the smoothing parameter $\eta_\varepsilon(L)$ is the smallest real $s > 0$ such that $\rho_{1/(\sqrt{2\pi}s)}(L^\vee \backslash \{0\}) \leqslant \varepsilon$. We also define a scaled variant $\eta'_\varepsilon(L) := \frac{1}{\sqrt{2\pi}}\eta_\varepsilon(L)$.

2.2 Threshold signatures

We recall the definition of an R-round threshold signature (*without* session identifiers) from [KRT24], as a tuple $\mathsf{TSS} = (\mathsf{Setup}, \mathsf{Keygen}, (\mathsf{Sign}_i)_{i\in[R]}, \mathsf{Combine}, \mathsf{Verify})$.

We denote $\mathsf{act} \subseteq [N]$ a set of T parties used for signing. Each signer i maintains a state st_i: short-lived session-specific information.

$\mathsf{Setup}(\kappa, N, T) \to \mathsf{pp}$**.** Takes as input a security parameter κ, the total number of parties N, the reconstruction threshold $T \leqslant N$. Outputs a set of public parameters pp.

$\mathsf{Keygen}(\mathsf{pp}) \to (\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]})$**.** Takes as input the public parameters. Outputs a verification key vk, and partial private keys $(\mathsf{sk}_i)_{i\in[N]}$.

$\mathsf{Sign}_r(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, (\mathsf{pm}_{r-1}^j)_{j\in\mathsf{act}}, i, \mathsf{sk}_i, \mathsf{st}_i) \to (\mathsf{pm}_r^i, \mathsf{st}_i)_{\{r=1,\cdots,R\}}$**.** For the r-th round, takes as input the verification key vk, the set of signers act, a message msg, the protocol messages exchanged during the previous round $(\mathsf{pm}_{r-1}^j)_{j\in\mathsf{act}}$, as well as the partial private key sk_i and state st_i of user i. Outputs a new message pm_r^i and updated state st_i. We define $\mathsf{pm}_0^i = \perp$. Note also that one may chose to delay passing $\mathsf{msg}, \mathsf{act}$ to a round later than the first.

$\mathsf{Combine}(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, (\mathsf{pm}_r^i)_{r\in[R], i\in\mathsf{act}}) \to \mathsf{sig}$**.** Takes as input the verification key vk, the set of signers act, the message msg, all messages exchanged during the protocol $(\mathsf{pm}_r^i)_{r\in[R],i\in\mathsf{act}}$. Outputs a signature sig.

$\mathsf{Verify}(\mathsf{vk}, \mathsf{msg}, \mathsf{sig}) \to 0$ **or** 1**.** Takes as input a verification key vk, a message msg, and a signature sig. Outputs 1 if sig is valid and 0 otherwise.

A key property of threshold signatures is unforgeability, which ensures that an adversary cannot produce valid signatures without the cooperation of honest parties. Several security models exist, differing in whether the adversary chooses corrupted users statically [DKM+24] or adaptively [KRT24], and whether the communication channel is controlled by the adversary [DKM+24], or requires consensus properties [ENP24, dPENP25], and for which messages a forgery is accepted. In this work, we consider the unforgeability notion from [DKM+24] adapted to remove the use of session identifiers. It ensures static security – i.e. corrupted users are chosen before executing the protocol, a communication channel fully controlled by the adversary, and forgeries are accepted for messages for which no signing oracle was called. We provide formal definitions of said properties in Section A.2. Note that static security implies adaptive security for small thresholds (T, N), with a theoretical loss of at most 5 bits up to $N = 6$ parties for Threshold ML-DSA, and 61 bits up to $N = 64$ for T-Raccoon. Still, this loss only appears to be due to techniques limitations rather than an actual weakness in our case.

2.3 Modulus Rounding

We recall the rounding and hint mechanisms used in ML-DSA and Raccoon to optimize the sizes of keys and signatures.

Rounding in Raccoon. Let $\nu \in \mathbb{N}\backslash\{0\}$. Any integer $x \in \mathbb{Z}$ can be *uniquely* decomposed as:

$$x = 2^\nu \cdot x_\top + x_\perp, \quad (x_\top, x_\perp) \in \mathbb{Z} \times [-2^{\nu-1}, 2^{\nu-1} - 1], \tag{1}$$

Define the function $\lfloor\cdot\rceil_\nu : \mathbb{Z} \to \mathbb{Z}$ such that $\lfloor x\rceil_\nu = \lfloor x/2^\nu\rceil = x_\top$, where $\lfloor\cdot\rceil : \mathbb{R} \mapsto \mathbb{Z}$ is the "rounding half-up" $\lfloor x\rceil = \lfloor x + \frac{1}{2}\rfloor$ where half-way values are rounded up: e.g. $\lfloor 2.5\rceil = 3$ and $\lfloor -2.5\rceil = -2$. When $q > 2^\nu$, we extend $\lfloor\cdot\rceil_\nu$ to take inputs in $\mathbb{Z}_q$ and we assume the output is an element in $\mathbb{Z}_{q_\nu}$ where $q_\nu = \lfloor q/2^\nu\rceil$. The function $\lfloor\cdot\rceil_\nu$ extends to vectors coefficient-wise.

Rounding and hints in ML-DSA We recall helper functions over $\mathbb{Z}_q$ for ML-DSA. These functions are applied coefficient-wise when applied to polynomials in $\mathcal{R}_q$.

$\mathsf{Power2Round}_q(r)$: rounds an integer $r \in \mathbb{Z}_q$. It outputs a pair (r_0, r_1), where $r_0 = r \bmod^{\pm} 2^d$ and $r_1 = (r - r_0)/2^d$.

$\mathsf{HighBits}(r, \alpha)$: returns the high-order bits r_1 of r, defined as $\mathsf{HighBits}(r, \alpha) = \frac{r-(r \bmod^{\pm} \alpha)}{\alpha}$ if $r - (r \bmod^{\pm} \alpha) \neq q - 1$ and 0 otherwise.

$\mathsf{MakeHint}_q(z, r, \alpha)$: produces a binary hint $h \in \{0, 1\}$ indicating that the high bits of z and $z + r$ differ. The hint mechanism ensures the correct reconstruction of high bits during verification.

$\mathsf{UseHint}_q(h, r, \alpha)$: takes a hint bit $h \in \{0, 1\}$ and an integer $r \in \mathbb{Z}_q$. Based on the hint, it corrects the computation of $\mathsf{HighBits}(r, 2\gamma_2)$. The goal is to recover $\mathsf{HighBits}(r+z, 2\gamma_2)$. If $h = 1$, a carry occurred, and the high bits need adjustment: the high bits by $+1$ if $r - \alpha \cdot r_1 > 0$, and otherwise by -1.

Lemma 2.1. *For $q, \alpha > 0$ with $q > 2\alpha$, $q = 1 \bmod \alpha$ and α is even. Let $\mathbf{r}, \mathbf{z} \in \mathcal{R}_q$ where $\|\mathbf{z}\|_\infty \leqslant \alpha/2$. Then,*

$$\mathsf{UseHint}_q(\mathsf{MakeHint}_q(\mathbf{z}, \mathbf{r}, \alpha), \mathbf{r}, \alpha) = \mathsf{HighBits}_q(\mathbf{r} + \mathbf{z}, \alpha).$$

2.4 Hardness Assumptions

We rely on the Hint-MLWE and STMSIS assumptions. STMSIS [KLS18] is a variant of MSIS where the problem is defined relative to some hash function. It underlies the security of ML-DSA and Raccoon. Hint-MLWE [KLSS23] generalizes MLWE, and tightly reduces to MLWE. We recall the variant with varying noise distributions from [ENP24].

Definition 2.1 (STMSIS). *Let k, ℓ, q be integers, $B_{\mathsf{stmsis}} > 0$ be a real number, and $\|\cdot\|$ be a norm. Let $\mathcal{C}$ be a subset of $\mathcal{R}_q$, and let $G : \mathcal{R}_q^k \times \{0,1\}^{2\kappa} \mapsto \mathcal{C}$ be a cryptographic hash function modeled as a random oracle. The advantage $\mathsf{Adv}_{\mathcal{A}}^{\mathsf{STMSIS}}(\kappa)$ of an adversary $\mathcal{A}$ against the Self Target MSIS problem $\mathsf{STMSIS}_{q,k,\ell,\mathcal{C},B_{\mathsf{stmsis}},\|\cdot\|}$ is defined as:*

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{STMSIS}}(\kappa) = \Pr\Big[\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k\times\ell},\ (\mathsf{msg}, \mathbf{z}) \leftarrow \mathcal{A}^G(\mathbf{A}),: \left(\mathbf{z} = \begin{bmatrix}\mathbf{z}' \\ c\end{bmatrix}\right) \\ \wedge\ (\|\mathbf{z}\| \leqslant B_{\mathsf{stmsis}})\ \wedge\ G(\begin{bmatrix}\mathbf{I} & \mathbf{A}\end{bmatrix} \cdot \mathbf{z}, \mathsf{msg}) = c\Big].$$

The $\mathsf{STMSIS}_{q,k,\ell,\beta}$ assumption states that any efficient adversary $\mathcal{A}$ has a negligible advantage $\mathsf{Adv}_{\mathcal{A}}^{\mathsf{STMSIS}}(\kappa)$.

Definition 2.2 (Hint-MLWE). *Let k, ℓ, q, Q be integers, $\chi, (\chi_i)_{i\in[Q]}$ be probability distributions over $\mathcal{R}^{k+\ell}$, and $\mathcal{M}$ be a distribution over $\mathcal{R}_q^{(k+\ell)\times(k+\ell)}$. The advantage of an algorithm $\mathcal{A}$ in solving the $\mathsf{Hint\text{-}MLWE}_{q,k,\ell,\chi,(\chi_i)_{i\in[Q]},\mathcal{M}}$ problem is defined as:*

$$\left|\Pr[\mathcal{A}(\mathbf{A}, \begin{bmatrix}\mathbf{I}_k & \mathbf{A}\end{bmatrix} \cdot \mathbf{s}, (\mathbf{M}_i, \mathbf{z}_i)_{i\in[Q]}) = 1] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{u}, (\mathbf{M}_i, \mathbf{z}_i)_{i\in[Q]}) = 1]\right|$$

where $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k\times\ell}, \mathbf{u} \xleftarrow{\$} \mathcal{R}_q, \mathbf{s} \leftarrow \chi, (\mathbf{M}_i)_{i\in[Q]} \leftarrow \mathcal{M}, \mathbf{p}_i \leftarrow \chi_i$, and $\mathbf{z}_i = \mathbf{M}_i \cdot \mathbf{s} + \mathbf{p}_i$ for $i \in [Q]$. We recover the MLWE problem with $Q = 0$. The Hint-MLWE assumption states that any efficient adversary $\mathcal{A}$ has a negligible advantage against Hint-MLWE.

Hint-MLWE is as hard as MLWE in some parameter regimes, as recalled in Section A.4.

2.5 The Rényi divergence

We rely on the *Rényi divergence of infinite order* R_∞ as well as the *smooth Rényi divergence* R_∞^ε to measure the distance between two distributions. We defer the definition of these divergences to Section A.1.

$\mathbf{Rej}(\mathbf{v}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M) \to \mathbf{z} \in \mathcal{R}^{k+\ell} \cup \{\perp\}$	$\mathbf{Ideal}(\chi_{\mathbf{z}}, M) \to \mathbf{z} \in \mathcal{R}^{k+\ell} \cup \{\perp\}$
1: $\mathbf{r} \leftarrow \chi_{\mathbf{r}}$ 2: $\mathbf{z} := \mathbf{v} + \mathbf{r}$ 3: $b \leftarrow \mathsf{Ber}\left(\min\left(\frac{\chi_{\mathbf{z}}(\mathbf{z})}{M\chi_{\mathbf{r}}(\mathbf{r})}, 1\right)\right)$ 4: **if** $\{\ b = 0\ \}$ **then** $\{\ \mathbf{z} = \perp\ \}$ 5: **return z**	1: $\mathbf{z} \leftarrow \chi_{\mathbf{z}}$ 2: $b \leftarrow \mathsf{Ber}\left(\frac{1}{M}\right)$ 3: **if** $\{\ b = 0\ \}$ **then** $\{\ \mathbf{z} = \perp\ \}$ 4: **return z**

Figure 1: Rejection algorithm and ideal algorithm.

2.6 Rejection Sampling

ML-DSA is a Fiat-Shamir with Aborts signature scheme. It heavily relies on the rejection sampling technique introduced in [Lyu09], which we recall with the same notations as [dPN25].

We extend the algorithm Rej to $\mathrm{Rej}(\mathbf{v}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M; \mathbf{r})$ to parse the randomness $\mathbf{r} \leftarrow \chi_{\mathbf{r}}$ as an input. We will also write $(\mathbf{z}|\mathrm{acc})_{\mathbf{v}}$ to denote the distribution of $\mathbf{z} := \mathbf{v} + \mathbf{r}$ conditioned on $b = 1$, and $(\mathbf{z}|\mathrm{rej})_{\mathbf{v}}$ to denote the distribution of $\mathbf{z} := \mathbf{v} + \mathbf{r}$ conditioned on $b = 0$ (note that in that case $\mathbf{z} = \perp$ but the distribution we are interested in is the one of $\mathbf{v} + \mathbf{r}$ so we abuse this notation).

Rejection sampling aims to map a distribution $\mathbf{v} + \chi_{\mathbf{r}}$ – that depends on the secret – to a distribution $\chi_{\mathbf{z}}$ that does not. We recall the following lemma that bounds the Rényi divergence between the real and ideal distributions above.

Lemma 2.2 (Lemma 4.1, [DFPS22]). *Assume that $M > 1$ and $\varepsilon < 1$ are such that $R^{\varepsilon}_{\infty}(\chi_{\mathbf{z}}||\mathbf{v}+\chi_{\mathbf{r}}) \leqslant M$, then, $R_{\infty}(\mathrm{Rej}||\mathrm{Ideal}) \leqslant 1 + \frac{\varepsilon}{M-1}$.*

2.7 Rejection sampling over hyperballs

Devevey et al. [DFPS22] showed that hyperballs-based rejection sampling achieves the most compact parameters and outperforms uniforms [Lyu09] and polytopes [BBRS24]. Hyperballs are as compact as using Gaussians but have the advantage of a simpler rejection condition in the form of a norm two-bound check.

Definition 2.3 (Continuous hyperball). *We denote:*

$$\mathcal{B}_{\mathcal{R},\ell}(r, \mathbf{c}) = \left\{\mathbf{x} \in \mathcal{R}^{\ell}_{\mathbb{R}} \mid \|x - \mathbf{c}\|_2 \leqslant r\right\}$$

the continuous hyperball with center $\mathbf{c} \in \mathcal{R}^{\ell}_{\mathbb{R}}$ and radius $r > 0$ in dimension $\ell > 0$. When $\mathbf{c} = 0$, we omit it.

We now recall useful definitions and lemmas to bound the smooth Rényi divergence of hyperball-based rejection sampling.

Definition 2.4 (Regularized Incomplete Beta Function). *The incomplete beta function is defined over $[0, 1] \times \mathbb{R}^+ \times \mathbb{R}^+$ as $B : (x; a, b) \mapsto \int_0^x t^{a-1}(1-t)^{b-1}\partial t$. We also define $I_x(a, b) = B(x; a, b)/B(1; a, b)$.*

Lemma 2.3 ([DFPS22, Lemma 5.1]). *Let $\ell \geqslant 1$ and $\mathbf{v} \in \mathbb{R}^{n\ell}$. Let $\varepsilon \in [0, 1/2)$ and $\phi \geqslant 1$ be such that $2\varepsilon = I_{1-1/\phi^2}\left(\frac{n\ell+1}{2}, \frac{1}{2}\right)$. Let $r, r' > 0$ such that $r'^2 \geqslant r^2 + \|\mathbf{v}\|_2^2 + 2r\|\mathbf{v}\|_2/\phi$. Then it holds that:*

$$R^{\varepsilon}_{\infty}\left(\mathcal{U}(\mathcal{B}_{\mathcal{R},\ell}(r))||\mathcal{U}(\mathcal{B}_{\mathcal{R},\ell}(r', \mathbf{v}))\right) = \left(\frac{r'}{r}\right)^{n\ell} \tag{2}$$

Let $M > 1$. If $r \geqslant \|\mathbf{v}\|_2 \cdot \frac{\frac{1}{\phi} + \sqrt{\frac{1}{\phi^2} + M^{2/(n\ell)} - 1}}{M^{2/(n\ell)} - 1}$ and $r' = M^{1/(n\ell)} r$, then the value in Eq. (2) is upper-bounded by M.

Lemma 2.4 (from [DFPS22, Section A.6]). *For $n > 1, \phi > 1$, we have $I_{1-1/\phi^2}(\frac{n+1}{2}, \frac{1}{2}) < \left(1 - \frac{1}{\phi^2}\right)^{n-1} \cdot n \cdot \left(1 - \frac{1}{\phi}\right)$.*

2.8 Short secret sharings

Secret sharings are useful primitives for designing threshold schemes. Both of our constructions rely on the *short secret sharing* notion introduced by del Pino et al. [dPENP25], which is particularly fit for lattice-based constructions. Precise definitions are recalled in Section A.3.

These secret sharings are special instances of Linear Secret Sharing Schemes (LSSS) that additionally guarantee shares with small norm and small reconstruction coefficients. These properties benefit the design of lattice-based schemes, which will leverage short terms to sample securely or verify signatures. While these constraints inherently partially leak the secret, their core observation was that these schemes often do not need perfect secrecy, and it suffices that the public key $\mathsf{vk} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$ remains pseudo-uniform, even after up to $T - 1$ shares are corrupted. It notably supports that corrupted shares reveal an offset of the secret key or hints based on the Hint-MLWE assumption.

3 Threshold Raccoon Revisited: T-Raccoon

Several lattice-based threshold signatures derive from the Raccoon signature. Most threshold variants of Raccoon rely on Shamir sharing for the signing key and additive secret sharing for the randomness [DKM+24, KRT24, EKT24, BKL+24], but this makes identifying aborts challenging due to the evaluation of PRFs on private inputs to mask partial signatures. While [dPKN+25] proposed a trick to use NIZK and avoid proving the PRF computation, it remains expensive to identify aborts in practice. del Pino et al. [dPENP25] proposed another approach based on short secret sharing to avoid the need of PRFs in Raccoon entirely and achieves identifiable aborts for free using Raccoon verification over the – now accessible – partial signatures. However, their construction has low scalability, allowing for only $N \leqslant 16$ for a perfect threshold or requiring ramp sharing with a gap of at least 3. Here, we introduce a new short secret-sharing method that efficiently scales up to $N = 64$, providing practical advanced threshold properties in T-Raccoon even for higher thresholds.

Our scheme builds on the scheme from [dPENP25], adapting it to accommodate the short secret sharing technique described in Section 3.1. The scheme is a simplified version of the *Threshold Raccoon* scheme [DKM+24], structured as a flooded Fiat-Shamir signature without aborts. Its security relies on both the MSIS and Hint-MLWE problems. For simplicity, we assume that each party receives only one share $\mathbf{s}_i$ instead of a full pool of shares S_i.

The scheme follows the usual Fiat-Shamir structure, where the secret keys are shared—for instance, using a trusted dealer or a Distributed Key Generation (DKG, see Section B.2). The shares $\mathbf{s}_i$ correspond to partial secret/public key pairs, concealed in a MLWE form: $\mathsf{vk}_i := \mathbf{A} \cdot \mathbf{s}_i + \mathbf{e}_i$, where $\mathbf{A} \in \mathcal{R}_q^{k \times \ell}$ is a random matrix.

Given a set of signers act, with their corresponding reconstruction coefficients $(\nu_i)_i$, the keys are linearly aggregated to compute the signing and verification keys: $(\mathbf{s}, \mathbf{e}, \mathsf{vk}) = \sum_{i \in \mathsf{act}} \nu_i \cdot (\mathbf{s}_i, \mathbf{e}_i, \mathsf{vk}_i)$. To sign, parties perform the following:

1. **First round:** Generate individual commitments $\mathbf{w}_i := \mathbf{A} \cdot \mathbf{r}_i + \mathbf{e}'_i$ in parallel, and commit to them publicly.
2. **Second round:** Broadcast the values to aggregate all individual commitments into a single commitment value $\mathbf{w} = \sum_{i \in S} \nu_i \cdot \mathbf{w}_i$.
3. **Third round:** Generate and broadcast individual responses $(\mathbf{z}_i, \mathbf{y}_i) := c \cdot (\mathbf{s}_i, \mathbf{e}_i) + (\mathbf{r}_i, \mathbf{e}'_i)$, where $c = \mathsf{H}_c(\mathsf{vk}, \mathsf{msg}, \mathbf{w})$.

The combiner sums the individual responses (weighted by the reconstruction coefficients v_i) to produce the signature $(c, \mathbf{z})$. Due to the linearity of all operations performed, the verification process is almost the same as for a standard Fiat-Shamir signature by: (i.) checking that $\mathbf{z}$ is small, and (ii.) verifying that $H(\mathsf{vk}, \mathsf{msg}, \mathbf{w}) = \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$.

As in the original scheme of [dPENP25], the key observation to perform identifiable abort is that we can perform partial verification of the shares using the partial public keys derived from the secret shares.

Table 1: Parameters used in our schemes.

Common parameters	
$\mathcal{R}_q$	Polynomial ring $\mathcal{R}_q = \mathbb{Z}[X]/(q, X^n + 1)$
(k, ℓ)	Dimension of the public matrix $\mathbf{A} \in \mathcal{R}_q^{k \times \ell}$
τ	Number of $\pm$ 1's in challenge c
T-Raccoon	
$\chi, \chi_{\mathbf{r}}$	Gaussian distributions for the secret and its shares, and randomness
$\nu_{\mathbf{t}}, \nu_{\mathbf{w}}$	Amount of bits dropped from the public key $\mathbf{t}$ and commitment $\mathbf{w}$
$B_2, B_2^{\mathsf{ind}}, J_{\max}$	Verification, and partial verification bounds
ML-DSA	
d	Amount of bits dropped from the public key $\mathbf{t}$
η	Secret key range: $\chi_{\mathbf{s}} = \mathcal{U}([-\eta, \eta]^{n(\ell+k)})$
$(\gamma_1, \gamma_2, \beta)$	Ranges for the signature generation and verification ($\beta = \tau \cdot \eta$)
ω	Number of 1's in the hint h
Threshold ML-DSA	
K	Number of parallel protocol repetitions
ν	Expansion factor for the first ℓ coordinates of the randomness
r'	Randomness ball rad. $\chi_{\mathbf{r}} = \left\{ \lfloor (\nu \mathbf{x}_1, \mathbf{x}_2) \rceil \mid (\mathbf{x}_1, \mathbf{x}_2) \xleftarrow{\$} \mathcal{B}_{\mathcal{R}, \ell+k}(r') \right\}$
r	Target ball rad. $\chi_{\mathbf{z}} = \left\{ \lfloor (\nu \mathbf{x}_1, \mathbf{x}_2) \rceil \mid (\mathbf{x}_1, \mathbf{x}_2) \xleftarrow{\$} \mathcal{B}_{\mathcal{R}, \ell+k}(r) \right\}$
M	Rejection parameter (per party) $M = \left(\frac{r'}{r}\right)^{n\ell}$

The formal description of the scheme, including the six algorithms (three rounds of signing, combine, and partial verification), is provided in Section B.1. Parameters are overviewed in Table 1.

3.1 Secret Sharing

We recall the secret sharing scheme from Desmedt, Di Crescenzo, and Burmester [DDB95]. It is essentially an algorithmic interpretation of Vandermonde's identity, which states that for $0 \leqslant b \leqslant N$:

$$\binom{N}{T} = \sum_{k=0}^{T} \binom{b}{k} \cdot \binom{N-b}{T-k} \tag{3}$$

The secret sharing from [DDB95] applies Eq. (3) recursively, taking $b = \lfloor N/2 \rfloor$ in order to minimize complexity. The scheme is described in [DDB95] and has recently been revisited by Battagliola et al. [BBC+25]. The initial purpose was achieving a multiplicative secret sharing of an element in a (non-abelian) group, thus with strong limitations on the operations allowed. A formal description of our adaptation as an additive sharing is given in Algs. 1 and 2. In addition, the sharing coefficients have a small norm (see Lemma 3.1), the reconstruction coefficients are in $\{0, 1\}$ (terminology from [CATZ24]). Thus, the sharing satisfies the requirements from Section 2.8. In Section B.1, we show how to embed the sharing in Keygen (Figure 9) for T-Raccoon.

Remark 1. Thanks to the scheme's recursive definition, we can "hardcode" different (optional) sharing strategies for specific pairs of T, N to improve the scheme efficiency. We provide two examples of these strategies for $T = 2$ and $T = N$ in Section B.3.

VandShare$(x, \mathcal{P}, T, \mathsf{idx} = \varnothing) \to \mathsf{Dict}$

$N = |\mathcal{P}|$
if $T = 1$ **then**
  return $\mathsf{Dict} := \{user : \{\mathsf{idx} : x\} \mid user \in \mathcal{P}\}$
else if $T = N$ **then** ▷ Optional, see Section B.3
  return $\mathsf{Dict} :=$ VandShare$_{\mathsf{N}}(x, \mathcal{P}, \mathsf{idx})$
else if $T = 2$ **then** ▷ Optional, see Section B.3
  return $\mathsf{Dict} :=$ VandShare$_2(x, \mathcal{P}, \mathsf{idx})$
else
  $\mathsf{Dict} = \{user : \{:\} \mid user \in \mathcal{P}\}$. $b = \lfloor N/2 \rfloor$
  Parse $\mathcal{P} = \mathcal{P}_L \sqcup \mathcal{P}_R$, with $\mathcal{P}_L$ the b smallest elements of $\mathcal{P}$
  for $k = \max(0, T - N + b), \ldots, \min(b, T)$ **do**
    $\mathsf{idx}_L := \mathsf{idx} \parallel$ `":L:"` $\parallel k$, $\mathsf{idx}_R := \mathsf{idx} \parallel$ `":R:"` $\parallel (T - k)$
    if $k = 0$ **then**
      $\mathsf{Dict} := \mathsf{Dict} \cup$ VandShare$(x, \mathcal{P}_R, T, \mathsf{idx}_R)$
    else if $k = T$ **then**
      $\mathsf{Dict} := \mathsf{Dict} \cup$ VandShare$(x, \mathcal{P}_L, T, \mathsf{idx}_L)$
    else
      $x_0 \leftarrow \chi$ ▷ This guarantees short shares
      $x_1 := (x - x_0) \bmod q$
      $\mathsf{Dict}_L :=$ VandShare$(x_0, \mathcal{P}_L, k, \mathsf{idx}_L)$
      $\mathsf{Dict}_R :=$ VandShare$(x_1, \mathcal{P}_R, T - k, \mathsf{idx}_R)$
      $\mathsf{Dict} := \mathsf{Dict} \cup \mathsf{Dict}_L \cup \mathsf{Dict}_R$
  return Dict ▷ The values of Dict are also dictionaries.

VandRecover$(\mathcal{P}, \mathsf{act}, \mathsf{idx} = \varnothing) \to \mathsf{Dict}$

$N = |\mathcal{P}|, T = |\mathsf{act}|$
if $T = 1$ **then**
  return $\mathsf{Dict} := \{user : \mathsf{idx} \mid user \in \mathcal{P}\}$
else if $T = N$ **then** ▷ Optional, see Section B.3
  return $\mathsf{Dict} :=$ VandRecover$_{\mathsf{N}}(\mathcal{P}, \mathsf{act}, \mathsf{idx})$
else if $T = 2$ **then** ▷ Optional, see Section B.3
  return $\mathsf{Dict} :=$ VandRecover$_2(\mathcal{P}, \mathsf{act}, \mathsf{idx})$
else
  $b = \lfloor N/2 \rfloor$
  Parse $\mathcal{P} = \mathcal{P}_L \sqcup \mathcal{P}_R$, with $\mathcal{P}_L$ the b smallest elements of $\mathcal{P}$
  $k = |\mathcal{P}_L|, \mathsf{act}_L = \mathsf{act} \cap \mathcal{P}_L, \mathsf{act}_R = \mathsf{act} \cap \mathcal{P}_R$
  $\mathsf{idx}_L := \mathsf{idx} \parallel$ `":L:"` $\parallel k$, $\mathsf{idx}_R := \mathsf{idx} \parallel$ `":R:"` $\parallel (T - k)$
  if $k = 0$ **then**
    return VandRecover$(\mathcal{P}_R, \mathsf{act}_R, \mathsf{idx}_R)$
  else if $k = T$ **then**
    return VandRecover$(\mathcal{P}_L, \mathsf{act}_L, \mathsf{idx}_L)$
  else
    $\mathsf{Dict}_L :=$ VandRecover$(\mathcal{P}_L, \mathsf{act}_L, \mathsf{idx}_L)$
    $\mathsf{Dict}_R :=$ VandRecover$(\mathcal{P}_R, \mathsf{act}_R, \mathsf{idx}_R)$
    return $\mathsf{Dict} := \mathsf{Dict}_L \sqcup \mathsf{Dict}_R$

3.1.1 Performances and comparison with other sharing schemes

In [BBC+25], the authors provide estimates on the total and per party number of shares, showing a superpolynomial growth in T and $\log N$. However, Figure 2 shows that the scheme remains practical for

all $T \leqslant N \leqslant 64$, with the individual number of shares staying below 4500. Note that only one share per user is used during the distributed signing since all coefficients are zero but one. Experimental results from our implementation corroborate these claims.

Other schemes with similar features have been proposed; however, for the range, we consider the scheme from [DDB95] as the most efficient. In the range of interest ($N \leqslant 64$), it outperforms:

- The Replicated Secret Sharing approach (see Figure 2).
- The generic construction from [BL90] to Boppana's monotone formula [Bop85] (see also [DT06]), used in [CATZ24], in which the number of shares grow as $O(\min(T, N-T)^{4.3} N \log N)$.

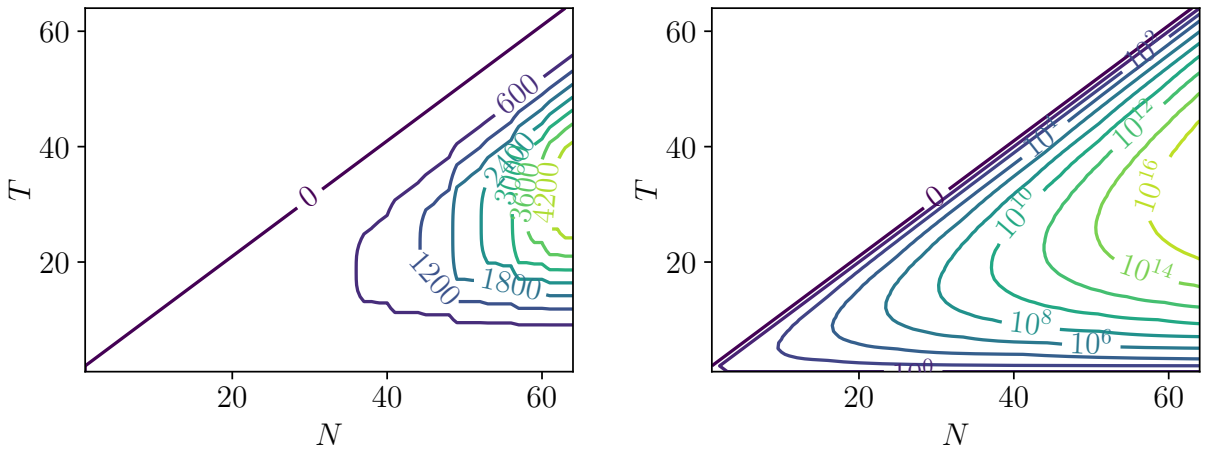
"Near-threshold" or "ramp" secret sharing schemes are sharings for which the privacy threshold T_p is different from the reconstruction threshold T_r. For a large number of parties, these schemes can provide better efficiency than the perfect threshold schemes. Some of these schemes have the potential to be used in our setting, but, to our knowledge, none of them is fully compatible or outperforms the scheme previously proposed:

- In [dPENP25], a ramp secret sharing scheme based on the Coupon Collector Problem is considered; however, the ratio T_r/T_p is quite large. For n such that $T_p = \Theta(n \log n)$, we have:

$$\frac{T_r}{T_p} \approx 1 + O\left(\frac{\kappa/m + \log(\kappa/p)}{\log n}\right). \tag{4}$$

  Setting $m = \omega(\kappa \log n)$ and $p = \omega(\kappa/n)$ gives a ratio $\frac{T_r}{T_p} = 1 + o(1)$ at the cost of $m \cdot p$ shares per party. This gap remains large for concrete instantiations [dPENP25, Figure 8]. In addition, in the adaptive setting (as opposed to static), the privacy threshold gets downgraded to $T_p = n - 1$.

- In [ANP23], Applebaum et al. show how to instantiate a ramp secret achieving any gap T_r/T_p arbitrarily close to 1. Moreover, the sharing and reconstruction can be performed with $O(N)$ additions. However, to our knowledge, there is no way to simulate the shares using hints from Hint-MLWE. Hence, it can't be used in our setting (an equivalent privacy related problem with this sharing is noted in [CATZ24]).



(a) Vandermonde

(b) Replicated

Figure 2: Contour plots of the maximum number of shares per party, as a function of N and T (undefined for $T > N$).

3.1.2 Security analysis

We prove that T-Raccoon, implemented with the Vandermonde-based short secret sharing, is unforgeable.

Theorem 3.1. *Let* $B_{\mathsf{stmsis}} = B_2 + \sqrt{\tau} + (\tau \cdot 2^{\nu_{\mathbf{t}}} + 2^{\nu_{\mathbf{w}}+1}) \cdot \sqrt{nk}$. *T-Raccoon is* TS-UF *secure up to* Q_s *queries to signing oracles if* $\mathsf{STMSIS}_{q,k,\ell+1,\mathcal{C},B_{\mathsf{stmsis}}}$ *and* $\mathsf{Hint\text{-}MLWE}_{q,k,\ell,\chi}$ *for* $Q_v(T,N)$ *hints of the form* $\mathbf{s}+\mathbf{s}'$, $\mathbf{s}' \leftarrow \chi$, *and* Q_s *hints of the form* $c \cdot \mathbf{s} + \mathbf{r}$, $c \leftarrow \mathcal{C}, \mathbf{r} \leftarrow \chi_{\mathbf{r}}$ *are hard, where* $Q_v(T,N)$ *is a function of* T, N.

We extend the results of [dPENP25], which only proved the unforgeability of T-Raccoon based on short secret sharing in a reliable broadcast communication model, while we assume an adversarially controlled channel. We defer the full proof to Section B.4 and provide an overview. We adapt the proof strategy of [dPENP25, dPN25] and capture all usages of the secret key as hints. Under the Hint-MLWE assumption, we replace the public key with an independent uniform sample. Finally, we reduce forging signatures to the STMSIS problem.

When using our Vandermonde sharing, the view of the adversary depends on the secret key through two components: (1) the corrupted and leaked shares to the adversary, and (2) the honest partial signatures:

1. First, we show that shares leaked to the adversary do not fully leak the secret. At every step of VandShare, we derive new shares by adding a Gaussian noise from χ to a secret dependent value, we can express all these shares using hints on the secret of the form $\mathbf{s}+\mathbf{s}'$ with $\mathbf{s}' \leftarrow \chi$.

2. Second, we need to evaluate the leakage induced by producing partial signatures of the form $c \cdot \mathsf{sk}_i[\mathsf{idx}] + \mathbf{r}_i$. For this, we prove that our sharing has the nice property that we can safely leak enough of the shares such that either $\mathsf{sk}_i[\mathsf{idx}]$ or $\mathbf{s} - \mathsf{sk}_i[\mathsf{idx}]$ can be computed from leaked shares. In the first case, we directly compute the hint on the partial secret, while in the second case, we derive it from a hint on $\mathbf{s}$: $c \cdot (\mathbf{s}+\mathbf{r}) - (\mathbf{s} - \mathsf{sk}_i[\mathsf{idx}])$, $\mathbf{r} \leftarrow \chi_{\mathbf{r}}$.

We also show that our scheme guarantees identifiable aborts (which in turn implies correctness). We rely on [dPENP25, Thm. 6], which generically studied this question for short secret sharings and ensures it under the following conditions.

Theorem 3.2. *Let* μ *be an overwhelming bound on our Vandermonde secret sharing norm. By the correctness of* VandRecover *and having exactly one reconstruction coefficient equal to 1 per party, [dPENP25, Thm. 6] states that T-Raccoon properly identifies abort if:*

- $B_2^{\mathsf{ind}} = \tau \cdot \mu + \alpha\sqrt{n(k+\ell)} \cdot \sigma_{\mathbf{r}}$
- $J_{\max} = \tau \cdot \mu + \sigma_{\mathbf{r}} \cdot \sqrt{2(\kappa+1)\ln 2}$
- $B_2 = B_{\mathsf{D}*} + (3 \cdot 2^{\nu_{\mathbf{w}}-1} + \tau \cdot 2^{\nu_{\mathbf{t}}}) \cdot \sqrt{nk}$, *where* $B_{\mathsf{D}*}$ *is defined by* $B_{\mathsf{D}*}^2 = \frac{T^2 J_{\max}^2}{2}\left(1 + \sqrt{1 + \frac{4(B_2^{\mathsf{ind}})^2}{J_{\max}^2 T}}\right) + (B_2^{\mathsf{ind}})^2 T$, *following the analysis of the Death star algorithm in [dPENP25] to detect aborts.*

where $(\alpha \geqslant 1 + \frac{\kappa \log 2}{3n})$ *or if* $(\alpha \geqslant 1 + \sqrt{\frac{4\kappa \log 2}{n}})$.

Lemma 3.1. *Assume* $\chi = D_{\mathcal{R}^{k+\ell},\sigma}$ *with* $\sigma \geqslant \sqrt{2} \cdot \eta'_{\varepsilon}(\mathbb{Z}^{n(k+\ell)})$ *for* $\varepsilon = \mathsf{negl}(\kappa)$, *then any share produced by our Vandermonde sharing has a norm bounded by* $\mu = \alpha\sigma\sqrt{n(k+\ell) \cdot T}$ *with overwhelming probability.*

The proof of this lemma is defered to Section B.6.

3.2 Optimized parameters

By a direct computation (see Section B.5 for the formula) we can verify that for $T \leqslant N \leqslant 64$ the number of hints $Q_v(T,N)$ in Theorem 3.1 is bounded by $30327 \approx 2^{15}$. From this, we can estimate the parameters of our scheme using the same methodology as in [dPENP25]: the unforgeability of our scheme relies on the STMSIS assumption, and on the functional simulatability of the underlying sharing, reducing to the Hint-MLWE assumption (and in turn to $\mathsf{MLWE}_{q,k,\ell,\sigma_{\mathsf{red}}}$). We quantify the hardness of $\mathsf{MLWE}_{q,k,\ell,\sigma_{\mathsf{red}}}$ using

Table 2: Parameter sets for T-Raccoon for security levels I and V. Sizes are given in kB.

Level	Q_s	q	n	k	ℓ	σ_{sk}	$\sigma_{\mathbf{r}}$	$\nu_{\mathbf{t}}$	$\nu_{\mathbf{w}}$	$\lvert\mathsf{vk}\rvert$	$\lvert\mathsf{sig}\rvert$
I	2^{59}	2^{49}	256	8	9	2^{14}	2^{31}	31	37	4.1	10.7
V	2^{60}	2^{53}	512	8	7	2^{14}	2^{37}	39	46	7.7	17.8

the open source Lattice Estimator [APS15]. As common in other works, see Dilithium [LDK+22, §C.3] and Raccoon [dEK+23, §4.3.5], we evaluate the hardness of STMSIS based on the best-known attacks. After some optimization work, we propose parameters in Table 2.

We recall that in the original proposal of [dPENP25], the scheme could *only accommodate up to 16 parties* for size *comparable* with this signature parametrization (10.3 kB —resp. 18.4 kB— in [dPENP25] vs. 10.7 kB —resp. 17.8 kB— for NIST-I —resp. NIST-V—).

4 Threshold ML-DSA

This section introduces our variant of ML-DSA (Module-Lattice-Based Digital Signature Algorithm) optimized for small threshold settings. It builds on Finally! [dPN25], a recent three-round threshold scheme offering significantly smaller signatures than earlier work. Yet, Finally! still lags behind ML-DSA 's compactness, with 2.7 kB signatures versus ML-DSA 's 2.4 kB for few parties.

Our first insight is that by reintroducing errors in the commitment $\mathbf{w}$ of ML-DSA, we can safely leverage the techniques of [dPN25] to reveal commitments without rounding, and even in case of signature rejection. With several tailored optimizations and careful parameter tuning, we show that this yields a *practical threshold ML-DSA*. Our construction securely distributes ML-DSA signing across parties and further improves [dPN25] by trading communication for higher signature capacity, enabling smaller parameters while staying ML-DSA-compatible. At a high level, our scheme samples several short secrets and derives a public key as the sum of their corresponding commitments. These secrets are shared among all parties such that any subset of at least T parties can reconstruct them. Signing proceeds via parallel rejection sampling performed independently by each party. While such parallelism could negatively affect the parameters, we mitigate this by employing tighter distributions for the rejection step, which is particularly effective in small-threshold regimes. Hence, our thresholdization exploits the non-tightness in ML-DSA's use of uniform sampling. We operate within the slack between rejection sampling over hyperballs (as noted in Section 2.7) and uniform distributions, allowing us to retain compact signature sizes even in the distributed setting. To balance a somewhat low success probability per iteration, we run $K > 1$ iterations of the scheme in parallel. That is, during the signing procedure, each party i produces K commitments $\mathbf{w}_{i,j}$, and a response $\mathbf{z}_{i,j}$ for each aggregated commitment $\sum_{i\in\mathsf{act}} \mathbf{w}_{i,j}$.

4.1 Construction

As defined in Section 2, threshold signature schemes primarily consist of key generation, signing, combining, and verifying procedures. To formalize these procedures for Threshold ML-DSA, we first outline the parameters used in the single-party ML-DSA scheme, along with the additional hyperball parameters $r_{N,T}$ and $r'_{N,T}$ required for our threshold variant. We collect all of them in Table 1.

Key Generation. The key generation procedure is detailed in Fig. 3. It assumes a trusted dealer to sample and distribute the shares. It builds upon the short replicated secret sharing (RSS) technique introduced in [dPENP25], and involves sampling $\binom{N}{N-T+1}$ ML-DSA secrets from $\chi_{\mathbf{s}}$. Each secret is then given to a distinct set I of $N-T+1$ non-corrupted user. Since there are at most $T-1$ corrupted parties, this ensures that at least one honest party receives each secret. The final public key is derived by summing all secrets and rounding the result as in ML-DSA, yielding a sample from the MLWE distribution. Thanks to at least one honest share per subset, the public key maintains pseudorandomness under the same MLWE assumption as ML-DSA.

Remark 2. Here, we do not adopt the more scalable secret sharing technique from Section 3, as the individual shares would have larger norms. While this is acceptable in T-Raccoon–where the share size only needs to be smaller than the overall randomness–it becomes critical in Threshold ML-DSA to keep share's norms minimal, so we reduce rejection sampling overhead.

$\mathsf{RSS}(N, T, \chi) \to (\mathbf{s}, \mathsf{sk} := (\mathsf{sk}_1, \ldots, \mathsf{sk}_N))$

1: **for** $I \subset [N]$ such that $|I| = N - T + 1$ **do**
2: $\quad \mathbf{s}_I \leftarrow \chi$
3: $\mathbf{s} := \sum_I \mathbf{s}_I$
4: **for** $i \in [N]$ **do** ▷ Distribute the shares
5: $\quad \mathsf{sk}_i := \{I : \mathbf{s}_I \mid I \subset [N] \text{ s.t. } i \in I \wedge |I| = N - T + 1\}$
6: **return** $(\mathbf{s}, \mathsf{sk} := (\mathsf{sk}_0, \cdots, \mathsf{sk}_{N-1}))$

$\mathsf{Keygen}(\kappa, N, T) \to (\mathsf{vk}, \mathsf{sk})$

1: $\rho \leftarrow \{0,1\}^{256}$
2: $\mathbf{A} := \mathsf{ExpandA}(\rho)$
3: $(\mathbf{s}, (\mathsf{sk}_i)_{i\in[N]}) \leftarrow \mathsf{RSS}(N, T, \chi_{\mathbf{s}})$
4: $\mathbf{t} := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{s}$
5: $(\mathbf{t}_\top, \mathbf{t}_\bot) := \mathsf{Power2Round}(\mathbf{t}, d)$
6: $tr \in \{0,1\}^{256} := H(\rho || \mathbf{t}_\top)$
7: **return** $(\mathsf{vk} := (\rho, \mathbf{t}_\top), \mathsf{sk} := (tr, \mathsf{sk}_i)_{i\in[N]})$

Figure 3: Key generation procedure of Threshold ML-DSA.

Signing. The signing procedure is detailed in Fig. 4. Any set act of T signers collectively knows all the secrets $\mathbf{s}_I$, since for any secret $\mathbf{s}_I$ only $N - (N - T + 1) = T - 1$ parties do not have said secret. The core idea is to decide on a partition $\bigsqcup_{i\in\mathsf{act}} \mathbf{m}_i = \{I\}_I$ of the secrets among the T parties using a deterministic function RSSRecover, and then run T Fiat-Shamir protocols in parallel to produce a partial signature for each partial secret $\mathbf{s}_i^{\mathsf{part}} = \sum_{I\in\mathbf{m}_i} \mathbf{s}_I$, i.e. partial commitments $\mathbf{w}_i = \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$, and signatures $\mathbf{z}_i = c \cdot \mathbf{s}_i^{\mathsf{part}} + \mathbf{r}_i$.

In particular, each party uses a full MLWE sample as commitment $\mathbf{w}_i$ (i.e. $\mathbf{w}_i = \mathbf{A}\mathbf{y} + \mathbf{e}$) instead of using $\mathbf{A} \cdot \mathbf{y}$ as in ML-DSA. This change allows for direct reveal of $\mathbf{w}_i$ before rounding, even in case of rejection. To ensure correctness, we verify that the sum of the responses $\mathbf{z} = \sum_{i\in\mathsf{act}} \mathbf{z}_i$ verifies the ML-DSA verification equation $\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{z} = c \cdot \mathbf{t} + \mathbf{w}$: the challenge c can be computed as $c = \mathsf{SampleInBall}(\tilde{c})$, where $\tilde{c} = \mathsf{H}(\mu || \mathsf{HighBits}(\mathbf{w}, 2\gamma_2))$ and $\mathbf{w} = \sum_{i\in\mathsf{act}} \mathbf{w}_i$.

Following prior works [DKM+24, dPN25], we design a three-round scheme that protects against rushing adversaries on the choice of $\mathbf{w}$:

1. **First round:** parties sample randomness $\mathbf{r} \leftarrow \chi_{\mathbf{r}}$ and compute the commitment $\mathbf{w}_i = \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$. They publish a hash of the commitment $\mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, i, \mathbf{w}_i)$.
2. **Second round:** parties reveal the values of $(\mathbf{w}_i)_{i\in\mathsf{act}}$.
3. **Third round:** Each party computes the response $\mathbf{z}_i = c \cdot \mathbf{s}_i^{\mathsf{part}} + \mathbf{r}_i$, where c is derived from the aggregated commitment $\mathbf{w} = \sum_{i\in\mathsf{act}} \mathbf{w}_i$ after proper rejection sampling.

In the complete scheme, we omit the second part $\mathbf{z}_i^{(2)} \in \mathcal{R}_q^k$ of the responses since the verifier can reconstruct it from public values. We are interested in their sum—retrieved from the public key—: the aggregated commitment $\mathbf{w}$ and the sum of the first part of the responses $\mathbf{z}^{(1)} = \sum_{i\in\mathsf{act}} \mathbf{z}_i^{(1)} \in \mathcal{R}_q^\ell$: $\sum_{i\in\mathsf{act}} \mathbf{z}_i^{(2)} = \mathbf{w} + c \cdot \mathbf{t} - \mathbf{A} \cdot \mathbf{z}^{(1)}$.

$\mathsf{RSSRecover}(\mathsf{act}) \to \mathcal{P}(S)^{\mathsf{act}}$

Compute partition $\mathbf{m} = (m_i)_{i \in \mathsf{act}}$ s.t. $\bigsqcup m_i = \{I \subset [N] \mid |I| = N - T + 1\}$ and minimizing $\max_{i \in \mathsf{act}} |m_i|$, detailed in Section C.1.
return $\mathbf{m}$

$\mathsf{ShareSign}_1(\mathsf{vk}, i, \mathsf{sk}_i, \mathsf{st}_i)$

$\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$
$\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$
$\mathsf{st}_i := \mathsf{st}_i \cup \{(\mathsf{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)\}$
$\mathsf{cmt}_i = \mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, i, \mathbf{w}_i) \in \{0,1\}^{2\kappa}$
return $(\mathsf{pm}_1^i := \mathsf{cmt}_i, \mathsf{st}_i)$

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

assert$\{$ $\mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}$ $\}$
assert$\{$ $(\mathsf{pm}_1^i, \cdot, \cdot) \in \mathsf{st}_i$ $\}$
Pick $(\mathsf{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)$ from st_i with $\mathsf{pm}_1^i = \mathsf{cmt}_i$
$\mathsf{st}_i := \mathsf{st}_i \backslash \{(\mathsf{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)\}$
$\mathsf{st}_i := \mathsf{st}_i \cup \{(\mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$
return $(\mathsf{pm}_2^i := \mathbf{w}_i, \mathsf{st}_i)$

$\mathsf{ShareSign}_3(\mathsf{vk}, i, \mathsf{pm}_2, i, \mathsf{sk}_i, \mathsf{st}_i)$

assert$\{$ $(\cdot, \mathsf{pm}_2^i, \cdot) \in \mathsf{st}_i$ $\}$
Pick $(\mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, \mathbf{w}_i, \mathbf{r}_i)$ from st_i with $\mathsf{pm}_2^i = \mathbf{w}_i$
Parse $\mathsf{pm}_1 = (\mathsf{cmt}_j)_j$ and $\mathsf{pm}_2 = (\mathbf{w}_j)_{j \neq i}$
assert$\{$ $\forall j \in \mathsf{act}, \mathsf{cmt}_j = \mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, j, \mathbf{w}_j)$ $\}$
$\mathsf{st}_i := \mathsf{st}_i \backslash \{(\mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$
$\mu \in \{0,1\}^{512} := H(tr || \mathsf{msg})$
$\mathbf{w} := \sum_{j \in \mathsf{act}} \mathbf{w}_i$
$\mathbf{w}_\top := \mathsf{HighBits}(\mathbf{w}, 2\gamma_2)$
$\tilde{c} \in \{0,1\}^{256} := \mathsf{H}(\mu || \mathbf{w}_\top)$
$c := \mathsf{SampleInBall}(\tilde{c})$ ▷ Global challenge
$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I \in m_i} \mathbf{s}_I$
$\mathbf{z}_i \leftarrow \mathrm{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M; \mathbf{r}_i)$ ▷ Individual response
if $\{$ $\mathbf{z}_i = \perp$ $\}$ **then** $\{$ **abort** $\}$
$\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)} := \mathbf{z} \in \mathcal{R}_q^\ell \times \mathcal{R}_q^k$
return $(\mathsf{pm}_3^i := \mathbf{z}_i^{(1)}, \mathsf{st}_i)$

Figure 4: Signing procedure in Threshold ML-DSA. When an assertion fails, the function returns $\perp$.

Combine and Verify. The combine and verify procedures are detailed in Fig. 5. Combine involves computing the hint $\mathbf{h}$ that allows verifiers to recover $\mathsf{HighBits}(\mathbf{w}, 2\gamma_2)$ from the value $\mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top$ used as an approximation of $\mathbf{w}$. Similarly to ML-DSA, we compute the difference $\boldsymbol{\delta} = \mathbf{w} - \left(\mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top\right)$ and use the function $\mathsf{MakeHint}$ with $\boldsymbol{\delta}$ to compute the hint. The high bits of $\mathbf{w}$ are correctly recovered as long as $\|\boldsymbol{\delta}\|_\infty \leqslant \gamma_2$; we thus restart the protocol if this is not the case. Verification proceeds just as in ML-DSA: the procedure also restarts if $\|\mathbf{z}^{(1)}\|_\infty \geqslant \gamma_1 - \beta$ or if $\mathbf{h}$ has more than ω 1's. While this check ensures security in ML-DSA, it is only used for correctness here. Indeed, the hiding property of the rejection step in $\mathsf{ShareSign}_3$ already ensures that $\mathbf{z}$ does not leak any information.

$\mathsf{Combine}(\mathsf{vk} = (\mathsf{seed}, \mathbf{t}_\top), \mathsf{act}, \mathsf{msg}, (\mathsf{pm}_k)_{k\in\{1,2,3\}})$

1: Parse $\mathsf{pm}_2 = (\mathbf{w}_i)_i$ and $\mathsf{pm}_3 = (\mathbf{z}_i^{(1)})_i$
2: $\mu \in \{0,1\}^{512} \coloneqq H(tr||\mathsf{msg})$
3: $\mathbf{w} \coloneqq \sum_{j\in\mathsf{act}} \mathbf{w}_i$ ▷ Aggregated commitment in $\mathcal{R}_q^k$
4: $\mathbf{w}_\top \coloneqq \mathsf{HighBits}(\mathbf{w}, 2\gamma_2)$
5: $\tilde{c} \coloneqq \mathsf{H}(\mu||\mathbf{w}_\top)$ ▷ $\tilde{c} \in \{0,1\}^{256}$
6: $c \coloneqq \mathsf{SampleInBall}(\tilde{c})$ ▷ Global challenge
7: $\mathbf{z}^{(1)} = \sum_{i\in\mathsf{act}} \mathbf{z}_i^{(1)}$
8: $\boldsymbol{\delta} \coloneqq \mathbf{w} - \left(\mathbf{A}\cdot\mathbf{z}^{(1)} - 2^d\cdot c\cdot\mathbf{t}_\top\right)$ ▷ Recovery error in $\mathcal{R}_q^k$
9: $\mathbf{h} \coloneqq \mathsf{MakeHint}_q(\boldsymbol{\delta}, \mathbf{A}\cdot\mathbf{z}^{(1)} - 2^d\cdot c\cdot\mathbf{t}_\top, 2\gamma_2)$
10: **if** $\left(\|\mathbf{z}^{(1)}\|_\infty \geqslant \gamma_1 - \beta\right)$ **or** $\left(\|\boldsymbol{\delta}\|_\infty > \gamma_2\right)$ **or** $\left(\|\mathbf{h}\|_1 > \omega\right)$ **then**
11: **abort**
12: **return** $\mathsf{sig} \coloneqq (\tilde{c}, \mathbf{z}^{(1)}, \mathbf{h})$

$\mathsf{Verify}(\mathsf{vk} \coloneqq (\rho, \mathbf{t}_\top), \mathsf{msg}, \mathsf{sig})$

1: Parse $\mathsf{sig} \coloneqq (\tilde{c}, \mathbf{z}^{(1)}, \mathbf{h})$
2: $\mathbf{A} \coloneqq \mathsf{ExpandA}(\rho)$
3: $\mu \in \{0,1\}^{512} \coloneqq H(tr||\mathsf{msg})$
4: $c \coloneqq \mathsf{SampleInBall}(\tilde{c})$
5: $\mathbf{w}'_\top \coloneqq \mathsf{UseHint}_q\left(\mathbf{h}, \mathbf{A}\cdot\mathbf{z}^{(1)} - 2^d\cdot c\cdot\mathbf{t}_\top, 2\gamma_2\right)$
6: **return** $\|\mathbf{z}^{(1)}\|_\infty < \gamma_1 - \beta$ **and** $\tilde{c} = \mathsf{H}(\mu||\mathbf{w}'_\top)$ **and** $\|\mathbf{h}\|_1 \leqslant \omega$

Figure 5: Combine and verification of Threshold ML-DSA.

4.2 Correctness and security

We prove that Threshold ML-DSA (i) is unforgeable, and (ii) p-terminates. For (i), Threshold ML-DSA is unforgeable if ML-DSA is unforgeable and the MLWE problem used in ML-DSA is hard. For (ii), we introduce a bound B such that for any $\mathbf{m}_i$ returned by RSSRecover and $(\mathbf{u}_1, \mathbf{u}_2) = \sum_{I\in\mathbf{m}_i} \mathbf{s}_I \in \mathcal{R}_q^{\ell+k}$, $\|(\frac{1}{\nu}\cdot c\cdot\mathbf{u}_1, c\cdot\mathbf{u}_2)\|_2 \leqslant B$ with overwhelming probability for a uniformly sampled challenge c and over the randomness of the key generation.

Let $\varepsilon > 0$. We assume that (r, r', ε) verify the conditions of Lemma 2.3 for secret vectors of norm B, i.e. $2\varepsilon = I_{1-1/\phi^2}\left(\frac{n(k+\ell)+1}{2}, \frac{1}{2}\right)$ for some ϕ, and $r'^2 \geqslant r^2 + B^2 + 2rB/\phi$, implying Lemma 4.1.

Lemma 4.1. *For the above constraints, we have for any $\|\mathbf{v}\|_2 \leqslant B$,*

$$R_\infty^\varepsilon\left(\chi_\mathbf{z} || \chi_\mathbf{r} + \mathbf{v}\right) = M$$

Proof. The equation checks out by applying Lemma 2.3 with the data processing inequality for Rényi divergence. □

Theorem 4.1. *Assume* $\Pr\left[\mathsf{HighBits}(\mathbf{w}, 2\gamma_2) = \mathsf{HighBits}(\mathbf{w}', 2\gamma_2)\right] = o(1)$, $\varepsilon = o(1)$. *Threshold ML-DSA p-terminates in the ROM for*

$$p = \mathbb{E}_{k\leftarrow\mathsf{Bin}\left(K, (1-\frac{1}{M})^T\right)}\left[v_k\right] + o(1)$$

where v_k, for $k \in [0, K]$, is the probability that in at least one of the k combinations we have $\|\mathbf{z}^{(1)}\|_\infty \leqslant \gamma_1 - \beta$ and $\|\boldsymbol{\delta}\|_\infty \leqslant \gamma_2$ and the number of 1's in $\mathbf{h}$ is less than ω, when $\mathbf{z}$ is sampled from $\sum_{i\in[T]} \chi_\mathbf{z}$ and $\boldsymbol{\delta} = c\cdot\mathbf{z}^{(2)} - c\cdot\mathbf{t}_\perp$.

Proof. First, observe that by design, any signature produced by the scheme is a valid ML-DSA signature. The signature necessarily (i) verifies $\|\mathbf{z}^{(1)}\| < \gamma_1 - \beta$, and (ii) $\mathbf{h}$ has at most ω 1's. Additionally, the scheme

ensures that $\boldsymbol{\delta} \coloneqq \mathbf{w} - \left(\mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top\right)$ has an infinity norm at most γ_2. By applying Lemma 2.1, we obtain $\mathsf{UseHint}_q\left(\mathbf{h}, \mathbf{A} \cdot \mathbf{z}^{(1)} - 2^d \cdot c \cdot \mathbf{t}_\top, 2\gamma_2\right) = \mathbf{w}_\top$ during verification. Thus, the verification bounds are satisfied, and the correct challenge c is successfully recovered.

It remains to ensure that the scheme succeeds with a probability of at least p. We prove this with a series of games starting from $\mathsf{Game}_0 \coloneqq \mathsf{Game}^{\mathsf{TS\text{-}corr}}_{\mathsf{TSS}}$. We denote $p_i = \Pr\left[\mathsf{Game}_i() \neq \bot\right]$.

Game_1. In this game, we assert that for any signing party and parallel session during the second round, we have $\|(\frac{1}{\nu} \cdot c \cdot \mathbf{s}_1^{\mathsf{part}}, c \cdot \mathbf{s}_2^{\mathsf{part}})\|_2 \leqslant B$, otherwise the game returns 0. We need to add at most $T \cdot K$ such assertions. As B is an overwhelming bound, we obtain by union-bound:

$$p_0 \geqslant p_1 - TK \cdot \mathsf{negl}(\kappa) = p_1 - \mathsf{negl}(\kappa)$$

Game_2. In this game, we ensure that the parallel signing sessions use different $\tilde{c}$ and $\mathbf{w}$ with different high bits. Otherwise, the game returns 0. It ensures that different challenges are effectively used in every session. As we have at most K independent sessions, we have from the union bound:

$$p_1 \geqslant p_2 - K^2 \cdot \left(\Pr\left[\mathsf{HighBits}(\mathbf{w}, 2\gamma_2) = \mathsf{HighBits}(\mathbf{w}', 2\gamma_2)\right] + 2^{-256}\right)$$

where $\mathbf{w}, \mathbf{w}'$ are sampled from $\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \sum_{i=1}^{T} \chi_{\mathbf{r}}$, and $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times \ell}$.

Game_3. In this game, we sample the challenge c of each session in advance, sample $\mathbf{z}_i \leftarrow \mathrm{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$ for each party. If $\mathbf{z}_i = \bot$, we instead sample $\mathbf{z}_i$ from the distribution $(\mathbf{z}|\mathrm{rej})_{\mathbf{v}=c\cdot\mathbf{s}_i^{\mathsf{part}}}$ of rejected $\mathbf{z}$. Then, we program the random oracle after computing $\mathbf{w} = \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{z} - c \cdot \mathbf{t}$'s, with $\mathbf{z} = \sum_{i \in \mathsf{act}} \mathbf{z}_i$ in each parallel session. The distribution of $\mathbf{w}$ is identical as in Game_2, and Game_2 ensured that we can program c without affecting the probability of winning as no two $\mathbf{w}$ have the same high bits, and then computing $\mathbf{z}$ first in this way is equivalent: $p_2 = p_3$.

Game_4. In this game, we sample the partial responses $\mathbf{z}_i$ using the Ideal rejection sampling from Fig. 1. Recall the Game_1 ensured that $c \cdot \mathbf{s}_i^{\mathsf{part}}$ has a twisted norm bounded by B. We can thus apply the result Lemma 2.2 on the Rényi divergence of rejection sampling, using the intermediate result on hyperballs of Lemma 2.3, since by assumption r, r', B verify its conditions. The Rényi divergence when replacing a single $\mathbf{z}_i$ is $1 + \frac{\varepsilon}{M-1}$.

Let E the event "$\forall (\mathbf{u}_1, \mathbf{u}_2) = \mathbf{s}^{\mathsf{part}}, \|(\frac{1}{\nu} \cdot c \cdot \mathbf{u}_1, c \cdot \mathbf{u}_2)\|_2 \leqslant B$". Formally, we consider the random variable $X = ((\mathsf{sk}_i)_{i \in [N]}, (\mathbf{z}_i^j)_{i \in \mathsf{act}, j \in [K]})$ conditioned on E, where the $\mathbf{z}_i^j$ are the responses output by the parties in each session. After conditioning on $(\mathsf{sk}_i)_{i \in [N]}$ we observe that the $\mathbf{z}_i^j$ become independent. We can thus apply the multiplicativity of the Rényi divergence (Lemma A.1) to bound the Rényi divergence between X in Game_3 and Game_4 by $\left(1 + \frac{\varepsilon}{M-1}\right)^{KT}$.

We then apply the data processing inequality and probability preservation of the Rényi divergence to obtain:

$$\Pr\left[\mathsf{Game}_4() = \bot \mid E\right] \cdot \left(1 + \frac{\varepsilon}{M-1}\right)^{KT} \geqslant \Pr\left[\mathsf{Game}_3() = \bot \mid E\right]$$

Noting that E has the same probability in both games, we obtain,

$$\Pr\left[\mathsf{Game}_4() = \bot\right] \cdot \left(1 + \frac{\varepsilon}{M-1}\right)^{KT} \geqslant \Pr\left[\mathsf{Game}_3() = \bot\right]$$

Finally, $p_3 \geqslant 1 + \left(1 + \frac{\varepsilon}{M-1}\right)^{KT} \cdot (p_4 - 1)$.

Conclusion. At this point, acceptance during the protocol occurs independently of the responses $\mathbf{z}_i$ with probability $1 - \frac{1}{M}$ for each party. Hence, each session succeeds during the protocol with probability $(1 - \frac{1}{M})^T$.

Additionally, we wish for the combination of responses to succeed. Note v_k for $k \in [0, K]$ the probability that in at least one of k sessions we have $\|\mathbf{z}\|_\infty \leqslant \gamma_1 - \beta$ and $\|\boldsymbol{\delta}\|_\infty \leqslant \gamma_2$ and #'s of 1's in $\mathbf{h}$ is less than ω, when $\mathbf{z}$ is sampled from $\sum_{i\in[T]} \chi_{\mathbf{z}}$.

Finally, we can combine the above results to evaluate the final success probability of the protocol: $p_4 = \mathbb{E}_{k\leftarrow \mathsf{Bin}\left(K,(1-\frac{1}{M})^T\right)}[v_k]$ □

For our parameter selection and analysis of our scheme's correctness, we introduce the following heuristic which assumes that the K parallel signing attempts are roughly independent to simplify the evaluation of the success probability of our scheme.

Heuristic 1. Heuristically, we consider that the rejections in Combine are roughly independent so that $p \approx 1 - \left(1 - \left(1 - \frac{1}{M}\right)^T \cdot v\right)^K$, where v is the probability that one combination succeeds.

In theory, these attempts are correlated by the dependency on the public key of the rejection of the hint at the end of the combination. However, the parameters of ML-DSA are selected such that this correlation is relatively small. Indeed, in ML-DSA there's a 1-2% chance of rejection due to an invalid hint, while the overall rejection rate is more than 75% [LDK+22, Section 3.4]. This is thus a natural simplification, which will yield very close estimates of the success probability.

We now state the unforgeability of Threshold ML-DSA.

Theorem 4.2 (Unforgeability). *For any parameter $\phi > 0$, let $Q_s = 2/(K \cdot I_{1-1/\phi^2}\left(\frac{n(k+\ell)+1}{2}, \frac{1}{2}\right))$.*

Our threshold signature scheme is TS-UF *secure in the ROM, for up to Q_s calls to the individual signing oracles, under the unforgeability of ML-DSA as well as the hardness of the* $\mathsf{MLWE}_{q,k,\ell,\chi}$ *assumptions for the distributions $\chi = \chi_{\mathbf{s}}, \chi_{\mathbf{r}}$ and $\chi_{\mathbf{z}}$.*

We deduce from Theorem 4.2 that breaking the unforgeability of Threshold ML-DSA is as hard as breaking ML-DSA. Indeed, the MLWE assumptions over $\chi_{\mathbf{r}}$ and $\chi_{\mathbf{z}}$ are almost statistically verified due to the large width of the hyperballs we use, and in particular are much harder than the assumptions underlying ML-DSA.

We provide below an overview of the proof and defer the formal statement and proof to Section C.3.

Our proof is very similar to the one of [dPN25, Theorem 4], except for two differences: (i) we update the proof to tackle the slightly different rounding of commitments $\mathbf{w}$, and (ii) instead of reducing to STMSIS [KLS18] at the end, we reduce to the unforgeability of ML-DSA. It proceeds with a sequence of games, where the first games in their proof allow them to move to a game the signing oracles return values independent of any secret material. Applying $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{s}}}$ twice, we then replace the public key with an ML-DSA public key, and as signing oracles do not leak the secret key, we can prove that a forgery in the last game directly provides a forgery for ML-DSA.

Game 2. In this game, we assert that an adversary cannot guess the value of honest commitments $\mathbf{w}_i$ before they are revealed in the second round. This is ensured by their high min-entropy. This allows the game to return a random hash in round 1, and lazily sample $\mathbf{w}_i$ later in the second round while programming the random oracle $\mathsf{H}_{\mathsf{cmt}}$ for consistency.

At a high-level, this game ensures that the adversary is unable to bias the aggregated commitment $\mathbf{w} = \sum_{i\in\mathsf{act}} \mathbf{w}_i$ as honest commitments will be chosen

Game 3-6. In these games, we sample the challenges c in advance during round 2. At a high-level, the pre-image and collision resistance of $\mathsf{H}_{\mathsf{cmt}}$ ensure that the adversary already decided on its commitments $\mathbf{w}_i$ before starting round 2. In particular, with the lazy sampling introduced in the previous game, honest commitments are sampled last. Hence, we can already compute the aggregated commitment $\mathbf{w} = \sum_i \mathbf{w}_i$ in round 2 when sampling the honest commitments $\mathbf{w}_i$, and as it has now a high min-entropy thanks to the honest commitments being sampled last, we can program the random oracle to return the challenge we just sampled without being detected by the adversary.

Game 7. In this game, we compute $\mathbf{z}_i$ first, even in case of rejection and then derive $\mathbf{w}_i = \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\mathsf{part}}$. That is, we first attempt to compute $\mathbf{z}_i = \mathrm{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$, and in case of rejection, we sample it from the distribution of rejection $\mathbf{z}_i$: $\mathbf{z}_i \leftarrow (\mathbf{z}|\mathrm{rej})_{v=c\cdot\mathbf{s}_i^{\mathsf{part}}}$. This is perfectly equivalent to the previous game, and just consists in rewriting distributions.

Game 8. This game ensures that the secret vector $c \cdot \mathbf{s}_i^{\mathsf{part}}$ has a norm smaller than B. This is true with overwhelming probability by assumption.

Game 9. In this game, we replace the commitments $\mathbf{w}_i$ by a uniform value when the corresponding $\mathbf{z}_i$ is rejected and will never be output. We show that this change is indistinguishable for an adversary if the assumptions $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}$ and $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{z}}}$ are hard. This is a direct application of the novel simulation results of [dPN25, Lemma 5].

Games 10-11. In these games, we remove the last dependencies on the secrets by replacing the sampling of $\mathbf{z}_i$ with rejection sampling by a sample obtained using the ideal functionality recalled in Fig. 1. We apply the Renyi divergence results of Section 2.7 to show that up to Q_s this increases the advantage of the adversary by at most 2 bits.

Games 12-13. In these games, we replace the public key of Threshold ML-DSA – which is composed of $\binom{N}{N-T+1}$, by an ML-DSA public key – containing a single secret. Since at this point the signing oracles do no longer depend on secret values, we can simply do so by applying twice the assumption $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{s}}}$.

Conclusion. At this point, any forgery output by the game is a valid forgery against an ML-DSA key, which allows to conclude the proof.

4.3 Parameter selection

To maintain backward compatibility with ML-DSA, we keep the same public parameters listed in Table 1, introducing only the new parameters K, r', r, and ν (respectively, the number of parallel repetitions, the respective radii of the randomness $\mathbf{r}'_i$ and rejected randomness $\mathbf{z}_i$, and the expansion factor for the first part of the rejected randomness $\mathbf{z}_i^{(1)}$). Following the security analysis in Section 4.2, we first choose ϕ such that $\varepsilon = I_{1-1/\phi^2}\left(\frac{n(k+\ell)+1}{2}, \frac{1}{2}\right)/2 \leqslant 1/(KQ_s)$. Then, given an upper bound B on the norm of partial secrets (holding with overwhelming probability), and choosing $r'^2 \geqslant r^2 + B^2 + \frac{2rB}{\phi}$, we ensure that Threshold ML-DSA retains the same level of security as ML-DSA for up to Q_s signing queries.

Concretely, we take $Q_s = 2^{50}$. We set $B = 1.3 \cdot \sqrt{\tau} \cdot \sqrt{n \cdot (k + \ell/\nu^2)} \cdot \sqrt{\mathsf{Var}(\mathcal{U}(-\eta, \eta))} \cdot \sqrt{\left\lceil \binom{N}{T-1}/T \right\rceil}$ as a heuristic overwhelming bound on the norm of partial secrets [1]. Moreover, RSSRecover ensures that each party uses at most $\left\lceil \binom{N}{T-1}/T \right\rceil$ secrets in a session for our selected parameters, c.f. Section C.1. Finally, we select K, r, r' for each possible pair (T, N), targeting a total success probability of at least $1/2$ for one protocol execution. We evaluate the success probability following Heuristic 1, evaluating v numerically.

We provide the communication cost for each threshold $2 \leqslant T \leqslant N \leqslant 6$ in Table 3. All security levels' full parameters and communication costs are given in Section C.2.

5 Performance Evaluation

We provide implementation details and experimental evaluation of the Threshold ML-DSA and T-Raccoon schemes.[2]

[1] We verify experimentally that $\|\mathbf{s}c\|$ behaves like a Gaussian distribution and take a bound $1.3 \cdot \|\mathbf{s}\| \cdot \|c\|$ corresponding to 13 standard deviations. If this heuristic were wrong, we would only incur a small loss in the number of queries since the bound on $\|\mathbf{s}c\|$ only affects the Rényi divergence of rejection sampling.

[2] Our implementations can be found at https://github.com/GuilhemN/threshold-ml-dsa-and-raccoon.

Table 3: Communication costs of Threshold ML-DSA-44 for $2 \leqslant T \leqslant N \leqslant 6$, aiming for a success probability 1/2. Full parameters are given in Section C.2.

N \ T	2	3	4	5	6
2	10.5 kB				
3	15.8 kB	21.0 kB			
4	15.8 kB	36.8 kB	42.0 kB		
5	15.8 kB	73.5 kB	157.4 kB	84.0 kB	
6	21.0 kB	99.8 kB	388.4 kB	524.8 kB	194.2 kB

Implementation We fully implemented both prototypes to ease integration with other systems and libraries. We chose `Golang` for its popularity in cryptographic implementations and low incidence of security issues [LJL+22, BSW24]. Our implementation of Threshold ML-DSA builds on the code of the CIRCL library [FHK19], while T-Raccoon uses the Lattigo library [lat24]. We extended CIRCL to support Threshold ML-DSA, including our secret-sharing functionality.

5.1 Experimental Analysis

We evaluated the performance of Threshold ML-DSA and T-Raccoon locally, as well as in LAN and WAN settings. All parties ran on a consumer-grade MacBook M3 for local and LAN experiments with 25 GB RAM. We conducted single-threaded local simulations of both schemes to benchmark key generation, signing (across parallel rounds), and verification. Each party's computation was emulated in a single process, capturing fine-grained timing data for each protocol phase. The runtime (i.e., computational cost) for various (T, N) values is shown in Table 4, for security level set to 44 for Threshold ML-DSA and I for T-Raccoon. Visualizations of bandwidth and runtime for Threshold ML-DSA-44 appear in Fig. 17, and runtime for T-Raccoon-I in Fig. 16; note that T-Raccoon 's signing bandwidth remains constant (≈ 35 kB) per party across (T, N).

For our WAN and LAN experiments, we implemented the communication layer of our schemes using `libp2p` [TSG21, TRT+22], a modular peer-to-peer networking stack. Each party acts as an independent `libp2p` peer, with its identity embedded as its network address, and communicates via a custom protocol over a direct TCP stream. Parties listen for incoming connections on a specified TCP port, and others connect using multiaddress. After establishing a connection, they exchange messages over a persistent bidirectional `libp2p` stream. This setup ensures execution over a realistic asynchronous, decentralized network, with `libp2p` handling connection management, peer identity, and stream multiplexing. NAT traversal and relaying were avoided for simplicity, assuming direct TCP connectivity.

LAN experiment We present our results in Table 5. Protocol instances run on separate machines over the same Wi-Fi network. Latency was estimated using an RTT over a TCP connection, yielding 124.166 µs.

WAN experiment We present our results in Table 6 for Threshold ML-DSA and T-Raccoon, both using a mesh network topology. The protocol ran on Amazon EC2 ``t2.small'' instances (Ubuntu 24.04.02, 2.0 GiB RAM), except for a MacOS M3 machine in Taipei, Taiwan, acting as "leader" and initiating connections. Other machines were distributed across Virginia (USA), London (UK), and Seoul (South Korea). Messages were sent via public IPs and in parallel according to best practices in distributed systems.

Latencies between machines are reported in Table 11, based on median round-trip measurements from [Clob] (consistent with latency variation over the Internet [HJAHB16]). WAN measurements reflect the slowest communication path (the "critical path" [ESTT22]), consistently between Taipei and Virginia, to capture worst-case delays. In real-world cloud deployments, infrastructure optimizations (e.g., load balancing, anycast routing) may reduce delays and overhead.

The measured signing latencies for both Threshold ML-DSA and T-Raccoon are within milliseconds, even in WAN setting over multiple continents, demonstrating our schemes' practicality in real-world distributed

Table 4: Local simulation for the average (mean) costs of Threshold ML-DSA-44 and T-Raccoon-I. All costs are computed on a MacOS M3 machine and grouped by N. Green and light green indicate the most and second-most optimal cases per group (N) and per scheme, where applicable.

(T, N)	**KeyGen** (ms)	**Sign+Combine** (ms)	**Verify** (ms)
Threshold ML-DSA			
(3,3)	0.3669	0.6810	
(2,4)	0.1709	0.4570	
(3,4)	0.2062	1.0961	
(4,4)	0.1655	1.3672	
(3,5)	0.2870	2.2263	0.0306
(4,5)	0.2940	4.9832	
(5,5)	0.1956	2.8453	
(4,6)	0.5016	12.1949	
(6,6)	0.2181	7.6784	
T-Raccoon			
(4,8)	4.5590	2.5090	
(4,16)	8.7690	1.8700	
(8,16)	20.4760	2.7770	
(16,16)	3.5500	5.0740	
(8,32)	134.8990	2.7780	0.6890
(16,32)	317.1920	4.7450	
(32,32)	4.8150	8.9030	
(32,64)	1118.7390	9.6540	
(64,64)	8.0430	16.4980	

Table 5: LAN measurements for Threshold ML-DSA-44 (M) and T-Raccoon-I (R). All costs are presented per party.

	M	R	M	R	M	R
(T, N)	**(2,6)**	**(2,8)**	**(4,6)**	**(4,16)**	**(6,6)**	**(8,32)**
Signing (ms)	0.548	3.586	18.216	1.947	8.683	3.486
TCP packets	≈15	≈25	≈269	≈25	≈137	≈25

systems. While both schemes require three rounds for a signing operation, these rounds are not executed sequentially in a blocking manner. Instead, messages are sent and processed in parallel across all parties. As a result, the observed latency reflects the most extended communication delay in each round's critical path—usually between the physically furthest machines—rather than the *sum* of point-to-point delays. Our costs are well under a second, far below a typical TLS handshake timeout of 10-20s [IBM23].

5.2 Applications and Discussion

Cryptocurrency wallets. One primary application of distributed cryptography (particularly threshold and multi-signature schemes) is *multi-device* cryptocurrency wallets [Eya21]. In these systems, the private key is split across multiple devices, which may include both the user's devices and servers operated by the wallet provider. In a threshold-based multi-device wallet, the private key is shared among N devices, and any T devices can collaborate to produce a signature to authorize actions such as transactions.

Despite their growing importance, research on multi-device wallets' practical and usability requirements is limited. Most studies focus on *single-device* wallets [FGA20, ECBS18, KJGW16]. Recent work [MDM+23] examined user perceptions of multi-device threshold wallets, revealing that users pre-

Table 6: WAN signing latency (in ms) for Threshold ML-DSA-44 and T-Raccoon-I across different topologies. L = London, S = Seoul, T = Taipei, V = Virginia.

Scheme	(T, N)	**Locations**	**Signing (ms)**
ML-DSA	(2,6)	T – S	27.34
ML-DSA	(2,6)	T – V	620.43
ML-DSA	(4,6)	T – V – L – L	750.65
ML-DSA	(6,6)	T – V – L – L – S – S	659.55
Raccoon	(2,8)	T – S	37.66
Raccoon	(2,8)	T – V	620.04
Raccoon	(4,16)	T – V – L – L	640.96
Raccoon	(6,32)	T – V – L – L – S – S	649.96

fer fewer, highly reputable parties to hold key shares and favor a lower threshold T for higher availability; this suggests that users prioritize reliability and reduced communication overhead over the security benefits of a larger N. Nonetheless, studies [YSDW24, AVBB21] show that users perceive multi-device wallets as more secure, making our schemes ideal for practical applications, as they perform best with lower (T, N). Research targeting cryptocurrency wallets has highlighted the need for diverse properties in threshold signature schemes. Some works, like [CATZ24], emphasize partial non-interactivity, where certain signing rounds can be performed independently of the message, improving usability in constrained environments. However, non-interactive schemes often require maintaining a state (e.g., precomputed nonces), which can introduce security risks and increased storage. Stateless schemes [KOR23, FYZ+24] address this by eliminating the need for an intermediate state. Our schemes offer configurability, supporting message-independent rounds without requiring pre-processing, allowing deployments to balance interactivity and statefulness based on the context.

TLS and the PKI While the traditional TLS architecture assumes a direct connection between a client and a server, modern deployments often involve a Content Delivery Network (CDN) to improve performance or security (this is especially notable in TLS 1.3, as CDNs drove its centralized deployment [HHA+20]). For protocols like HTTP, integrating the CDN is relatively straightforward: the origin server (e.g., `example.com`) determines what content should be cached and serves it via a subdomain (e.g., `cdn.example.com`). DNS is then used to route requests to the nearest CDN edge server. However, with HTTPS, the situation is more complex. TLS is designed to establish a secure, authenticated channel between the client and the *origin* server. To support HTTPS in this context, the origin server and the CDN must establish a trust relationship that allows edge servers to terminate TLS connections on behalf of the origin. In the conventional approach, the CDN generates its key pair and obtains certificates for specific origin subdomains (e.g., `cdn.example.com`). This lets edge servers impersonate the origin server for TLS purposes, offloading all cryptographic operations. While convenient, this model has a serious drawback: compromising an edge server exposes the private key, allowing an attacker to impersonate the origin until the certificate is revoked or expires. CDNs introduced a model known as *Keyless SSL* [SS15] to address this risk. In this model, the edge servers never possess the private key, and, instead, TLS handshakes are proxied: the client and edge server establish the session keys, but all private key operations (such as signing or decryption) are forwarded securely to the origin server [BBF+17]. However, private key material or sensitive operations are still handled across multiple machines, even in this proxied model, introducing potential security risks. An alternative and increasingly attractive approach is to use threshold signing, in which only shares of the private key are distributed among multiple parties (the edge servers or servers controlled by the origin entity). In this setup, no single entity ever holds the whole key.

As noted by [Her23, CEN24, DKR23], deploying threshold signatures in this context demands both practicality and compatibility with standardized signature schemes. Similar considerations apply in post-quantum settings: any candidate scheme must be practical, standardized (or on a clear path to standardization), and admit efficient threshold variants. Our schemes fit perfectly in this setting as practical variants of the ML-DSA standardized signature scheme. However, the choice of threshold parameters (T, N) directly

impacts both the availability and security of the infrastructure. For example, in a 3-of-5 deployment, the system can tolerate the failure or disconnection of up to two edge machines while maintaining signing capability, a key requirement as edge servers may be taken offline or experience spikes. This parameterization also resists partial compromise: an attacker must gain control of at least three independent parties to forge a signature. Threshold signing thus enables deployments that are simultaneously distributed, fault-tolerant, and robust, which aligns with the operational dynamics of CDN infrastructure. For particularly sensitive deployments, such as election's traffic [Cloa], we consider T-Raccoon with threshold configurations with up to $N = 64$ parties ideal. In such settings, the key material can be fragmented across dozens of machines, often spanning geographic regions or jurisdictions. A threshold of, for instance, $T = 32$ ensures strong resistance against compromise while still tolerating partial outages or edge node restarts. Our schemes, hence, can be practically used in this setting as they are based on standardized algorithms and allow for bigger (T, N), respectively.

The Tor Network and Onion Services The Tor network serves millions of users each day to provide anonymity to its users from the websites they connect to and conceal what they are connecting to from their Internet service provider and any other intermediary in their path [TTP24]. To maintain security, the network needs to produce a fresh random value every day in such a way that it cannot be predicted or influenced by an attacker. This random number is generated every midnight by trusted directory authorities[3]. The technique currently used for this procedure, a distributed random generator scheme, is based on a commit-and-reveal technique. However, it is vulnerable to various attacks, as noted in the Tor security analysis [Pro24]. To address these concerns, [Hop14] proposed that directory authorities use a threshold signature scheme to sign the value H(time-period) in each consensus document (a document that contains information about all known Tor relays), binding it to a specific time period. In this approach, each authority generates a publicly verifiable signature share during voting. As long as at least $\lceil N/2 \rceil$ authorities are honest, the full signature can be reconstructed by each client. The approach has not yet been integrated into the network, primarily because there is no standardized, practical threshold algorithm. Our results suggest that lattice-based threshold schemes such as Threshold ML-DSA and T-Raccoon offer a compelling solution to this longstanding need. Both schemes are post-quantum secure and feature lightweight signing rounds with minimal computational overhead. While latency is a critical factor in the Tor network (where connections often experience delays up to 5× greater than direct Internet paths), the proposed threshold signing schemes would impact only the consensus document generation, which occurs approximately once per hour[4]. Moreover, communication is restricted to the roughly ten directory authorities[5], which are primarily located in North America and Europe. Our experimental results show that Threshold ML-DSA and T-Raccoon fit this model well: they are based on well-scrutinized cryptographic assumptions, achieve low signing latencies across WAN topologies, and exhibit excellent performance under realistic network conditions.

5.3 Experimental Comparisons

We conclude with a comparison with Ringtail [BKL+24], a 2-round competitor to T-Raccoon *without* identifiable aborts that performed the first evaluation of WAN deployments of lattice-based threshold signatures. We evaluated against its public implementation[6], in a MacOS M3 machine. For fairness, we report per-party runtimes and bandwidths at NIST Level I (matching Threshold ML-DSA-44 and T-Raccoon-I), averaging over 10 runs (Table 7). Although Ringtail includes WAN measurements, its model appears sequential, assuming parties send messages one after another. As previously discussed, this deviates from realistic distributed systems, where broadcasts are parallel and dominated by critical-path latency. Their reported latency of 1888.147 ms thus reflects a cumulative delay rather than actual signing time. A more realistic WAN evaluation of Ringtail is left to future work. Notably, Ringtail achieves a two-round protocol, unlike our three-round schemes. While this may improve responsiveness in constrained settings (e.g., mo-

[3] See https://spec.torproject.org/rend-spec/shared-random.html.
[4] https://support.torproject.org/glossary/consensus/
[5] In 2025: https://metrics.torproject.org/rs.html#search/flag:authority
[6] https://github.com/daryakaviani/ringtail

Table 7: Comparison of Threshold ML-DSA, T-Raccoon and Ringtail. The most efficient scheme is in green. The measurements of Threshold ML-DSA appear only for supported thresholds (T, N).

Scheme	(T, N)	**Runtime Signing** (ms)	**Bandwidth Signing** (kB)
Threshold ML-DSA	(2, 2)	0.564	10.5
Ringtail	(2, 2)	58.287	633.0
T-Raccoon	(2, 2)	1.787	35.0
Threshold ML-DSA	(4, 4)	1.367	42.0
Ringtail	(4, 4)	63.615	633.0
T-Raccoon	(4, 4)	2.106	35.0
Ringtail	(8, 8)	78.483	633.0
T-Raccoon	(8, 8)	3.132	35.0
Ringtail	(16, 16)	108.622	633.0
T-Raccoon	(16, 16)	5.136	35.0
Ringtail	(32, 32)	173.741	633.0
T-Raccoon	(32, 32)	9.119	35.0
Ringtail	(64, 64)	297.574	633.0
T-Raccoon	(64, 64)	18.019	35.0

bile), in well-provisioned systems, the extra round is often amortized and unlikely to affect performance significantly.

References

[ANP23] Benny Applebaum, Oded Nir, and Benny Pinkas. How to recover a secret with $o(n)$ additions. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 236–262. Springer, Cham, August 2023.

[APS15] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. Cryptology ePrint Archive, Report 2015/046, 2015.

[ASY22] Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *ICALP 2022*, volume 229 of *LIPIcs*, pages 8:1–8:20. Schloss Dagstuhl, July 2022.

[AVBB21] Svetlana Abramova, Artemij Voskobojnikov, Konstantin Beznosov, and Rainer Böhme. Bits under the mattress: Understanding different risk perceptions and security behaviors of crypto-asset users. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI '21, New York, NY, USA, 2021. Association for Computing Machinery.

[BBC+25] Michele Battagliola, Giacomo Borin, Giovanni Di Crescenzo, Alessio Meneghetti, and Edoardo Persichetti. Enhancing threshold group action signature schemes: Adaptive security and scalability improvements. *IACR Cryptol. ePrint Arch.*, page 85, 2025.

[BBF+17] Karthikeyan Bhargavan, Ioana Boureanu, Pierre-Alain Fouque, Cristina Onete, and Benjamin Richard. Content delivery over tls: a cryptographic analysis of keyless ssl. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 1–16, 2017.

[BBRS24] Henry Bambury, Hugo Beguinet, Thomas Ricosset, and Éric Sageloli. Polytopes in the fiat-shamir with aborts paradigm. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 339–372. Springer, Cham, August 2024.

[BKL+24] Cecilia Boschini, Darya Kaviani, Russell W. F. Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. Ringtail: Practical two-round threshold signatures from learning with errors. Cryptology ePrint Archive, Report 2024/1113, 2024.

[BL90] Josh Cohen Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In Shafi Goldwasser, editor, *CRYPTO'88*, volume 403 of *LNCS*, pages 27–35. Springer, New York, August 1990.

[BLL+15] Shi Bai, Adeline Langlois, Tancrède Lepoint, Damien Stehlé, and Ron Steinfeld. Improved security proofs in lattice-based cryptography: Using the Rényi divergence rather than the statistical distance. In Tetsu Iwata and Jung Hee Cheon, editors, *ASIACRYPT 2015, Part I*, volume 9452 of *LNCS*, pages 3–24. Springer, Berlin, Heidelberg, November / December 2015.

[Bop85] Ravi B. Boppana. Amplification of probabilistic Boolean formulas. In *26th FOCS*, pages 20–29. IEEE Computer Society Press, October 1985.

[BS23] Katharina Boudgoust and Peter Scholl. Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part I*, volume 14438 of *LNCS*, pages 371–404. Springer, Singapore, December 2023.

[BSW24] Jenny Blessing, Michael A. Specter, and Daniel J. Weitzner. Cryptography in the wild: An empirical analysis of vulnerabilities in cryptographic libraries. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, ASIA CCS '24, page 605–620, New York, NY, USA, 2024. Association for Computing Machinery.

[CATZ24] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Partially non-interactive two-round lattice-based threshold signatures. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 268–302. Springer, Singapore, December 2024.

[CCL+23] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Bandwidth-efficient threshold EC-DSA revisited: Online/offline extensions, identifiable aborts proactive and adaptive security. *Theor. Comput. Sci.*, 939:78–104, 2023.

[CEN24] Sofia Celi, Daniel Escudero, and Guilhem Niot. Share the MAYO: thresholdizing MAYO. Cryptology ePrint Archive, Paper 2024/1960, 2024.

[CGG+20] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1769–1787. ACM Press, November 2020.

[CKM23] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023.

[Cloa] Cloudflare. Helping keep elections safe and secure. Cloudflare website.

[Clob] Cloudping. Aws latency monitoring. Cloudping website.

[CS19] Daniele Cozzo and Nigel P. Smart. Sharing the LUOV: Threshold post-quantum signatures. In Martin Albrecht, editor, *17th IMA International Conference on Cryptography and Coding*, volume 11929 of *LNCS*, pages 128–153. Springer, Cham, December 2019.

[DDB95] Yvo Desmedt, Giovanni Di Crescenzo, and Mike Burmester. Multiplicative non-abelian sharing schemes and their application to threshold cryptography. In Josef Pieprzyk and Reihaneh Safavi-Naini, editors, *ASIACRYPT'94*, volume 917 of *LNCS*, pages 21–32. Springer, Berlin, Heidelberg, November / December 1995.

[dEK+23] Rafael del Pino, Thomas Espitau, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, Mélissa Rossi, and Markku-Juhani Saarinen. Raccoon. Technical report, National Institute of Standards and Technology, 2023. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[DFPS22] Julien Devevey, Omar Fawzi, Alain Passelègue, and Damien Stehlé. On rejection sampling in Lyubashevsky's signature scheme. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part IV*, volume 13794 of *LNCS*, pages 34–64. Springer, Cham, December 2022.

[DKLS25] Antonín Dufka, Semjon Kravtšenko, Peeter Laud, and Nikita Snetkov. Trilithium: Efficient and universally composable distributed ML-DSA signing. Cryptology ePrint Archive, Paper 2025/675, 2025.

[DKM+24] Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 219–248. Springer, Cham, May 2024.

[DKR23] Jack Doerner, Yashvanth Kondi, and Leah Namisa Rosenbloom. Sometimes you can't distribute random-oracle-based proofs. NIST presentation, 2023.

[DM20] Luca De Feo and Michael Meyer. Threshold schemes from isogeny assumptions. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 187–212. Springer, Cham, May 2020.

[dPENP25] Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Simple and efficient lattice threshold signatures with identifiable aborts. Cryptology ePrint Archive, Paper 2025/871, 2025.

[dPKN+25] Rafael del Pino, Shuichi Katsumata, Guilhem Niot, Michael Reichle, and Kaoru Takemure. Unmasking TRaccoon: A lattice-based threshold signature with an efficient identifiable abort protocol. Cryptology ePrint Archive, Paper 2025/849, 2025.

[dPN25] Rafael del Pino and Guilhem Niot. Finally! a compact lattice-based threshold signature. In Tibor Jager and Jiaxin Pan, editors, *Public-Key Cryptography – PKC 2025*, pages 169–199, Cham, 2025. Springer Nature Switzerland.

[DT06] Ivan Damgård and Rune Thorbek. Linear integer secret sharing and distributed exponentiation. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006*, volume 3958 of *LNCS*, pages 75–90. Springer, Berlin, Heidelberg, April 2006.

[ECBS18] Shayan Eskandari, Jeremy Clark, David Barrera, and Elizabeth Stobert. A first look at the usability of bitcoin key management. *CoRR*, abs/1802.04351, 2018.

[EKT24] Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. Two-round threshold signature from algebraic one-more learning with errors. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 387–424. Springer, Cham, August 2024.

[ENP24] Thomas Espitau, Guilhem Niot, and Thomas Prest. Flood and submerse: Distributed key generation and robust threshold signature from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 425–458. Springer, Cham, August 2024.

[ESTT22] Brian Eaton, Jeff Stewart, Jon Tedesco, and N. Cihan Tas. Distributed latency profiling through critical path tracing: Cpt can provide actionable and precise latency analysis. *Queue*, 20(1):40–79, March 2022.

[Eya21] Ittay Eyal. On cryptocurrency wallet design. Cryptology ePrint Archive, Report 2021/1522, 2021.

[FGA20] Michael Fröhlich, Felix Gutjahr, and Florian Alt. Don't lose your coin! investigating security practices of cryptocurrency users. In *Proceedings of the 2020 ACM Designing Interactive Systems Conference*, DIS '20, page 1751–1763, New York, NY, USA, 2020. Association for Computing Machinery.

[FHK19] Armando Faz-Hernandez and Kris Kwiatkowski. *Introducing CIRCL: An Advanced Cryptographic Library*. Cloudflare, June 2019. Available at https://github.com/cloudflare/circl. v1.6.0 Accessed Jan, 2025.

[FMT25] Marc Fischlin, Aikaterini Mitrokotsa, and Jenit Tomy. BUFFing threshold signature schemes. PKC, 2025.

[FYZ+24] Qi Feng, Kang Yang, Kaiyi Zhang, Xiao Wang, Yu Yu, Xiang Xie, and Debiao He. Stateless deterministic multi-party EdDSA signatures with low communication. Cryptology ePrint Archive, Report 2024/358, 2024.

[GHKR08] Rosario Gennaro, Shai Halevi, Hugo Krawczyk, and Tal Rabin. Threshold RSA for dynamic and ad-hoc groups. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 88–107. Springer, Berlin, Heidelberg, April 2008.

[GKS24] Kamil Doruk Gür, Jonathan Katz, and Tjerand Silde. Two-round threshold lattice-based signatures from threshold homomorphic encryption. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, pages 266–300. Springer, Cham, June 2024.

[Her23] Armando Faz Hernandez. Requirements for threshold tls. NIST presentation, 2023.

[HHA+20] Ralph Holz, Jens Hiller, Johanna Amann, Abbas Razaghpanah, Thomas Jost, Narseo Vallina-Rodriguez, and Oliver Hohlfeld. Tracking the deployment of tls 1.3 on the web: a story of experimentation and centralization. *SIGCOMM Comput. Commun. Rev.*, 50(3):3–15, July 2020.

[HJAHB16] Toke Høiland-Jørgensen, Bengt Ahlgren, Per Hurtig, and Anna Brunstrom. Measuring latency variation in the internet. In *Proceedings of the 12th International on Conference on Emerging Networking EXperiments and Technologies*, CoNEXT '16, page 473–480, New York, NY, USA, 2016. Association for Computing Machinery.

[Hop14] Nicholas Hopper. A threshold signature-based proposal for a shared rng. Tor Dev Mailing List, 2014.

[HPRR20] James Howe, Thomas Prest, Thomas Ricosset, and Mélissa Rossi. Isochronous gaussian sampling: From inception to implementation. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020*, pages 53–71. Springer, Cham, 2020.

[IBM23] IBM. Handshake timer, 2023. https://www.ibm.com/docs/en/zos/3.1.0?topic=considerations-handshake-timer. Accessed 06/02/24.

[KCLM22] Irakliy Khaburzaniya, Konstantinos Chalkias, Kevin Lewi, and Harjasleen Malvai. Aggregating and thresholdizing hash-based signatures using STARKs. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIACCS 22*, pages 393–407. ACM Press, May / June 2022.

[KG20] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O'Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020.

[KJGW16] Katharina Krombholz, Aljosha Judmayer, Matthias Gusenbauer, and Edgar R. Weippl. The other side of the coin: User experiences with bitcoin security and privacy. In Jens Grossklags and Bart Preneel, editors, *FC 2016*, volume 9603 of *LNCS*, pages 555–580. Springer, Berlin, Heidelberg, February 2016.

[KLS18] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 552–586. Springer, Cham, April / May 2018.

[KLSS23] Duhyeong Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Toward practical lattice-based proof of knowledge from hint-MLWE. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 549–580. Springer, Cham, August 2023.

[KOR23] Yashvanth Kondi, Claudio Orlandi, and Lawrence Roy. Two-round stateless deterministic two-party Schnorr signatures from pseudorandom correlation functions. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 646–677. Springer, Cham, August 2023.

[KRT24] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, August 2024.

[lat24] Lattigo v6. Online: https://github.com/tuneinsight/lattigo, August 2024. EPFL-LDS, Tune Insight SA.

[LDK+22] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

[LJL+22] Wenqing Li, Shijie Jia, Limin Liu, Fangyu Zheng, Yuan Ma, and Jingqiang Lin. Cryptogo: Automatic detection of go cryptographic api misuses. In *Proceedings of the 38th Annual Computer Security Applications Conference*, ACSAC '22, page 318–331, New York, NY, USA, 2022. Association for Computing Machinery.

[Lyu09] Vadim Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 598–616. Springer, Berlin, Heidelberg, December 2009.

[Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 738–755. Springer, Berlin, Heidelberg, April 2012.

[MDM+23] Easwar Vivek Mangipudi, Udit Desai, Mohsen Minaei, Mainack Mondal, and Aniket Kate. Uncovering impact of mental models towards adoption of multi-device crypto-wallets. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, CCS '23, page 3153–3167, New York, NY, USA, 2023. Association for Computing Machinery.

[Pei10] Chris Peikert. An efficient and parallel Gaussian sampler for lattices. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 80–97. Springer, Berlin, Heidelberg, August 2010.

[Pro24] The Tor Project. Security analysis. Tor Specifications, 2024.

[SS15] Douglas Stebila and Nick Sullivan. An Analysis of TLS Handshake Proxying . In *2015 IEEE Trustcom/BigDataSE/IPA*, pages 279–286, Los Alamitos, CA, USA, August 2015. IEEE Computer Society.

[TRT+22] Dennis Trautwein, Aravindh Raman, Gareth Tyson, Ignacio Castro, Will Scott, Moritz Schubotz, Bela Gipp, and Yiannis Psaras. Design and evaluation of ipfs: a storage layer for the decentralized web. In *Proceedings of the ACM SIGCOMM 2022 Conference*, SIGCOMM '22, page 739–752, New York, NY, USA, 2022. Association for Computing Machinery.

[TSG21] Dennis Trautwein, Moritz Schubotz, and Bela Gipp. Introducing peer copy - a fully decentralized peer-to-peer file transfer tool. In *2021 IFIP Networking Conference (IFIP Networking)*, pages 1–2, 2021.

[TTP24] Inc The Tor Project. Tor metrics, 2024. https://metrics.torproject.org/.

[YSDW24] Yaman Yu, Tanusree Sharma, Sauvik Das, and Yang Wang. "don't put all your eggs in one basket": How cryptocurrency users choose and secure their wallets. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, CHI '24, New York, NY, USA, 2024. Association for Computing Machinery.

A Additional definitions

A.1 On the Rényi Divergence

We define it as in [DFPS22].

Definition A.1. *Let $\mathcal{P}, \mathcal{Q}$ two distributions such that $\mathcal{P}$ is absolutely continuous with respect to $\mathcal{Q}$. The Rényi divergence of infinite order is* $R_\infty(\mathcal{P}||\mathcal{Q}) = \mathsf{ess\ sup} \frac{\partial \mathcal{P}}{\partial \mathcal{Q}}(x)$.

We additionally recall a relaxed version of the Rényi divergence from [DFPS22, Def. 2.1], termed the *smooth Rényi divergence*, where one can remove a few problematic points from the support, including those that may lie in $\mathsf{Supp}(\mathcal{P})\backslash\mathsf{Supp}(\mathcal{Q})$.

Definition A.2. *Let $\varepsilon > 0$. Let $\mathcal{P}, \mathcal{Q}$ two probability distributions such that $\int_{\mathsf{Supp}(\mathcal{Q})} \mathcal{P}(x)\partial\mu(x) \geqslant 1-\varepsilon$ for the measure μ. Their ε-smooth Rényi divergence is $R^{\varepsilon}_{\infty}(\mathcal{P}||\mathcal{Q}) = \inf\{M > 0 \mid \Pr_{x\leftarrow\mathcal{P}}(\mathcal{P}(x) \leqslant M\mathcal{Q}(x)) \geqslant 1-\varepsilon\}$.*

The Renyi divergence provides interesting cryptographic properties, as noted in [BLL+15, HPRR20].

Lemma A.1 ([BLL+15, Lemma 2.9] and [HPRR20, Prop. 4]). *For two distributions $\mathcal{P}, \mathcal{Q}$, the Renyi divergence satisfies the following properties:*

- ***Data processing inequality.*** *$R_{\infty}(\mathcal{P}^f||\mathcal{Q}^f) \leqslant R_{\infty}(\mathcal{P}||\mathcal{Q})$ for any function f, where $\mathcal{P}^f$ (resp. $\mathcal{Q}^f$) denotes the distribution of $f(y)$ induced by sampling $y \leftarrow \mathcal{P}$ (resp. $y \leftarrow \mathcal{Q}$).*
- ***Multiplicativity.*** *Assume that $\mathcal{P}$ and $\mathcal{Q}$ are the joint distributions of random variables $(x_i)_i$. Note $\mathcal{P}_{i,|x_{<i}=v}, \mathcal{Q}_{i,|x_{<i}=v}$, the conditional distributions of x_i given $x_{<i} = v_{<i}$. Assume $R_{\infty}(\mathcal{P}_{i,|x_{<i}=v_{<i}}||\mathcal{Q}_{i,|x_{<i}=v_{<i}}) \leqslant r_i$ for any possible $v_{<i}$. Then, $R_{\infty}(\mathcal{P}||\mathcal{Q}) \leqslant \prod_i r_i$.*
- ***Probability preservation.*** *for any event $E \subseteq \mathsf{Supp}(\mathcal{Q})$, we have $\mathcal{Q}(E) \geqslant \mathcal{P}(E)/R_{\infty}(\mathcal{P}||\mathcal{Q})$.*

A.2 Formal Security Definitions of Threshold Signatures

Definition A.3 (Unforgeability). *For a R-round TSS, the advantage of an adversary $\mathcal{A}$ (with oracle access to a random oracle H) against the unforgeability of TSS is defined as:*

$$\mathsf{Adv}^{\mathsf{TS\text{-}UF}}_{\mathsf{TSS},\mathcal{A}}(\kappa, N, T) \coloneqq \Pr\left[\mathsf{Game}^{\mathsf{TS\text{-}UF}}_{\mathsf{TSS},\mathcal{A}}(\kappa, N, T) = 1\right]$$

where $\mathsf{Game}^{\mathsf{TS\text{-}UF}}$ is described in Figure 6. We say that TSS is unforgeable in the random oracle model, if for all PPT adversary $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{TS\text{-}UF}}_{\mathsf{TSS},\mathcal{A}}(\kappa, N, T) \leqslant \mathsf{negl}(\kappa)$.

$\mathsf{Game}^{\mathsf{TS\text{-}UF}}_{\mathsf{TSS},\mathcal{A}}(\kappa, N, T)$

1: $Q_{\mathsf{msg}} \coloneqq \varnothing$
2: $\mathsf{pp} \leftarrow \mathsf{Setup}(\kappa, N, T)$
3: $\mathsf{corrupt}, \mathsf{st}_{\mathcal{A}} \leftarrow \mathcal{A}(\mathsf{pp})$
4: **assert**$\{\ \mathsf{corrupt} \subseteq [N]$ **and** $|\mathsf{corrupt}| < T\ \}$
5: $\mathsf{honest} \coloneqq [N]\backslash\mathsf{corrupt}$
6: $(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}) \leftarrow \mathsf{Sign.Keygen}(\mathsf{pp})$
7: **for** $\{\ i \in \mathsf{honest}\ \}$ **do** $\{\ \mathsf{st}_i \coloneqq \varnothing\ \}$
8: $(\mathsf{msg}, \mathsf{sig}) \leftarrow \mathcal{A}^{\mathsf{H},(\mathsf{OSign}_i(\cdot))_{i\in[R]}}(\mathsf{st}_{\mathcal{A}}, \quad \mathsf{vk}, (\mathsf{sk}_i)_{i\in\mathsf{corrupt}})$
9: **req** $\{\ \mathsf{msg} \notin Q_{\mathsf{msg}}\ \}$
10: **return** $\mathsf{Verify}(\mathsf{vk}, \mathsf{msg}, \mathsf{sig})$

$\mathsf{OSign}_r(\mathsf{act}, \mathsf{msg}, i, (\mathsf{pm}^j_{r-1})_{j\in\mathsf{act}})$

1: **req** $\{\ i \in [N] \wedge i \in \mathsf{act} \cap \mathsf{honest}\ \}$
2: $Q_{\mathsf{msg}} \coloneqq Q_{\mathsf{msg}} \cup \{\mathsf{msg}\}$
3: $(\mathsf{pm}^i_r, \mathsf{st}_i) \leftarrow \mathsf{ShareSign}_r(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, (\mathsf{pm}^j_{r-1})_{j\in\mathsf{act}}, i, \mathsf{sk}_i, \mathsf{st}_i)$
4: **return** pm^i_r

Figure 6: Unforgeability game for TSS, where H denotes the random oracle. We consider that the OSign_r oracles return $\perp$ if the corresponding $\mathsf{ShareSign}_r$ fails to output a protocol message. The game returns 0 if a requirement **req** fails. Finally, $\mathcal{A}$ wins if the game $\mathsf{TS\text{-}UF}$ returns 1, i.e. if it forged a new signature.

We also wish to ensure *correctness*, i.e. that a valid signature is output when signers behave honestly. We however tolerate a probability of abort to support the *Fiat-Shamir with Aborts* paradigm.

Definition A.4 (Correctness of TSS with aborts). *We say that a R-round* TSS *with aborts p-terminates if:*

$$\Pr\left[\mathsf{Game}^{\mathsf{TS\text{-}corr}}_{\mathsf{TSS}}(\kappa, N, T, \mathsf{msg}, \mathsf{act}) \neq \bot\right] \geqslant p \tag{5}$$

$$\Pr\left[\mathsf{Game}^{\mathsf{TS\text{-}corr}}_{\mathsf{TSS}}(\kappa, N, T, \mathsf{msg}, \mathsf{act}) = 0\right] = \mathsf{negl}(\kappa) \tag{6}$$

where $\mathsf{Game}^{\mathsf{TS\text{-}corr}}_{\mathsf{TSS}}$ *is defined in Figure 7.*

$\mathsf{Game}^{\mathsf{TS\text{-}corr}}_{\mathsf{TSS}}(\kappa, N, T, \mathsf{msg}, \mathsf{act})$

$\mathsf{pp} \leftarrow \mathsf{Setup}(\kappa, N, T)$
$(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}) \leftarrow \mathsf{Sign.Keygen}(\mathsf{pp})$
for $i \in [N]$ **do**
  $\mathsf{st}_i := \varnothing$
for $r \in [R]$ **do**
  for $i \in \mathsf{act}$ **do**
    $(\mathsf{pm}^i_r, \mathsf{st}_i) \leftarrow \mathsf{ShareSign}_r(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, (\mathsf{pm}^j_{r-1})_{j\in\mathsf{act}}, i, \mathsf{sk}_i, \mathsf{st}_i)$
if $\{ \exists i \in \mathsf{act}, \mathsf{pm}^i_R = \textbf{abort} \}$ **then** $\{$ **return** $\bot \}$
$\mathsf{sig} := \mathsf{Combine}(\mathsf{vk}, \mathsf{act}, (\mathsf{pm}^i_r)_{r\in[R], i\in\mathsf{act}})$
return $\mathsf{Verify}(\mathsf{vk}, \mathsf{msg}, \mathsf{sig})$

Figure 7: Correctness game for TSS with aborts.

A.3 Formal Notions of Short Secret Sharing

We recall formal definitions of the short secret sharing introduced in [dPENP25], and that we use to capture the security of our Vandermonde sharing and to prove the unforgeability of T-Raccoon from Section 3.

The core observation made in [dPENP25] is that lattice constructions often simply require that the public key $\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix}\cdot\mathbf{s}$ appears pseudo-uniform, and in particular, perfect secrecy is not needed. They capture this property by introducing the notion of *functional simulatability* for a (Distributed) Key Generation (KG), ensuring that the public key corresponding to a shared secret still appears pseudo-uniform even after leaking up to $T-1$ partial secret keys, and additionally leaking hints of the form $c \cdot \mathsf{sk}_i[\mathsf{idx}] + \mathbf{r}_i$ on the honest shares, capturing the production of partial signatures in Threshold Raccoon. We recall it below.

Definition A.5 ((Distributed) Key Generation). *A (Distributed) Key Generation scheme is defined for a reconstruction threshold $T \leqslant N$ as a tuple of algorithms* $(\mathsf{Keygen}, \mathsf{Recover}, \mathsf{Verify})$ *where:*

- $\mathsf{Keygen}(1^\kappa, N, T) \to (\mathsf{pk}, \mathsf{aux}, (\mathsf{sk}_i := (S_i, \cdot), \mathsf{status}_i)_{i\in[N]})$ *is either an interactive protocol between N participants or an algorithm executed by a trusted party producing:*
  - *A public key* pk.
  - *Auxiliary information* aux *that may be useful for a particular application.*
  - *A partial private key* sk_i *for each participant, containing of a pool of shares S_i. We consider that secrets are indexed and secret u of user i is accessed with $S_i[\mathsf{idx}]$. We may note $\mathsf{sk}_i[\mathsf{idx}]$ as an abuse of notations.*
  - *A termination status* status_i *for each user indicating whether the protocol succeeded for user i. It can be either* **abort** *(with a set of identified malicious parties) or* **accept**.
- $\mathsf{Recover}(\mathsf{pk}, \mathsf{aux}, \mathsf{act}) \to (\nu_{i,\mathsf{idx}})_{i\in\mathsf{act}, \mathsf{idx}\in[|S_i|]}$ *is the algorithm providing coefficients allowing the reconstruction of a common secret* $\mathsf{sk} = \sum_{i\in\mathsf{act}, \mathsf{idx}\in|S_i|} \nu_{i,\mathsf{idx}} \cdot S_i[\mathsf{idx}]$ *from individual shares for any given set* act *of at least T parties.*

- $\mathsf{Verify}(\mathsf{pk}, \mathsf{aux}, (\mathsf{sk}_i)_{i\in\mathsf{honest}}) \to \{0,1\}$ *verifies that the public key, auxiliary information and partial private keys produced by a* KG *are consistent.*

Definition A.6 (KG functional simulatability). *We say that a (distributed)* KG *is functionally simulatable for a target key distribution χ and a function $f : (\mathsf{pk}, \mathsf{aux}, (S_i)_{i\in\mathsf{honest}}, \mathsf{in}) \to V$ if there exist two functions* SimKeygen, SimF *such that any adversary $\mathcal{A}$ has advantages in the games $(\mathsf{Game}^b_{\mathsf{KG\text{-}sim}})_{b\in\{0,1\}}$ that are negligibly close for sets of up to $T-1$ corrupted users, where:*

- *A simulator* $\mathsf{SimKeygen} : (\mathcal{A}, \mathsf{corrupt}, \mathsf{pk}) \to (\mathsf{aux}, \mathsf{st}, \mathsf{status})$ *that simulates the* KG *execution for the adversary given a public key sampled from χ, auxiliary information, and computes bias information in* st *for the functional evaluation. It also simulates key generation aborts with* status*. Typically, the public key is sampled from a perfect uniform distribution, while the private shares are potentially biased by the adversary.*
- $\mathsf{SimF} : (\mathsf{pk}, \mathsf{aux}, \mathsf{st}, \mathsf{in}) \mapsto V$ *simulates the evaluation of the function f with public key information, the bias information from* st *and input* in*.*

In our case, we are interested in taking $\chi = \mathcal{U}(\mathcal{R}_q^k)$, partial information on the shares $\mathsf{aux} = (\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot S_i[\mathsf{idx}])_{i\in[N],\mathsf{idx}\in|S_i|}$, and $f = f_{\mathsf{hint}} : (\mathsf{pk}, \mathsf{aux}, (S_i)_{i\in\mathsf{honest}}, \mathsf{in} = \mathsf{act})$ a function producing a hint for partially reconstructed secrets: $h_i = c \cdot \sum_{\mathsf{idx}\in|S_i|} S_i[\mathsf{idx}] + r$ for $c \leftarrow \mathcal{C}$ and r a Gaussian noise.

$\mathsf{Game}^b_{\mathsf{KG\text{-}sim}}$ **for** $b \in \{0,1\}$

1: $(N, T, \mathsf{corrupt}) \leftarrow \mathcal{A}(1^\kappa)$ ▷ $\mathcal{A}$ chooses corrupted parties
2: **assert**$\{$ $\mathsf{corrupt} \subseteq [N] \wedge |\mathsf{corrupt}| \leqslant T_{\mathsf{corrupt}}$ $\}$
3: $\mathsf{honest} := [N] \backslash \mathsf{corrupt}$
4: **if** $b = 0$ **then** ▷ Real sharing
5: $\quad \mathsf{pk}, \mathsf{aux}, (\mathsf{sk}_i = S_i)_{i\in\mathsf{honest}} \leftarrow \mathsf{KG}(1^\kappa, N, T)$
6: $\quad$ **req** $\{$ no abort $\}$
7: **else** ▷ Simulated sharing
8: $\quad \mathsf{pk} \leftarrow \chi$
9: $\quad \mathsf{aux}, \mathsf{st}, \mathsf{status} \leftarrow \mathsf{SimKeygen}(\mathcal{A}, \mathsf{corrupt}, \mathsf{pk})$
10: $\quad$ **req** $\{$ $\mathsf{status} = \mathbf{accept}$ $\}$
11: $b' \leftarrow \mathcal{A}^{\mathsf{OEval}}(\mathsf{pk}, \mathsf{aux})$
12: **return** $b' = 0$

$\mathsf{OEval}(\mathsf{in})$

1: **if** $b = 0$ **then**
2: $\quad$ **return** $f(\mathsf{pk}, \mathsf{aux}, (S_i)_{i\in\mathsf{honest}}, \mathsf{in})$
3: **else**
4: $\quad$ **return** $\mathsf{SimF}(\mathsf{pk}, \mathsf{aux}, \mathsf{st}, \mathsf{in})$

Figure 8: Simulatability game for a KG scheme. The adversary $\mathcal{A}$ wins if the game KG-sim returns 1, i.e. if it is guessed correctly whether it is provided with real or simulated shares. Failed assertions stop the game and return 0 or 1 with probability 1/2.

A.4 Hardness of Hint-MLWE

We recall that Hint-MLWE is as hard as MLWE for some parameter regimes when using Gaussian secrets and noises. This was proved by Kim et al. in [KLSS23]. Espitau et al. [ENP24] extended that result even when noises have different standard deviations. This reduction is essentially optimal and the noise standard deviation has to grow with $\sqrt{Q}$ to ensure security as demonstrated in [ASY22, Section 3.3].

The spectral norm $s_1(\mathbf{M})$ of a matrix $\mathbf{M}$ is defined as the value $\max_{\mathbf{x}\neq 0} \frac{\|\mathbf{M}\mathbf{x}\|_2}{\|\mathbf{x}\|_2}$. Given a polynomial $\mathbf{c} \in \mathcal{R}$, we may abusively use the term "spectral norm $s_1(\mathbf{c})$ of $\mathbf{c}$" when referring to the spectral norm of the anti-circulant matrix $\mathcal{M}(\mathbf{c})$ associated to $\mathbf{c}$. We extend it to matrices of elements in $\mathcal{R}$. Finally, if $\mathbf{c} =$

$\sum_{0<i<n} c_i x^i$, then the Hermitian adjoint of $\mathbf{c}$, which we denote by $\mathbf{c}^*$, is defined as $\mathbf{c}^\circ = c_0 - \sum_{0<i<n} c_{n-i} x^i$. Note that $\mathcal{M}(\mathbf{c})^t = \mathcal{M}(\mathbf{c}^*)$.

We recall this result below.

Theorem A.1 (Hardness of Hint-MLWE with varying noises). *Let k, ℓ, Q, q, be positive integers, $\mathcal{C}$ be a distribution over $\mathcal{R}^Q$, $\sigma_{\mathbf{s}}$ the standard deviation of the secret vector, and $(\sigma_{\mathbf{r}}^{(i)})_{i\in[Q]}$ noise standard deviations. Let B_{HRLWE} be an overwhelming bound on $\sum_{i\in[Q]} \frac{s_1(c_i^* c_i)}{(\sigma_{\mathbf{r}}^{(i)})^2}$.*

Define $\sigma > 0$ a real number as $\frac{1}{\sigma^2} = 2 \cdot \left(\frac{1}{\sigma_{\mathbf{s}}^2} + B_{\mathsf{HRLWE}}\right)$. If $\sigma \geqslant \sqrt{2} \cdot \eta_\varepsilon(\mathbb{Z}^{d(k+\ell)})$ for some $\varepsilon \in (0, 1/2]$, then there exists an efficient reduction from $\mathsf{MLWE}_{q,k,\ell,\sigma}$ to $\mathsf{Hint\text{-}MLWE}_{q,k,\ell,Q,\sigma_{\mathbf{s}},(\sigma_{\mathbf{r}}^{(i)})_{i\in[Q]},\mathcal{C}}$ that reduces the advantage by at most 2ε.

B Additional material for T-Raccoon

B.1 Formal description of the short Threshold Raccoon variant of del Pino et al.

We provide the algorithms for Keygen (in Fig. 9); $\mathsf{ShareSign}_1$, $\mathsf{ShareSign}_2$ and $\mathsf{ShareSign}_3$ (in Fig. 10); Combine, PartialVerify and Verify (in Fig. 11).

$\mathsf{Keygen}(T, N)$

1: $\mathsf{seed} \leftarrow \{0,1\}^\kappa$
2: $\mathbf{A} := \mathsf{ExpandA}(\mathsf{seed}) \in \mathcal{R}_q^{k\times\ell}$
3: $\mathbf{s} \leftarrow \chi$
4: $\mathbf{t} := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{s}$
5: $\mathbf{t}_\top := \lfloor \mathbf{t} \rceil_{\nu_{\mathbf{t}}}$
6: $\mathsf{Shares} \leftarrow \mathsf{VandShare}(\mathbf{s}, [N], T, "")$
7: $\mathsf{aux} := \mathsf{Dict}\{\}$
8: **for** $i \in [N]$ **do**
9: $\quad \mathsf{sk}_i := \mathsf{Shares}[i]$
10: $\quad$ **for** $\mathsf{idx} \in \mathsf{Shares}[i]$ **do** ▷ Compute all partial public keys
11: $\quad\quad \mathsf{aux}[\mathsf{idx}] := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathsf{Shares}[i][\mathsf{idx}]$
12: **return** $(\mathsf{vk} := (\mathsf{seed}, \mathbf{t}_\top), \mathsf{ak} := \mathsf{aux}, \mathsf{sk} := (\mathsf{sk}_i)_{i\in[N]})$

Figure 9: Key generation procedure of T-Raccoon.

B.2 DKG for T-Raccoon

We define here a Distributed Key Generation (DKG) protocol for T-Raccoon (see Section B.1) that is based on the same ideas used in [dPENP25, Section 5.3] to distribute the key generation of the Ramp Secret Sharing based on the Coupon's collector problem. Notation is the same as in [dPENP25]. We provide the algorithms for $\mathsf{ShareKeygen}_1$ and $\mathsf{ShareKeygen}_2$ in Fig. 12, and CombineKey in Fig. 13.

We assume that the parties have jointly generated a random seed seed and expanded it to a matrix $\mathbf{A} = \mathsf{ExpandA}(\mathsf{seed}) \in \mathcal{R}_q^{k\times\ell}$ (e.g. as done in Algorithms 17, 18 and 19 of [dPENP25]). The core idea is that each party samples a secret $\mathbf{s}_i$, in such a way that the secret key is implicitly set to $\mathbf{s} = \sum_{i=1}^N \mathbf{s}_i$, then $\mathbf{s}_i$ is shared using VandShare from Alg. 1. Each partial share is sent to each of the parties with a commitment to a partial evaluation of the public key on the shares, as explained in $\mathsf{ShareKeygen}_1$. The parties, then, use $\mathsf{ShareKeygen}_2$ to check that each of the received shares is valid, i.e. have small norm, and consistent with the partial key commitment, eventually raising complaints. They publish a partial public key $\mathbf{b}_i$ and the complaints.

$\mathsf{ShareSign}_1(\mathsf{vk}, i, \mathsf{sk}_i, \mathsf{st}_i)$

1: $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$
2: $\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$
3: $\mathsf{st}_i := \mathsf{st}_i \cup \{(\mathsf{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)\}$
4: $\mathsf{cmt}_i = \mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, i, \mathbf{w}_i) \in \{0,1\}^{2\kappa}$
5: **return** $(\mathsf{pm}_1^i := \mathsf{cmt}_i, \mathsf{st}_i)$

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

1: **assert**$\{$ $\mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}$ $\}$
2: **assert**$\{$ $(\mathsf{pm}_1^i, \cdot, \cdot) \in \mathsf{st}_i$ $\}$
3: Pick $(\mathsf{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)$ from st_i
   with $\mathsf{pm}_1^i = \mathsf{cmt}_i$
4: $\mathsf{st}_i := \mathsf{st}_i \backslash \{(\mathsf{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)\}$
5: $\mathsf{st}_i := \mathsf{st}_i \cup \{(\mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$
6: **return** $(\mathsf{pm}_2^i := \mathbf{w}_i, \mathsf{st}_i)$

$\mathsf{ShareSign}_3(\mathsf{vk}, i, \mathsf{pm}_2, i, \mathsf{sk}_i, \mathsf{st}_i)$

1: **assert**$\{$ $(\cdot, \mathsf{pm}_2^i, \cdot) \in \mathsf{st}_i$ $\}$
2: Pick $(\mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, \mathbf{w}_i, \mathbf{r}_i)$
   from st_i with $\mathsf{pm}_2^i = \mathbf{w}_i$
3: Parse $\mathsf{pm}_1 = (\mathsf{cmt}_j)_j$, $\mathsf{pm}_2 = (\mathbf{w}_j)_{j \neq i}$
4: **for** $j \in [N]$ **do**
5: **if** $\mathsf{cmt}_j \neq \mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, j, \mathbf{w}_j)$ **then**
6: **return** $(\mathbf{abort}, \{j\})$
7: $\mathbf{w} := \left\lfloor \sum_{j \in [N]} \mathbf{w}_j \right\rceil_{\nu_{\mathbf{w}}} \in \mathcal{R}_{q_{\mathbf{w}}}^k$ ▷ Rounded commitment
8: $c := \mathsf{H}_c(\mathsf{vk}, \mathsf{msg}, \mathbf{w})$ ▷ Global challenge
9: $\mathsf{Indices} := \mathsf{VandRecover}(\mathsf{pk}, \mathsf{ak}, \mathsf{act})$
10: $\mathbf{z}_i := c \cdot \mathsf{sk}_i[\mathsf{Indices}[i]] + \mathbf{r}_i$ ▷ Individual response in $\mathcal{R}_q^\ell$
11: $\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)} = \mathbf{z}_i \in \mathcal{R}_q^\ell \times \mathcal{R}_q^k$
12: **return** $\mathsf{pm}_3^i := \mathbf{z}_i^{(1)}$

Figure 10: Signing procedure of T-Raccoon.

With CombineKey, parties can finally combine the partial public keys to get pk. During this procedure they check the correctness of the complain and that any set of shares for a valid reconstruction set given by VandRecover correctly recovers the partial public key $\mathbf{b}_i$ from the partial evaluations. The secret key is finally reconstructed by summing the shares received by all parties, i.e. $\mathsf{sk}_i[\mathsf{idx}] = \sum_{u=1}^{N} \mathsf{sk}_u^i[\mathsf{idx}]$ for all $\mathsf{idx} \in \mathsf{sk}_i^i$.

B.3 Optional algorithms of our Vandermonde Short Secret Sharing

In Figure 14 we present optional algorithms for the Vandermonde short secret sharing as used in Alg. 1.

The idea of $\mathsf{VandShare}_{\mathsf{N}}$ (with $\mathsf{VandRecover}_{\mathsf{N}}$) is to take a shortcut when $T = N$ and directly perform an N-out-of-N sharing. This neither increases or decreases the number of required shares, but allows to increase the number of shares sampled uniformly, that are independent of the secret, and can be compressed to the seed used for their generation.

Instead $\mathsf{VandShare}_2$ (with $\mathsf{VandRecover}_2$) is a 2-out-of-N sharing that slightly reduces the total number of shares and relies on the simple idea of applying a binary sharing to the secret. The core idea is that, given two different users $u, v \in [N]$[7] their binary representations need to differ in at least one position.

[7] We are identifying the users with their index in $\mathcal{P}$.

Combine($\mathsf{vk} = (\mathsf{seed}, \mathbf{t}_\top), \mathsf{ak}, \mathsf{msg}, (\mathsf{pm}_k)_{k\in\{1,2,3\}}$)

1: Parse $\mathsf{pm}_2 = (\mathbf{w}_i)_i$ and $\mathsf{pm}_3 = (\mathbf{z}_i^{(1)})_i$
2: $\mathbf{w} := \left\lfloor \sum_{j\in\mathsf{act}} \mathbf{w}_i \right\rceil_{\nu_\mathbf{w}}$
3: $c := H_c(\mathsf{vk}, \mathsf{msg}, \mathbf{w})$
4: $\mathbf{z}^{(1)} = \sum_{i\in[N]} \mathbf{z}_i^{(1)}$
5: $\mathbf{u} := \left\lfloor \mathbf{A}\cdot\mathbf{z}^{(1)} - 2^{\nu_\mathbf{t}}\cdot c\cdot\mathbf{t}_\top \right\rceil_{\nu_\mathbf{w}}$
6: $\mathsf{invalid} := \mathsf{PartialVerify}\left(\mathsf{act}, \mathsf{ak}, c, (\mathbf{z}_i^{(1)}, \mathbf{w}_i)_{i\in\mathsf{act}}\right)$
7: **if** $\mathsf{invalid} \neq \varnothing$ **then**
8: **return** (**abort**, invalid) ▷ Verify partial signatures
9: $\mathbf{h} := \mathbf{w} - \mathbf{u}$ ▷ Hint in $\mathcal{R}_{q_\mathbf{w}}^k$
10: **return** $\mathsf{sig} := (c, \mathbf{z}^{(1)}, \mathbf{h})$

PartialVerify(act,ak, c, $(\mathbf{z}_i^{(1)}, \mathbf{w}_i)_i$)

1: Fetch all the $(\mathbf{b}[\mathsf{idx}])_{\mathsf{idx}}$ from ak
2: $\mathsf{Indices} := \mathsf{VandRecover}(\mathsf{pk}, \mathsf{ak}, \mathsf{act})$
▷ Determine the keys in use for act
3: $\mathsf{invalid} = \varnothing$
4: **for** $i \in \mathsf{act}$ **do**
5: $\mathbf{y}_i := (\mathbf{A}\cdot\mathbf{z}_i - c\cdot\mathbf{b}[\mathsf{Indices}[i]]) - \mathbf{w}_i$
6: **if** $\{\left\|(\mathbf{z}_i^{(1)}, \mathbf{y}_i)\right\| > B_2^{\mathsf{ind}}\}$ **then**
7: $\mathsf{invalid} := \mathsf{invalid} \sqcup \{i\}$
8: **for** $i \in \mathsf{act}$ **do**
9: $X' := \sum_{i\in\mathsf{act}\setminus\{i\}} (\mathbf{z}_i^1, \mathbf{y}_i)$
10: **if** $\langle(\mathbf{z}_i^{(1)}, \mathbf{y}_i), X'\rangle / \|X'\| > J_{\max}$ **then**
11: $\mathsf{invalid} := \mathsf{invalid} \sqcup \{i\}$
12: **return** invalid

Verify($\mathsf{vk} := (\mathsf{seed}, \mathbf{t}_\top), \mathsf{sig}$)

1: Parse $\mathsf{sig} := (c, \mathbf{z}^{(1)}, \mathbf{h})$
2: $\mathbf{u} := \left\lfloor \mathbf{A}\cdot\mathbf{z}^{(1)} - 2^{\nu_\mathbf{t}}\cdot c\cdot\mathbf{t}_\top \right\rceil_{\nu_\mathbf{w}}$
3: $\mathbf{w} = \mathbf{u} + \mathbf{h}$
4: **if** $\{H(\mathsf{vk}, \mathsf{msg}, \mathbf{w}) = c\}$ **and** $\{\left\|(\mathbf{z}^{(1)}, 2^{\nu_\mathbf{w}}\mathbf{h})\right\| \leqslant B_2\}$ **then**
5: **return** true
6: **return** false

Figure 11: Combine and Verify procedure of T-Raccoon.

Thus, for each $i = 0, \ldots, n-1$, where n is the bit length of N, we can perform a 2-out-of-2 sharing of secret $x = x_0 + x_1$ and then send x_0 to all the users with i-th bit equal to 0 and x_1 to all the users with i-th bit equal to 1.

We leave as an open problem to find more efficient sharings for specific values of T and N.

B.4 Proof of unforgeability of T-Raccoon

[dPENP25] introduced a generic notion coined *functional simulatability* for short secret sharing – which we recall and explain in Section A.3, capturing the property that the public key appears pseudo-random, even when leaking up to $T-1$ partial secret keys, and, additionally, hints on the honest shares of the

$\mathsf{ShareKeygen}_1(\mathsf{st}_i, T, N)$

$\mathbf{s}_i \leftarrow \chi$
$\mathbf{b}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{s}_i$
$\mathsf{cmt}_i \leftarrow \mathsf{H}_{\mathsf{cmt}}(\mathbf{b}_i)$
$\mathsf{Shares} \leftarrow \mathsf{VandShare}(\mathbf{s}_i, [N], T, "")$
$\mathsf{msg}_i \leftarrow \varnothing$
for $u \in [N]$ **do**
  $\mathsf{sk}_i^u := \mathsf{Shares}[u]$
  $\mathsf{aux}_i^u := \mathsf{Dict}\{\}$, $\mathsf{cmt}_i^u := \mathsf{Dict}\{\}$
  for $\mathsf{idx} \in \mathsf{Shares}[u]$ **do**
    $\mathsf{aux}_i^u[\mathsf{idx}] := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathsf{Shares}[u][\mathsf{idx}]$
    $\mathsf{cmt}_i^u[\mathsf{idx}] := \mathsf{H}_{\mathsf{cmt}}(\mathsf{aux}_i^u[\mathsf{idx}])$
  $\mathsf{msg}_i := \mathsf{msg}_i \cup \{\mathsf{SKE.Encrypt}(K_{i\to u}^{\mathsf{SKE}}, \mathsf{sk}_i^u)\}$
$\mathsf{st}_i := \mathsf{st}_i \cup \{(\mathbf{s}_i, \mathbf{b}_i, (\mathsf{aux}_i^u)_{u\in[N]})\}$
return $\mathsf{pm}_1^i := (\mathsf{msg}_i, \mathsf{cmt}_i, (\mathsf{cmt}_i^u)_{u\in[N]})$

$\mathsf{ShareKeygen}_2(\mathsf{st}_i, \mathsf{pm}_1)$

parse pm_1 as $(\mathsf{msg}_u, \mathsf{cmt}_u, (\mathsf{cmt}_u^j)_{j\in[N]})_{u\in[N]}$
for $u \in [N]$ **do**
  get sk_u^i from msg_u using $\mathsf{SKE.Decrypt}$ with $K_{u\to i}^{\mathsf{SKE}}$
  $\mathsf{complaints}_i := \varnothing$
  for $\mathsf{idx} \in \mathsf{sk}_u^i$ **do**
    $\mathbf{b}_{bf} := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \mathsf{sk}_u^i[\mathsf{idx}]$
    if $\|\mathsf{sk}_u^i[\mathsf{idx}]\| > \mu$ or $\mathsf{cmt}_i^u[\mathsf{idx}] \neq \mathsf{H}_{\mathsf{cmt}}(\mathbf{b}_{bf})$ **then**
      $\mathsf{complaints}_i := \mathsf{complaints}_i \cup \{u, i, \mathsf{idx}, K_{u\to i}^{\mathsf{SKE}}, \mathsf{sig}_{u\to i}^{\mathsf{SKE}}\}$
  $\mathsf{st}_i := \mathsf{st}_i \cup \{(\mathsf{sk}_u^i, \mathbf{b}_b, \mathsf{aux}_i^u)\}$
return $\mathsf{pm}_2^i := (\mathsf{complaints}_i, \mathbf{b}_i, (\mathsf{aux}_i^u)_{u\in[N]})$

Figure 12: Share generation for the DKG protocol of T-Raccoon.

form $c \cdot \mathsf{sk}_i[\mathsf{idx}] + \mathbf{r}_i$. They proved that their variant of Threshold Raccoon (presented in Section B.1) is unforgeable under this notion.

However, their proof holds in a broadcast model, assuming strong properties from the communication layer. We adapt here the bookkeeping strategy from [dPN25] to Threshold Raccoon based on short secret sharing to support an adversary controlled communication layer, and prove the unforgeability of our threshold variant if the underlying short secret sharing is *functionally simulatable*. We prove this property for the Vandermonde secret sharing as defined in Section B.5. Note that the following property from [DKM+24] will be useful in the security proof.

Definition B.1. *For* $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k\times\ell}$, *and any* $\mathbf{u} \in \mathcal{R}_q^k$, *let* $\mathcal{D}_{q,l,\ell,\sigma,\nu}^{\mathsf{bd-MLWE}}(\mathbf{A}, \mathbf{u})$ *be the distribution defined as* $\{\lfloor \mathbf{A}\cdot\mathbf{s} + \mathbf{e} + \mathbf{u}\rceil_\nu \mid (\mathbf{s}, \mathbf{e}) \leftarrow \mathcal{D}_\sigma^\ell \times \mathcal{D}_\sigma^k\}$. *That is, it samples an* $\mathsf{MLWE}_{q,k,\ell}$ *instance shifted by* $\mathbf{u}$ *with* ν *bits dropped.*

Lemma B.1. *For any* $\sigma > 2n \cdot q^{\frac{1}{k+\ell}+\frac{2}{n\ell}}$ *and* $\nu < \log(q) - 2$, *we have:*

$$\Pr_{\mathbf{A}\xleftarrow{\$}\mathcal{R}^{k\times\ell}}\left[H_\infty(\mathcal{D}_{q,l,\ell,\sigma,\nu}^{\mathsf{bd-MLWE}}(\mathbf{A}, \mathbf{u})) \geqslant n-1\right] \geqslant 1 - 2^{-n+1}$$

We refer to [DKM+24, Lemma 3.8] for the proof of Lemma B.1.

Theorem B.1 (Unforgeability of T-Raccoon). *Assume* $\sigma > 2n \cdot q^{\frac{1}{k+\ell}+\frac{2}{n\ell}}$ *and* $\nu_{\mathbf{w}} < \log(q) - 2$.

CombineKey $(\mathsf{pm}_1, \mathsf{pm}_2)$

parse pm_1 as $(\mathsf{msg}_u, \mathsf{cmt}_u, (\mathsf{cmt}_u^j)_{j\in[N]})_{u\in[N]}$
parse pm_2 as $(\mathsf{complaints}_i, \mathbf{b}_i, (\mathsf{aux}_i^u)_{u\in[N]})_{i\in[N]}$
for $i \in [N]$ **do**
  if $\mathsf{H}_{\mathsf{cmt}}(\mathbf{b}_i) \neq \mathsf{cmt}_i$ **then**
    return ***abort***$,\{i\}$
$\mathsf{invalid} := \varnothing$
for $i \in [N]$ **do**
  for $\mathsf{act} \subset [N] : |\mathsf{act}| = T$ **do**
    $\mathsf{Indices} \leftarrow \mathsf{VandRecover}([N], \mathsf{act})$
    if $\mathbf{b}_i \neq \sum_{u\in\mathsf{act}} \mathsf{aux}_i^u[\mathsf{Indices}[u]]$ **then**
      $\mathsf{invalid} := \mathsf{invalid} \cup \{i\}$
  for $u, i, \mathsf{idx}, K_{u\to i}^{\mathsf{SKE}}, \mathsf{sig}_{u\to i}^{\mathsf{SKE}} \in \mathsf{complaints}_i$ **do**
    if $\mathsf{sig}_{u\to i}^{\mathsf{SKE}}$ not valid **then** $\mathsf{invalid} := \mathsf{invalid} \cup \{i\}$
    if $\mathsf{sk}_u^i[\mathsf{idx}]$ valid for $\mathsf{cmt}_i^u[\mathsf{idx}]$ **then**
      $\mathsf{invalid} := \mathsf{invalid} \cup \{i\}$
if $\mathsf{invalid} \neq \varnothing$ **then**
  return ***abort***$,\mathsf{invalid}$
$\mathbf{t} \leftarrow \left\lfloor \sum_{i\in[N]} \mathbf{b}_i \right\rceil_{\nu_{\mathbf{t}}}$
for $u \in [N]$ **do**
  $\mathsf{aux}^u := \mathsf{Dict}\{\mathsf{idx} : \sum_{j\in[N]} \mathsf{aux}_j^u[\mathsf{idx}] \mid \mathsf{idx} \in \mathsf{aux}_1^u\}$
return $\mathsf{vk} := (\mathsf{seed}, \mathbf{t})$, $\mathsf{ak} := (\mathsf{aux}^u)_{u\in[N]}$

Figure 13: Combine keys algorithm of the DKG protocol of T-Raccoon.

Formally, let $\mathcal{A}$ be an adversary against the TS-UF *security of our threshold scheme making Q_s calls to signing oracles, and at most $Q_{\mathsf{H}_{\mathsf{cmt}}}, Q_{\mathsf{H}_c}$ queries respectively to the random oracles $\mathsf{H}_{\mathsf{cmt}}, \mathsf{H}_c$. There exist adversaries $\mathcal{B}_1$ against the* KG-sim *security of our short secret sharing, $\mathcal{B}_2$ against the $\mathsf{STMSIS}_{q,k,\ell+1,\mathcal{C},B_{\mathsf{stmsis}}}$ problem running in time $\mathcal{T}_{\mathcal{B}_1} \approx \mathcal{T}_{\mathcal{B}_2} \approx \mathcal{T}_{\mathcal{A}}$ such that:*

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{TS\text{-}UF}} \leqslant \mathsf{Adv}_{\mathcal{B}_1}^{\mathsf{KG\text{-}sim}} + \mathsf{Adv}_{\mathcal{B}_2}^{\mathsf{STMSIS}} + Q_s \cdot (Q_{\mathsf{H}_{\mathsf{cmt}}} + Q_{\mathsf{H}_c}) \cdot 2^{-n+2} + \frac{Q_{\mathsf{H}_{\mathsf{cmt}}} \cdot T \cdot Q_s + (Q_{\mathsf{H}_{\mathsf{cmt}}} + Q_s)^2}{2^{2\kappa}}$$

For completeness, we include the full formal proof of the unforgeability of our T-Raccoon scheme. This is a straightforward adaptation of the proof and hybrids of [dPN25, Theorem 4].

For simplicity, we use the same notations as our Vandermonde sharing, and notably use a single share in $\mathsf{ShareSign}_3$, although the proof directly adapts to having more than one non-zero reconstruction coefficients as long as the functional simulatability of the underlying short secret sharing holds.

Proof. We proceed with a series of hybrids starting from the TS-UF game. Throughout this proof, we denote the advantage of an adversary $\mathcal{A}$ against a search game G by $\mathsf{Adv}_{\mathcal{A}}^{G} := \Pr[G(\mathcal{A}) \to 1]$ – in particular, we omit passing κ as input.

Game_1. This is the TS-UF game from Fig. 6.

Game_2. In this hybrid we defer the computation of the honest $\mathbf{w}_i$ to the second round of the protocol, and program the random oracle to be consistent. We also introduce a flag f_{fail} to explicitly mark additional cases where the adversary loses. Notably, f_{fail} is set to $\top$ when the random oracle is programmed on a previously queried value. This is the same as Game_2 of the Threshold ML-DSA proof in Fig. 18.

$\mathsf{VandShare}_{\mathsf{N}}(x, \mathcal{P}, \mathsf{idx} = \varnothing) \to \mathsf{Dict}$

$N = |\mathcal{P}|$
$\mathsf{idx} := \mathsf{idx} \,\|\, \texttt{":N:"}$ ▷ $\mathcal{P} = \{user_1, \ldots, user_N\}$
$s \leftarrow 0$
for $i = 1, \ldots, N-1$ **do**
  $x_i \leftarrow \chi$
  $s \leftarrow s + x_i$
$x_N \leftarrow x - s$ ▷ $s = x_1 + \cdots + x_{N-1}$
$\mathsf{Dict} = \{user_i : \{\mathsf{idx} \| \texttt{i} : x_i\} \mid \texttt{i} \mid i = 1, \ldots, N\}$.
return Dict

$\mathsf{VandShare}_{2}(x, \mathcal{P}, \mathsf{idx} = \varnothing) \to \mathsf{Dict}$

$N = |\mathcal{P}|$ ▷ $\mathcal{P} = \{user_1, \ldots, user_N\}$
$n = \lceil \log(N) \rceil$
$\mathsf{idx} := \mathsf{idx} \,\|\, \texttt{":B:"}$
for $i = 0, \ldots, n-1$ **do**
  $x_0 \leftarrow \chi$
  $x_1 \leftarrow x - x_0$
  for $u = 1, \ldots, N$ **do**
    $bit \leftarrow (u >> i)\ \%2$ ▷ Get the i-th bit of u
    $\mathsf{Dict}[user_u][\mathsf{idx} \| i \| \texttt{":"} \| bit] := x_{bit}$
return Dict

$\mathsf{VandRecover}_{2}(\mathcal{P}, \mathsf{act}, \mathsf{idx} = \varnothing) \to \mathsf{Dict}$

$N = |\mathcal{P}|$, $u = \mathsf{act}[0]$, $v = \mathsf{act}[1]$
$i := 0$
$\mathsf{idx} := \mathsf{idx} \,\|\, \texttt{":B:"}$
$bit_u \leftarrow (u >> i)\ \%2$, $bit_v \leftarrow (v >> i)\ \%2$
while $bit_u = bit_v$ **do** ▷ Get the first different bit entry
  $i := i + 1$
  $bit_u \leftarrow (u >> i)\ \%2$, $bit_v \leftarrow (v >> i)\ \%2$
return $\{u : \mathsf{idx} \| i \| \texttt{":"} \| bit_u\ \} \cup \{v : \mathsf{idx} \| i \| \texttt{":"} \| bit_v\ \}$

$\mathsf{VandRecover}_{\mathsf{N}}(\mathcal{P}, \mathsf{act}, \mathsf{idx} = \varnothing) \to \mathsf{Dict}$

$\mathsf{idx} := \mathsf{idx} \,\|\, \texttt{":N:"}$ ▷ All the shares required
return $\{user_i : \mathsf{idx} \| \texttt{i} \mid i = 1, \ldots, N\}$

Figure 14: Optional algorithms for $T \in \{2, N\}$ applying to the Vandermonde short secret sharing.

The view of the adversary $\mathcal{A}$ differs if they called the random oracle of one of the $\mathbf{w}_i$ before round 2 was executed, i.e.

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_1} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_2}\right| \leqslant \Pr[E]$$

where E is the event "$\mathsf{H}_{\mathsf{cmt}}$ was queried on a honest $\mathbf{w}_i$ before round 2 in Game_1".

By union-bound, we can reduce this to a single call to OSign_1.

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_1} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_2}\right| \leqslant \sum_{s=1}^{Q_s} \Pr[E'_s] \tag{7}$$

where E'_s is the event "The $\mathbf{w}_i$ produced by the s-th call to OSign_1 is queried on $\mathsf{H}_{\mathsf{cmt}}$ before round 2 in

Game_1".

Lemma B.1 tells us that the min-entropy of a single $\mathbf{w}_i$ is at least $n-1$ except with probability at most 2^{-n+1}. As there at most $Q_{\mathsf{H}_{\mathsf{cmt}}}$, we deduce that $\Pr[E'_s]$ is bounded by $Q_{\mathsf{H}_{\mathsf{cmt}}} \cdot 2^{-n+2}$.

Summing the above inequality over $s \in [Q_s]$, we obtain:

$$\left|\mathsf{Adv}^{\mathsf{Game}_1}_{\mathcal{A}} - \mathsf{Adv}^{\mathsf{Game}_2}_{\mathcal{A}}\right| \leqslant Q_s \cdot Q_{\mathsf{H}_{\mathsf{cmt}}} \cdot 2^{-n+2}$$

Remark 3. As an useful remark for following hybrids, we note that a given $\mathbf{w}_i$ can't be used twice by the same party after Game_2 as then the random oracle programming would fail, and the adversary loses.

Game_3. In this hybrid, we sample a challenge c in advance during round 2. When the last honest hash cmt_i obtains a corresponding $\mathbf{w}_i$ programmed, we program the values $\mathsf{H}_c(\mathsf{vk}, \mathsf{msg}, \mathbf{w}) = c$. This is analogous to Game_3 of Theorem 4.2 in Fig. 20.

The proof for this hybrid is analogous to the previous one. First, these changes do not bias H_c as the sampled challenges c are used at most once for programming H_c, and are not provided to the adversary before programming. The view of the adversary hence differs only if it queried the random oracle H_c on some tuple $(\mathsf{vk}, \mathsf{msg}, \mathbf{w})$, where $\mathbf{w}$ are the high bits of $\hat{\mathbf{w}} = \sum_{i \in \mathsf{act}} \mathbf{w}_i$ before it is programmed in round 2.

Again, we can bound this probability due to the high min-entropy of the last honest $\mathbf{w}_i$ that is sampled. As it is independent of the other $\mathbf{w}_j$, $j \neq i$, we can capture their sum in the vector $\mathbf{u}$ in Lemma B.1 and deduce that $\mathbf{w}$ has a high min-entropy, except with probability 2^{-n+1}. We apply the union-bound over the Q_s programmation, and Q_{H_c} calls to H_c and deduce

$$\left|\mathsf{Adv}^{\mathsf{Game}_2}_{\mathcal{A}} - \mathsf{Adv}^{\mathsf{Game}_3}_{\mathcal{A}}\right| \leqslant Q_s \cdot Q_{\mathsf{H}_c} \cdot 2^{-n+1}$$

Game_4. In this hybrid, we ensure that $\mathsf{H}_{\mathsf{cmt}}$ has no collisions and that the adversary cannot find preimages of hashes cmt_i after passing them to the second signing oracle. We introduce a table Cmt that records every cmt sampled in $\mathsf{H}_{\mathsf{cmt}}$ or in OSign_1, as well as those passed to the second signing oracle. Whenever a new hash cmt is sampled, we ensure that it was not previously added to Cmt, or the challenger sets the flag f_{fail} to $\top$ in order to abort the game. This is the same as Game_4 of the proof of Theorem 4.2 in Fig. 22.

The view of the adversary differs in Game_4 if it was previously able to (i) find a pre-image of a cmt_i after passing it to OSign_2, or (ii) if a given cmt was sampled twice.

We can bound the probability of (i) due to the pre-image resistance of the hash function $\mathsf{H}_{\mathsf{cmt}}$, and with an union bound over all the commits passed by the adversary to OSign_2 which gives a bound $Q_{\mathsf{H}_{\mathsf{cmt}}} \cdot T \cdot Q_s \cdot 2^{-2\kappa}$.

As for (ii), we rely on the collision resistance of $\mathsf{H}_{\mathsf{cmt}}$. Since the commitments are sampled uniformly from $\{0,1\}^{2\kappa}$, we can bound the probability of (ii) by $\frac{(Q_{\mathsf{H}_{\mathsf{cmt}}}+Q_s)^2}{2^{2\kappa}}$.

We conclude that,

$$\left|\mathsf{Adv}^{\mathsf{Game}_3}_{\mathcal{A}} - \mathsf{Adv}^{\mathsf{Game}_4}_{\mathcal{A}}\right| \leqslant \frac{Q_{\mathsf{H}_{\mathsf{cmt}}} \cdot T \cdot Q_s + (Q_{\mathsf{H}_{\mathsf{cmt}}} + Q_s)^2}{2^{2\kappa}}$$

Game_5. In this hybrid, we ensure that the challenge used in round 3 is the same as the one sampled in round 2, and we precompute the responses $\mathbf{z}_i$ in advance. This is the same as Game_6 in the proof Theorem 4.2 except that we sample responses as hints here instead of doing rejection sampling. It was formalized in Fig. 24.

We want to show that responses are identically distributed in Game_5. This is the case if programmed responses use the same c in both round 2 and round 3, and if they are used at most once.

Now, we can first observe that if the hash checks in round 3 pass, then all the $\mathsf{cmt}_i := \mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, \mathbf{w}_i)$ are correctly defined. Recall that Game_4 ensured that $\mathsf{H}_{\mathsf{cmt}}$ has no collisions, and that pre-images cannot be found after round 2 is called. Hence, if these checks pass, then it means that in round 2, all the hashes cmt_i either: (i) already had pre-images, or (ii) were produced by an honest party in OSign_1 and were waiting for programming. Eventually, all the missing cmt_i from (ii) must thus have been programmed in OSign_2

and as the challenger programs H_c as soon as all the $\mathbf{w}_i$ are defined, then it ensures that the same c is used in round 2 and in round 3.

Additionally, responses are used at most once by a given party due to the collision resistance of H_{cmt} and Remark 4 which ensures that party i will accept cmt_i at most once.

All in all, we conclude that the adversary's view is identically distributed in Game_4 and Game_5 so,

$$\mathsf{Adv}^{\mathsf{Game}_5}_{\mathcal{A}} = \mathsf{Adv}^{\mathsf{Game}_4}_{\mathcal{A}}$$

Game_6. In this hybrid, we reverse the sampling of $\mathbf{w}_i$ and $\mathbf{z}_i$. We start by sampling $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}, c \leftarrow \mathcal{C}$ and $\mathbf{z}_i = c \cdot \mathsf{sk}_i[\mathsf{Indices}[i]] + \mathbf{r}_i$. Then, we compute $\mathbf{w}_i$ as $\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{z}_i - \mathsf{aux}[\mathsf{Indices}[i]]$, where $\mathsf{aux}[\mathsf{Indices}[i]] = \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathsf{sk}_i[\mathsf{Indices}[i]]$ if the partial public key constructed from $\mathsf{sk}_i[\mathsf{Indices}[i]]$. This is analogous to what was done in Game_7 in Theorem 4.2.

We can see that the view of the adversary is identically distributed. Indeed, in Game_6 $\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{z}_i = c \cdot \underbrace{\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathsf{sk}_i[\mathsf{Indices}[i]]}_{\mathsf{aux}[\mathsf{Indices}[i]]} + \underbrace{\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i}_{\mathbf{w}_i}$ for $\mathbf{z}_i = \mathbf{r}_i + c \cdot \mathsf{sk}_i[\mathsf{Indices}[i]]$. We can thus equivalently write $\mathbf{w}_i = \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{z}_i - c \cdot \mathsf{aux}[\mathsf{Indices}[i]]$ and sample $\mathbf{z}_i$ first.

So,

$$\mathsf{Adv}^{\mathsf{Game}_6}_{\mathcal{A}} = \mathsf{Adv}^{\mathsf{Game}_5}_{\mathcal{A}}$$

Game_7. In this hybrid, we replace the public key by the rounding of a uniform value, i.e. $\lfloor \mathbf{t} \rceil_{\nu_{\mathbf{t}}}$ where $\mathbf{t} \xleftarrow{\$} \mathcal{R}_q^k$, and we replace the key generation and hints used for generating partial signatures by the *functional simulatability* simulators of our Vandermonde secret sharing. This is formalized in Fig. 25.

Formally, there exists an adversary $\mathcal{B}_1$ against the $\mathsf{KG\text{-}sim}$ property of our secret sharing:

$$\left|\mathsf{Adv}^{\mathsf{Game}_{11}}_{\mathcal{A}} - \mathsf{Adv}^{\mathsf{Game}_{12}}_{\mathcal{A}}\right| \leqslant \mathsf{Adv}^{\mathsf{KG\text{-}sim}}_{\mathcal{B}_1}$$

Conclusion. At this point, the view of the adversary only depends on the view the uniformly and honestly sampled public key vk. We conclude as in the last hybrid of [dPN25, Theorem 4], that forging a signature for this public key is at least as hard as solving $\mathsf{STMSIS}_{q,k,\ell+1,\mathcal{C},B_{\mathsf{stmsis}}}$.

We finally collect all the bounds to obtain the theorem statement, noting that all the intermediary adversaries have an execution time roughly equal to $\mathcal{T}(\mathcal{A})$. □

B.5 Proof of functional simulatability of our Vandermonde sharing

We now prove the *functional simulatability* for our Vandermonde sharing, which underlies the unforgeability proof of our revisited Threshold Raccoon in Section B.4.

Lemma B.2. *For each T, N there is a constant $Q_v(T,N)$ such that if $\mathsf{Hint\text{-}MLWE}_{q,k,\ell}$ is hard for $Q_v(T,N)$ hints of the form $\mathbf{s}+\mathbf{s}'$, $\mathbf{s}' \leftarrow \chi$, and Q_f hints of the form $c \cdot \mathbf{s} + \mathbf{r}$, $\mathbf{r} \leftarrow \chi_{\mathbf{r}}$, then our Vandermonde sharing is functionally simulatable.*

Proof. Let $\mathsf{corrupt} \subseteq \mathcal{P}$ be the set of corrupted parties, so $S_i[\mathsf{idx}]$ for $i \in \mathsf{corrupt}$ and $\mathsf{idx} \in |S_i|$ are the leaked shares. Let us start with a high-level intuition for this proof. We wish to replace the public key by a uniform sample and we need to prove that leakages through leaked shares, and through the evaluation function f_{hint} over the partial secrets $S_i[\mathsf{idx}]$ do not affect its pseudo-randomness.

1. First, we need to show that shares leaked to the adversary do not fully leak the secret. Interestingly, since at every step of the algorithm we derive new shares by adding a Gaussian noise from χ to a secret dependent value, we will be able to express all these shares using hints on the secret of the form $\mathbf{x}+\mathbf{y}$ with $\mathbf{y} \leftarrow \chi$.

2. Second, we need to evaluate the partial public keys composing the auxiliary information aux, and hints produced by f_{hint} on the shares, of the form $S_i[\mathsf{idx}] + \mathbf{r}$, for $\mathbf{r} \leftarrow \chi_{\mathbf{r}}$. For this we prove that our sharing has the nice property that we can safely leak enough of the shares such that either $S_i[\mathsf{idx}]$ or $\mathbf{x} - S_i[\mathsf{idx}]$ can be computed from leaked shares. In the first case, we directly compute the hint on the partial secret, while in the second case, we derive it from a hint on $\mathbf{x}$: $(\mathbf{x} + \mathbf{r}) - (\mathbf{x} - S_i[\mathsf{idx}])$.

Let us fix a secret vector $\mathbf{x} \in \mathcal{R}_q^{k+\ell}$ and its associated public key $\mathsf{pk} \in \mathcal{R}_q^{k+\ell}$. We wish to replace all computations done using the secret vector $\mathbf{x}$ using hints and pk, before applying the Hint-MLWE assumption to decorelate pk and $\mathbf{x}$.

Concretely, we proceed by induction on the size N of the set of parties $\mathcal{P}$ – assuming that at most $T-1$ parties in $\mathcal{P}$ are corrupted – to show that leaked shares in every recursive call to $\mathsf{VandShare}(\mathbf{x}, \mathcal{P}, T, \mathsf{idx})$ can be simulated from hints on $\mathbf{x}$ of the form $\mathbf{x} + \mathbf{y}$, $\mathbf{y} \leftarrow \chi$. We show also a stronger result, i.e. that we can even leak more shares such that for every unleaked share $S_i[\mathsf{idx}]$ for $i \notin \mathsf{corrupt}$, $\mathbf{x} - S_i[\mathsf{idx}]$ can be recomputed linearly from the leaked shares.

Consider a set of parties $\mathcal{P} = \{p_1, \ldots, p_N\}$ such that $T < N$. We prove our induction hypothesis on the base case $T = 1$, $T = 2$ and $T = N$. Then we show it to be true for the general case using our induction hypothesis on $\mathcal{P}_L$ and $\mathcal{P}_R$.

Case $T = 1$. In this case, there are no corrupted parties, and thus no leaked share. Additionally, each share is directly $\mathbf{x}$, thus $\mathbf{x} - S_i[\mathsf{idx}] = \mathbf{0}$ is known.

Case $T = N$. In this case, the sharing party samples $N-1$ shares from χ and sets the last one as $S_{p_N}[\mathsf{idx}\|N] = \mathbf{x} - \sum_{i=1}^{N-1} S_{p_i}[\mathsf{idx}\|i]$.

If $p_N \notin \mathsf{corrupt}$, then we can leak all of the shares for $i \neq p_N$ as they are completely independent of the secret. Also, $\mathbf{x} - S_{p_N}[\mathsf{idx}\|N] = \sum_{i=1}^{N-1} S_{p_i}[\mathsf{idx}\|i]$ which is easily computed from the leaked shares.

Otherwise, we leak all the shares except a single $i_0 \in \mathcal{P}\backslash\mathsf{corrupt}$. Again, only the leaked share $S_{p_N}[\mathsf{idx}\|N]$ depends on the secret. And, we can actually reexpress it as $S_{p_N}[\mathsf{idx}\|N] = (\mathbf{x} - S_{i_0}[\mathsf{idx}\|i_0]) - (\sum_{1 \leqslant i \leqslant N-1, i \neq i_0} S_i[\mathsf{idx}\|i])$, where the first part is actually a hint on $\mathbf{x}$ since S_{i_0} is not leaked. Then, we have the relation $\mathbf{x} - S_{i_0}[\mathsf{idx}\|i_0] = \sum_{1 \leqslant i \leqslant N, i \neq i_0} S_i[\mathsf{idx}\|i]$, i.e. we can express $\mathbf{x} - S_{i_0}[\mathsf{idx}\|i_0]$ as the sum of leaked shares.

Case T=2. Let n be the bit length of N and p_i be the only corrupt user. Then for each $j = 0, \ldots, n-1$ let b_j the j-th bit entry of i. Then, by definition of $\mathsf{VandShare}_2$ we have that $\mathbf{x} = S_{p_i}[\mathsf{idx}\|b_j] + S_p[\mathsf{idx}\|1-b_j]$ for some $p \in \mathcal{P}\backslash\mathsf{corrupt}$. If $b_j = 0$ then $S_{p_i}[\mathsf{idx}\|b_j]$ is sampled from χ, so no information on $\mathbf{x}$ is leaked, otherwise we reexpress $S_{p_i}[\mathsf{idx}\|b_j]$ as the hint $\mathbf{x} - S_p[\mathsf{idx}\|1-b_j]$ since $S_p[\mathsf{idx}\|1-b_j]$ is not leaked. It is clear that $\mathbf{x} - S_p[\mathsf{idx}\|1-b_j]$ is (a linear combination of) the leaked share $S_{p_i}[\mathsf{idx}\|b_j]$.

General case. Let k be one of the values taken by the for statement (Alg. 1, line 1) of the general case. If $k = 0$ or $k = T$, we simply conclude by the induction hypothesis applied to $\mathcal{P}_L$ or $\mathcal{P}_R$. Otherwise, either (i) $|\mathcal{P}_L \cap \mathsf{corrupt}| \leqslant k-1$ (i.e. $\mathbf{x}_0$ can not be recovered) or (ii) $|\mathcal{P}_R \cap \mathsf{corrupt}| \leqslant T-k-1$ (i.e. $\mathbf{x}_1$ can not be recovered).

First, assume (i). Then, we consider that we leak $\mathbf{x}_1$ as an hint of the form $\mathbf{x} + \mathbf{r}$, and Dict_R can be sampled directly from $\mathbf{x}_1$ and the shares all leaked. For Dict_L, we apply the induction hypothesis to the call $\mathsf{VandShare}(\mathbf{x}_0, \mathcal{P}_L, k, \mathsf{idx}_L)$, ensuring that leaked shares can be simulated using hints on $\mathbf{x}_0$ and that unleaked shares $S_i[\mathsf{idx}] \in \mathsf{Dict}_L$ verify $\mathbf{x}_0 - S_i[\mathsf{idx}]$ is a function of leaked shares. The core observation is that we can replace $\mathbf{x}_0$ by $\mathbf{x} - \mathbf{x}_1$ in these statements, and as $\mathbf{x}_1$ is leaked it is a linear function of leaked shares, thus also $\mathbf{x} - S_i[\mathsf{idx}]$. This concludes this induction step as $\mathbf{x}_1$ is itself a hint on $\mathbf{x}$.

The case (ii) is analogous, and we instead apply the induction hypothesis to Dict_R and consider that $\mathbf{x}_0$ is leaked, allowing to directly depend on $\mathbf{x}$ observing that $\mathbf{x}_1 = \mathbf{x} - \mathbf{x}_0$.

It then remains to show how to securely evaluate the partial public keys, and the hints using the eval function f_{hint}. The former can easily be achieved recalling that for every unleaked share $S_i[\mathsf{idx}]$, we have that $\mathbf{x} - S_i[\mathsf{idx}]$ is a linear function of leaked shares. Thus, we compute the partial public key associated

to $S_i[\mathsf{idx}]$ from pk and the leaked shares. The latter is slightly more involved as we would like to capture a hint on every partial share used given a signing set act using a single hint on the full secret $\mathbf{x}$.

To do so, we actually observe that the induction we described before always ensures that all but one of the shares returned by VandRecover are leaked. Thus, we are again reduced to needing a hint on a single unleaked share for every evaluation of f_{hint}. Then, this can be reduced to a hint on the full secret $\mathbf{x}$ observing again that $\mathbf{x} - S_i[\mathsf{idx}]$ is a linear function of leaked shares, and we hence compute $S_i[\mathsf{idx}] + \mathbf{r} = (\mathbf{x} - S_i[\mathsf{idx}]) - (\mathbf{x} - \mathbf{r})$.

At this point, all computations only depend on pk and hints on $\mathbf{x}$ and we can apply the Hint-MLWE assumption to replace pk by a uniform sample. We now compute the constant $Q_v(T, N)$, which is the number of necessary hints, by unfolding the above induction proof. This is done using the following recursive function:

$Q_v(T, N)$:

1. If $T = 1$, return 0.
2. If $T = N$, return 1.
3. If $T = 2$, return $\lceil \log(N) \rceil$.
4. Otherwise, return the sum for $\max(0, T - N + b) \leqslant k \leqslant \min(b, T)$ – with $b = \lfloor N/2 \rfloor$ – of
   (a) $Q_v(k, b)$ when $k = T$
   (b) $Q_v(T - k, N - b)$ when $k = 0$
   (c) $\max(1 + Q_v(k, b), Q_v(T - k, N - b))$ otherwise

□

B.6 Ommitted proof

Proof of Lemma 3.1. We first recall a useful convolution and tail-bound lemma for this proof.

Lemma B.3 (Simplified convolution lemma [Pei10, Thm. 4.5]). *Let positive reals σ_1, σ_2 such that $\sigma_1^{-2} + \sigma_2^{-2} \leqslant \eta'_\varepsilon(\mathbb{Z}^n)^{-2}$ for some positive $\varepsilon \leqslant 1/4$. Then the distribution $D_{\mathbb{Z}^n,\sigma_1} + D_{\mathbb{Z}^n,\sigma_2}$ is within statistical distance 8ε of $D_{\mathbb{Z}^n,\sqrt{\sigma_1^2+\sigma_2^2}}$.*

Lemma B.4 (Lemma 4.4 in [Lyu12], extended). *For any $\alpha > 1$, and any lattice $\Delta \subseteq \mathbb{R}^n$,*

$$\Pr[\|\mathbf{z}\| > \alpha\sigma\sqrt{n} \mid \mathbf{z} \leftarrow \mathcal{D}_{\Delta,\sigma}] \leqslant C^n$$

where $C = \alpha \cdot e^{\frac{1}{2}(1-\alpha^2)} < 1$. In particular, if $(\alpha \geqslant 1 + \frac{\kappa \log 2}{3n})$ or if $(\alpha \geqslant 1 + \sqrt{\frac{4\kappa \log 2}{n}})$, then $C^n \leqslant 2^{-\kappa}$.

We first observe that our sharing algorithm works by adding Gaussian noises sampled from χ to the input secret value. Hence, in the end, all shares are the sum of Gaussian samples.

We show by induction on the number of parties N that the maximal number of samples that compose a share (except for the input secret) is bounded by $T - 1$.

1. In the case $T = 1$, the shares are equal to the secret, and no additional sample is added.
2. In the case $T = N$, all shares are one Gaussian sample, except for the last, which is the sum of $N - 1 = T - 1$ samples added to the secret.
3. In the case $T = 2$, all shares have at most one new Gaussian sample.
4. In the general case, let k be as in line 1 of Alg. 1. For $k = 0$ or $k = T$ no sample is added and we conclude by induction. Otherwise, we add at most one new Gaussian sample per share before sharing recursively. We can conclude using the induction hypothesis since we have that $k, T - k < T$.

Applying Lemmas B.3 and B.4, we deduce that any share has a norm bounded by $\alpha\sigma\sqrt{n(k + \ell) \cdot T}$ with overwhelming probability. □

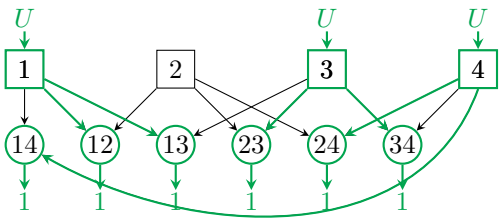


Figure 15: Illustration of the replicated secret sharing with $(N, T) = (4, 3)$: users are on top (rectangles), shares at the bottom (circles). Balanced assignment of shares to active users is performed via a max-flow solving algorithm (in green).

C Additional material for Threshold ML-DSA

C.1 Balanced partition of the shares for Replicated Secret Sharing

The sizes of partial secrets $\mathbf{s}_i^{\mathsf{part}}$ in Threshold ML-DSA impact greatly the efficiency of parameters as it directly correlates with the norm 2 of the hyperballs that are used, and it is important to use as few secrets as possible in a session for each party. To tackle this, we design a recovery algorithm that computes a balanced partition of the shares among the party during the signing session, that is for a partition $(m_i)_{i\in\mathsf{act}}$ of the secrets, it minimizes $\max_{i\in\mathsf{act}} |m_i|$.

As a general solution when fixing $U = \max_{i\in\mathsf{act}} |m_i|$, we can efficiently look for a solution using a max-flow modelization of the problem. We consider a bipartite graph consisting of (i) the users on one side, (ii) the secrets on the other side, and we add an edge between an user and a secret when said user owns that secret. We can solve the above balanced partition problem as follows:

1. We direct the edges from the users to the secrets.
2. We add a flow U entering the user nodes $i \in \mathsf{act}$.
3. We add an exit flow 1 on the secret nodes.

This is represented in Fig. 15. If the max-problem solution covers all the secrets, then we obtain a partition of maximal weight K. We can find the optimal weight K by testing values in increasing order.

For our concrete parameters, we consider $N \leqslant 6$, for which we can easily verify that the optimal assignations verify $K = \left\lceil \binom{N}{T-1}/T \right\rceil$.

Concrete implementation. For simplicity and efficiency, we do not wish to implement a max-flow solver in the Threshold ML-DSA code. Instead, we first observe that when $T = N$, then the partition is easily obtained as each signing party possesses exactly one secret. For the other cases $2 \leqslant T < N \leqslant 6$, we explicit an optimal solution for $\mathsf{act} = \{1, ..., T\}$ and each values (T, N). We obtain a partition for all other possible signing set act by symmetry by permutting the index of parties. The idea is formalized in Alg. 21.

C.2 Complete Threshold ML-DSA Parameters and Figures

In the following, we detail the parameters (r, r', K, ν) as set for every value $2 \leqslant T \leqslant N \leqslant 6$ for each security level of ML-DSA. For Threshold ML-DSA-44, they are in Table 8, for Threshold ML-DSA-65 in Table 9, and for Threshold ML-DSA-87 in Table 10. Note that we also provide a visual representation for the bandwidth and runtime costs of Threshold ML-DSA-44 in Fig. 17, and of the runtime costs of T-Raccoon-I in Fig. 16.

C.3 Omitted proof of unforgeability of Threshold ML-DSA

In this section, we fully formalize the unforgeability of Threshold ML-DSA. We provide a complete formal statement and its proof, adapting the proof from [dPN25]. We limit ourselves to the case $K = 1$, as the general case can be easily derived by considering $Q'_s = K \cdot Q_s$, the number of calls to signing queries.

Alg. 21 $\mathsf{RSSRecover}(\mathsf{act}) \to \mathcal{P}(S)^{\mathsf{act}}$

if $T = N$ **then**
  return $(m_i := (\{i\}))_{i \in \mathsf{act}}$
if $T = 2 \wedge N = 3$ **then**
  $\mathsf{sh} := \{0 : \{01, 02\}, 1 : \{12\}\}$
else if $T = 2 \wedge N = 4$ **then**
  $\mathsf{sh} := \{0 : \{013, 023\}, 1 : \{012, 123\}\}$
else if $T = 3 \wedge N = 4$ **then**
  $\mathsf{sh} := \{0 : \{01, 03\}, 1 : \{12, 13\}, 2 : \{23, 02\}\}$
else if $T = 2 \wedge N = 5$ **then**
  $\mathsf{sh} := \{0 : \{0134, 0234, 0124\}, 1 : \{1234, 0123\}\}$
else if $T = 3 \wedge N = 5$ **then**
  $\mathsf{sh} := \{0 : \{034, 013, 014, 023\}, 1 : \{012, 123, 124, 134\},$
    $2 : \{234, 024\}\}$
else if $T = 4 \wedge N = 5$ **then**
  $\mathsf{sh} := \{0 : \{01, 03, 04\}, 1 : \{12, 13, 14\}, 2 : \{23, 02, 24\},$
    $3 : \{34\}\}$
else if $T = 2 \wedge N = 6$ **then**
  $\mathsf{sh} := \{0 : \{02345, 01235, 01245\}, 1 : \{12345, 01234, 01345\}\}$
else if $T = 3 \wedge N = 6$ **then**
  $\mathsf{sh} := \{0 : \{0134, 0124, 0135, 0345, 0125\}, 1 : \{0145, 1345,$
    $1235, 1234, 1245\}, 2 : \{0235, 0245, 0234, 0123, 2345\}\}$
else if $T = 4 \wedge N = 6$ **then**
  $\mathsf{sh} := \{0 : \{014, 023, 015, 012, 045\}, 1 : \{135, 134, 125, 145,$
    $124\}, 2 : \{245, 024, 235, 234, 025\}, 3 : \{034,$
    $013, 123, 345, 035\}\}$
else if $T = 5 \wedge N = 6$ **then**
  $\mathsf{sh} := \{0 : \{01, 02, 05\}, 1 : \{12, 13, 15\}, 2 : \{23, 24, 25\},$
    $3 : \{03, 34, 35\}, 4 : \{45, 04, 14\}\}$
$\phi := \{i : i\}_{i \in [N]}$ ▷ Define a permutation of the user indexes
$i_1 := 0, i_2 := T$
for $j \in [N]$ **do**
  if $j \in \mathsf{act}$ **then**
    $\phi[i_1] = j$
    $i_1 = i_1 + 1$
  else
    $\phi[i_2] = j$
    $i_2 = i_2 + 1$
for $i \in [T]$ **do** ▷ Translate the ideal sharing for act
  $m_{\phi(i)} = \{\phi(u) \mid u \in \mathsf{sh}[i]\}$
return $(m_i)_{i \in \mathsf{act}}$

Theorem C.1 (Unforgeability of Threshold ML-DSA). *Formally, let $\mathcal{A}$ be an adversary against the* TS-UF *security of our threshold scheme making Q_s calls to signing oracles, and at most $Q_{\mathsf{H}_{\mathsf{cmt}}}, Q_{\mathsf{H}}$ queries respectively to the random oracles $\mathsf{H}_{\mathsf{cmt}}, \mathsf{H}$. There exist adversaries $\mathcal{B}_{\mathbf{s}}$ against the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{s}}}$, $\mathcal{B}_{\mathbf{r}}$ against the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}$ game, $\mathcal{B}_{\mathbf{z}}$ against the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{z}}}$ game, and $\mathcal{B}_{\mathsf{UF}}$ against the unforgeability of ML-DSA*

(T, N)	r	r'	K	**Comm. per party**
(2, 2)	252778	252833	2	10.5 kB
(2, 3)	310060	310138	3	15.8 kB
(3, 3)	246490	246546	4	21.0 kB
(2, 4)	31060	310138	3	15.8 kB
(3, 4)	279235	279314	7	36.8 kB
(4, 4)	243463	243519	8	42.0 kB
(2, 5)	285363	285459	3	15.8 kB
(3, 5)	282800	282912	14	73.5 kB
(4, 5)	259427	259526	30	157.4 kB
(5, 5)	239924	239981	16	84.0 kB
(2, 6)	300265	300362	4	21.0 kB
(3, 6)	277014	277139	19	99.8 kB
(4, 6)	268705	268831	74	388.4 kB
(5, 6)	250590	250686	100	524.8 kB
(6, 6)	219245	219301	37	194.2 kB

Table 8: Parameter sets for Threshold ML-DSA 44, aiming for a success probability 1/2. All sets use the same multiplicative factor $\nu = 3$.

(T, N)	r	r'	K	**Comm. per party**
(2, 2)	501495	501613	3	22.9 kB
(2, 3)	540212	540378	5	38.1 kB
(3, 3)	510387	510504	9	68.5 kB
(2, 4)	540212	540378	6	45.7 kB
(3, 4)	506761	506928	20	152.3 kB
(4, 4)	433594	433711	26	198.0 kB
(2, 5)	552371	552575	8	61.0 kB
(3, 5)	552909	553145	62	472.2 kB
(4, 5)	474331	474535	205	1561.3 kB
(5, 5)	425914	426032	78	594.1 kB
(2, 6)	571208	571412	8	61.0 kB
(3, 6)	536793	537058	95	723.5 kB
(4, 6)	488704	488969	804	6123.3 kB
(5, 6)	461324	461529	1200	9139.2 kB
(6, 6)	414896	415013	250	1904.0 kB

Table 9: Parameter sets for Threshold ML-DSA 65, aiming for a success probability 1/2. All sets use the same multiplicative factor $\nu = 6$.

running in time $\mathcal{T}_{\mathcal{B}_\mathsf{s}} \approx \mathcal{T}_{\mathcal{B}_\mathsf{r}} \approx \mathcal{T}_{\mathcal{B}_\mathsf{z}} \approx \mathcal{T}_{\mathcal{B}_{\mathsf{UF}}} \approx \mathcal{T}_{\mathcal{A}}$ *such that:*

$$\begin{aligned}
\mathsf{Adv}^{\mathsf{TS\text{-}UF}}_{\mathcal{A}} \leqslant\ & \frac{Q_s Q_{\mathsf{H}_{\mathsf{cmt}}}}{q^{nk}} + Q_{\mathsf{H}} Q_s^2 \cdot \left(\frac{2\gamma_2 + 1}{q}\right)^{kn} + Q_{\mathsf{H}} Q_s \cdot 2^{-256} \\
& + \frac{T \cdot Q_s Q_{\mathsf{H}_{\mathsf{cmt}}} + (Q_{\mathsf{H}_{\mathsf{cmt}}} + Q_s)^2}{2^{2\kappa}} + Q_{\mathsf{H}}^2 \cdot 2^{-512} + \mathsf{negl}(\kappa) \\
& + \left(1 + \frac{2\varepsilon}{1-\varepsilon}\right)^{Q_s} \cdot \left(\mathsf{Adv}^{\mathsf{MLWE}}_{\mathcal{B}_\mathsf{s}} + \mathsf{Adv}^{\mathsf{UF}}_{\mathcal{B}_{\mathsf{UF}}}\right) \\
& + Q_s \cdot \frac{3M - 2}{M - 1} \cdot \left(\mathsf{Adv}^{\mathsf{MLWE}}_{\mathcal{B}_\mathsf{r}} + \mathsf{Adv}^{\mathsf{MLWE}}_{\mathcal{B}_\mathsf{z}} + \frac{2\varepsilon}{1-\varepsilon}\right)
\end{aligned}$$

(T, N)	r	r'	K	**Comm. per party**
(2, 2)	503119	503192	3	31.1 kB
(2, 3)	631601	631703	4	41.5 kB
(3, 3)	483107	483180	6	62.2 kB
(2, 4)	632903	633006	4	41.5 kB
(3, 4)	551752	551854	11	114.1 kB
(4, 4)	487958	488031	14	145.2 kB
(2, 5)	607694	607820	5	51.9 kB
(3, 5)	577400	577546	26	269.6 kB
(4, 5)	518384	518510	70	725.8 kB
(5, 5)	468214	468287	35	362.9 kB
(2, 6)	665106	665232	5	51.9 kB
(3, 6)	577541	577704	39	404.4 kB
(4, 6)	517689	517853	208	2156.6 kB
(5, 6)	479692	479819	295	3058.6 kB
(6, 6)	424124	424197	87	902.0 kB

Table 10: Parameter sets for Threshold ML-DSA 87, aiming for a success probability 1/2. All sets use the same multiplicative factor $\nu = 7$.

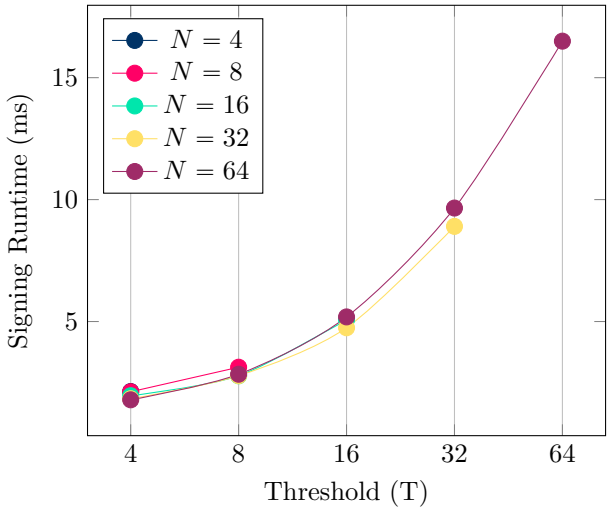


Figure 16: Runtime signing costs for T-Raccoon-1. Note that bandwidth costs remain constant across (T, N): around 35 kB. Note that the values for $T = \{4, 8, 16\}$ overlap.

For completeness, we include the full formal proof of the unforgeability of our Threshold ML-DSA scheme. This is a straightforward adaptation of the proof and hybrids of [dPN25, Theorem 4].

We first recall results from [dPN25] to tighly simulate rejected transcripts in the Fiat-Shamir with Aborts paradigm, and to evaluate rejection probabilities as a function of ε.

Lemma C.1 (Adaptation of [dPN25, Lemma 4]). *Let any $M > 1$, $B > 0, \varepsilon < 1$, $\mathbf{v} \in \mathcal{R}^{k+\ell}$, distribu-*

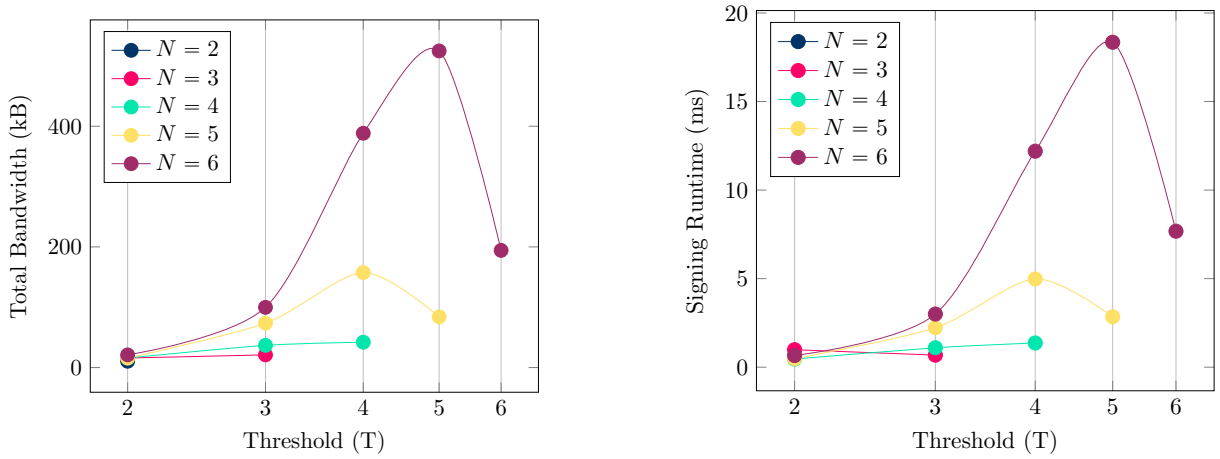


Figure 17: Bandwidth costs (left) and runtime signing costs (right) for Threshold ML-DSA-44. Note that the values for $N = 2$ and $N = 3$ overlap for all T.

Table 11: WAN median latencies between different AWS EC2 machines, as noted in [Clob].

	Taipei	**London**	**Virginia**	**Seoul**
Taipei		177.82	232.46	14.22
London	177.82		52.07	163.54
Virginia	232.46	52.07		130.24
Seoul	14.22	163.54	130.24	

tions $\chi_{\mathbf{r}}$, $\chi_{\mathbf{z}}$ over $\mathcal{R}^{k+\ell}$ such that $R^{\varepsilon}_{\infty}(\chi_{\mathbf{z}}||\chi_{\mathbf{r}} + \mathbf{v}) \leqslant M$. We have:

$$\frac{1-\varepsilon}{M} \leqslant \Pr\left[\mathrm{Rej}(\mathbf{v}, \chi_{\mathbf{r}}, \chi_{\mathbf{s}}, M) \neq \perp\right] \leqslant \frac{1}{M}$$
$$\delta \leqslant \frac{2\varepsilon}{1-\varepsilon}$$

where $\delta = \Delta(\mathbf{z}|\mathrm{acc}, \chi_{\mathbf{z}})$.

Lemma C.2 ([dPN25, Lemma 5]). *Given a secret $\mathbf{s}$, and challenge c, note $(\mathbf{z}|\mathrm{rej})_{\mathbf{v}=c\cdot\mathbf{s}}$ the distribution of rejected vectors $\mathbf{z}_i$ conditioned on $(\mathbf{s}, c)$. Let $\mathcal{A}$ be an adversary against the $\mathsf{MLWE}_{q,k,\ell,(\mathbf{z}|\mathrm{rej})_{\mathbf{v}=c\cdot\mathbf{s}}}$ problem. There exist adversaries $\mathcal{B}_1, \mathcal{B}_2$ respectively against the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}$ and $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{z}}}$ problems, running in time $\mathcal{T}_{\mathcal{B}_1} \approx \mathcal{T}_{\mathcal{B}_2} \approx \mathcal{T}_{\mathcal{A}}$ such that:*

$$\mathsf{Adv}^{\mathsf{MLWE}_{q,k,\ell,(\mathbf{z}|\mathrm{rej})_{\mathbf{v}=c\cdot\mathbf{s}}}}_{\mathcal{A}} \leqslant \frac{1}{p} \cdot \left(\mathsf{Adv}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}}_{\mathcal{B}_1} + \mathsf{Adv}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{z}}}}_{\mathcal{B}_2} + \delta\right)$$

where $p = \Pr\left[\mathrm{Rej}(c\mathbf{s}, \chi_{\mathbf{r}}, \chi_{\mathbf{s}}, M; \mathbf{r} \leftarrow \chi_{\mathbf{r}}) = \perp\right]$ is the probability of rejection of $\mathbf{z}$ conditioned on $(\mathbf{s}, c)$ and δ is the statistical distance between $(\mathbf{z}|\mathrm{acc})_{\mathbf{v}=c\cdot\mathbf{s}}$ and the target distribution $\chi_{\mathbf{z}}$.

Applying Lemma C.1, if $R^{\varepsilon}_{\infty}(\chi_{\mathbf{z}}||\chi_{\mathbf{r}} + c\cdot\mathbf{s}) \leqslant M$, then $p \geqslant 1 - \frac{1}{M}$.

Proof. We proceed with a series of hybrids starting from the TS-UF game. Throughough this proof, we denote the advantage of an adversary $\mathcal{A}$ against a search game G by $\mathsf{Adv}^{G}_{\mathcal{A}} := \Pr\left[G(\mathcal{A}) \to 1\right]$ – in particular, we omit passing κ as input.

Game_1. This is the TS-UF game from Fig. 6.

Game_2. In this hybrid we defer the computation of the honest $\mathbf{w}_i$ to the second round of the protocol, and program the random oracle to be consistent. We also introduce a flag f_{fail} to explicitly mark additional cases where the adversary loses. Notably, f_{fail} is set to $\top$ when the random oracle is programmed on a previously queried value. This is formalized in Fig. 18.

Game_2

Q_{msg} $\mathsf{UnopenedCmt}[\cdot] := \varnothing$
$f_{\mathsf{fail}} := \bot$
$\mathsf{pp}, \mathsf{st}_{\mathcal{A}} \leftarrow \mathsf{Setup}(\kappa, N, T)$
$\mathsf{corrupt} \leftarrow \mathcal{A}(\mathsf{pp})$
assert$\{\ \mathsf{corrupt} \subseteq [N]\ \}$
assert$\{\ |\mathsf{corrupt}| < T\ \}$
$\mathsf{honest} := [N] \backslash \mathsf{corrupt}$
$(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}) \leftarrow \mathsf{Sign.Keygen}(\mathsf{pp})$
for $i \in \mathsf{honest}$ **do**
  $\mathsf{st}_i := \varnothing$
$(\mathsf{msg}, \mathsf{sig}) \leftarrow \mathcal{A}^{\mathsf{H},(\mathsf{OSign}_i(\cdot))_{i\in[R]}}(\mathsf{st}_{\mathcal{A}}, \mathsf{vk}, (\mathsf{sk}_i)_{i\in\mathsf{corrupt}})$
if $f_{\mathsf{fail}} = \top$ **then**
  return 0
req $\{\ \mathsf{msg} \notin Q_{\mathsf{msg}}\ \}$
return $\mathsf{Verify}(\mathsf{vk}, \mathsf{msg}, \mathsf{sig})$
return 1

$\mathsf{ShareSign}_1(\mathsf{vk}, i, \mathsf{sk}_i, \mathsf{st}_i)$

$\mathsf{cmt}_i \xleftarrow{\$} \{0,1\}^{2\kappa}$
$\mathsf{UnopenedCmt}[(i, \mathsf{cmt}_i)] = \top$
return $(\mathsf{pm}_1^i := \mathsf{cmt}_i, \mathsf{st}_i)$

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

assert$\{\ \mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}\ \}$
assert$\{\mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] = \top\}$
$\mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] := \bot$
$\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$
$\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$
$\mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, \mathbf{w}_i) := \mathsf{pm}_1^i$
▷ Program random oracle
$\mathsf{st}_i := \mathsf{st}_i \cup \{(\mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$
return $(\mathsf{pm}_2^i := \mathbf{w}_i, \mathsf{st}_i)$

Figure 18: Second hybrid of the unforgeability proof of Threshold ML-DSA. If the random oracle is programmed on a previously queried index, consider that the adversary loses, i.e. f_{fail} is set to $\top$. Difference with the previous hybrid are highlighted.

The view of the adversary $\mathcal{A}$ differs if they called the random oracle of one of the $\mathbf{w}_i$ before round 2 was executed, i.e.

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_1} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_2}\right| \leqslant \Pr[E]$$

where E is the event "$\mathsf{H}_{\mathsf{cmt}}$ was queried on a honest $\mathbf{w}_i$ before round 2 in Game_1".

By union-bound, we can reduce this to a single call to OSign_1.

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_1} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_2}\right| \leqslant \sum_{s=1}^{Q_s} \Pr[E'_s] \tag{8}$$

where E'_s is the event "The $\mathbf{w}_i$ produced by the s-th call to OSign_1 is queried on $\mathsf{H}_{\mathsf{cmt}}$ before round 2 in Game_1".

We denote the above individual probabilities p_s for $s \in [Q_s]$. Formally, we introduce in Fig. 19 an intermediary game Int_1^s in which $\mathcal{A}$ wins with probability exactly p_s. We also introduce a tweak of Int_1^s, named Int_2^s, where we replace the s-th commitment $\mathbf{w}_i$ by a uniform value in $\mathcal{R}_q^k$.

First, the difference in advantage between Int_1^s and Int_2^s reduces to $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}$. Formally, we can define an adversary $\mathcal{E}^s$ against the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}$ problem such that:

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Int}_1^s} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Int}_2^s}\right| = \left|p_s - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Int}_2^s}\right| \leqslant \mathsf{Adv}_{\mathcal{E}^s}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}} \tag{9}$$

Also, we note that in Int_2^s, the probability that one call of $\mathcal{A}$ to the random oracle queries the s-th $\mathbf{w}_i$ is bounded by the min-entropy of $\mathbf{w}_i \xleftarrow{\$} \mathcal{R}_q^k$, i.e. by q^{-nk}. By union bound, $\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Int}_2^s} \leqslant Q_{\mathsf{H}_{\mathsf{cmt}}} \cdot q^{-nk}$.

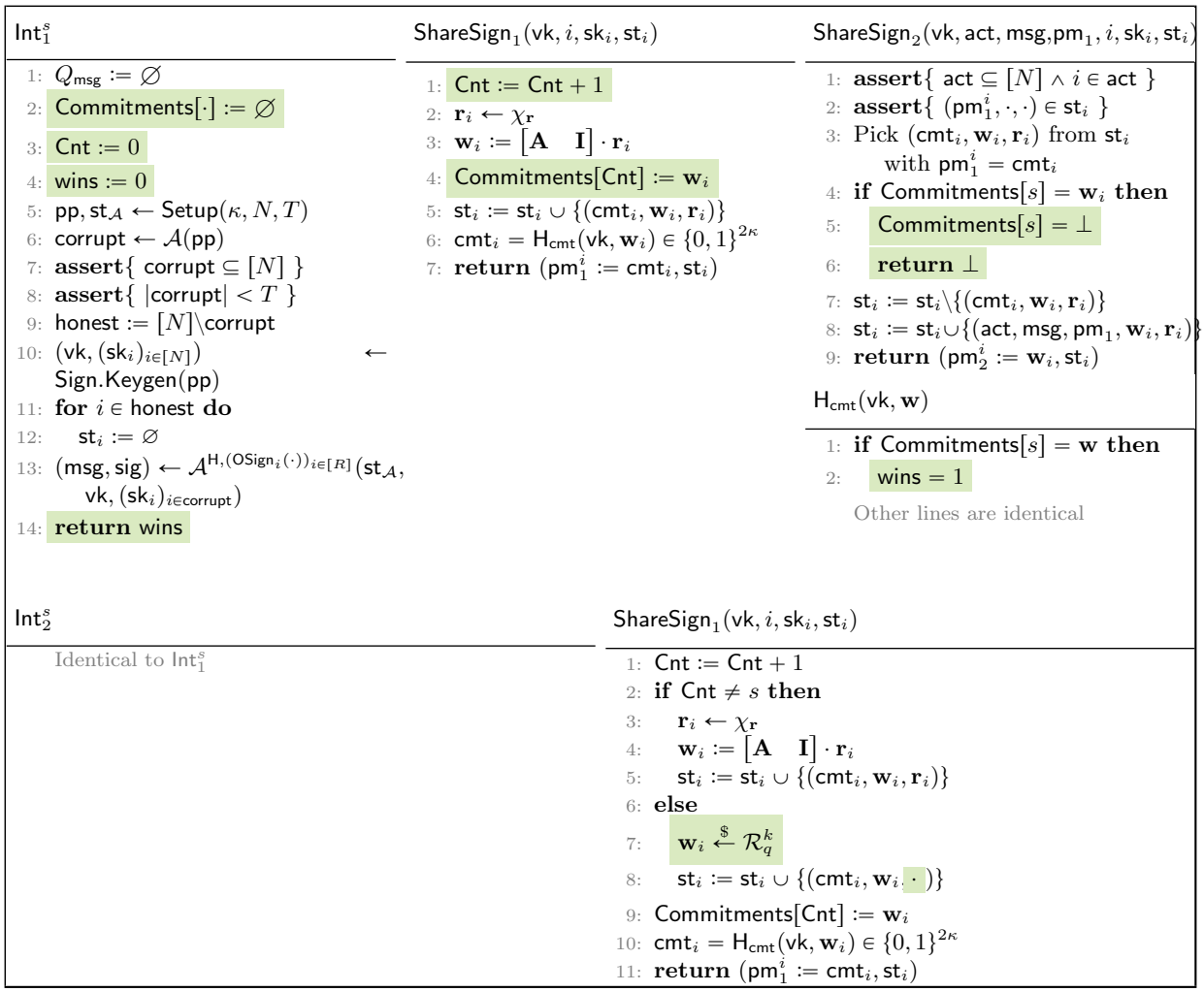


Figure 19: Intermediary games for the second hybrid of the unforgeability proof of Threshold ML-DSA. Difference with Game_1 are highlighted .

By combining the above inequality with Eq. (8) and Eq. (9), we conclude:

$$\left|\mathsf{Adv}^{\mathsf{Game}_1}_{\mathcal{A}} - \mathsf{Adv}^{\mathsf{Game}_2}_{\mathcal{A}}\right| \leqslant Q_s \cdot Q_{\mathsf{H}_{\mathsf{cmt}}} \cdot q^{-nk} + Q_s \cdot \mathsf{Adv}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}}_{\mathcal{B}_1}$$

where $\mathcal{B}_1$ is the adversary $\mathcal{E}^s$ maximizing $\mathsf{Adv}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}}_{\mathcal{E}^s}$.

Remark 4. As an useful remark for following hybrids, we note that a given $\mathbf{w}_i$ can't be used twice by the same party after Game_2 as then the random oracle programming would fail, and the adversary loses.

Game_3. In this hybrid, we sample a challenge c and its seed $\tilde{c}$ in advance during round 2. When the last honest hash cmt_i obtains a corresponding $\mathbf{w}_i$ programmed, we program the values $\mathsf{H}(\mu||\mathbf{w}_\top) = \tilde{c}$ and $\mathsf{SampleInBall}(\tilde{c}) = c$. This is formalized in Fig. 20.

The proof for this hybrid is analogous to the previous one. First, these changes do not bias H and $\mathsf{SampleInBall}$ as the sampled challenges c and seeds $\tilde{c}$ are used at most once for programming H and $\mathsf{SampleInBall}$, and are not provided to the adversary before programming. The view of the adversary hence differs only if it queried the random oracle (i) H on some $\mu||\mathbf{w}_\top$, where $\mathbf{w}$ are the high bits of $\hat{\mathbf{w}} = \sum_{i \in \mathsf{act}} \mathbf{w}_i$ before it is programmed in round 2, or (ii) $\mathsf{SampleInBall}$ on $\tilde{c}$ before it is programmed.

Game_3

$Q_{\mathsf{msg}}, \mathsf{UnopenedCmt}[\cdot] := \varnothing$

$\mathsf{ToProgram}, \mathsf{Programmed}[\cdot] := \varnothing$

Other lines are identical to Game_2

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

assert$\{\ \mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}\ \}$
assert$\{\ \mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] = \top\ \}$
$\mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] := \bot$
if $(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \cdot) \notin \mathsf{ToProgram}$ **then**
  $\tilde{c} \xleftarrow{\$} \{0,1\}^{256}; c \leftarrow \mathcal{C}$
  $\mathsf{ToProgram} := \mathsf{ToProgram} \cup \{(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \tilde{c}, c)\}$
Pick (a_1, a_2, a_3, c)
from $\mathsf{ToProgram}$ s.t. $(a_1, a_2, a_3) = (\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg})$
$\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$
$\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$
$\mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, \mathbf{w}_i) := \mathsf{pm}_1^i$
for $(\mathsf{act}', (\mathsf{cmt}'_j)_{j\in\mathsf{act}'}, \mathsf{msg}', c') \in \mathsf{ToProgram}$ **do**
  if $(i \in \mathsf{act}') \wedge (\mathsf{cmt}'_i = \mathsf{pm}_1^i) \wedge (\forall j \in \mathsf{act}'\backslash\{i\}, \exists \mathbf{w}_j, \mathsf{cmt}_j = \mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, j, \mathbf{w}_j))$ **then**
    ▷ Find all the new programmable challenges
    $\mathsf{ToProgram} := \mathsf{ToProgram}\backslash\{(\mathsf{act}', (\mathsf{cmt}'_j)_{j\in\mathsf{act}'}, \mathsf{msg}', \tilde{c}', c')\}$
    $\mathbf{w} := \sum_{j\in\mathsf{act}'} \mathbf{w}_j$; $(\mathbf{w}_\top, \mathbf{w}_\bot) = \mathsf{HighBits}_q(\mathbf{w}, 2\gamma_2)$
    $\mathsf{H}(\mu||\mathbf{w}_\top) := \tilde{c}'$; $\mathsf{SampleInBall}(\tilde{c}') := c'$ ▷ Program random oracle
$\mathsf{st}_i := \mathsf{st}_i \cup \{(\mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$
return $(\mathsf{pm}_2^i := \mathbf{w}_i, \mathsf{st}_i)$

Figure 20: Third hybrid of the unforgeability proof of Threshold ML-DSA. If the random oracle is programmed on a previously queried index, consider that the adversary loses, i.e. f_{fail} is set to $\top$. Difference with the previous hybrid are highlighted.

First, we can easily bound the probability that $\mathsf{SampleInBall}$ is queried on some $\tilde{c}$ before it is programmed by $Q_{\mathsf{H}}Q_s \cdot 2^{-256}$.

Then, we are interested in the probability that H is queried on some $\mu||\mathbf{w}_\top$ before it is programmed. For each $s \in Q_s$, we reexpress as an advantage the probability p_s that the s-th $\mathbf{w}_i$ produced during round 2 is used to compute a $\mathbf{w}_\top$ that was previously queried on the random oracle H. Formally, we define the game Int_3^s in Fig. 21, that verifies $p_s = \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Int}_3^s}$. By union bound, we have $\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_2} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_3}\right| \leqslant \sum_{s\in[Q_s]} p_s$.

We additionally define the game Int_4^s in Fig. 21, where we replace the s-th $\mathbf{w}_i$ by a uniform value in $\mathcal{R}_q^k$. The advantage difference between Int_3^s and Int_4^s again reduces to $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}$: $\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Int}_3^s} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Int}_4^s}\right| \leqslant \mathsf{Adv}_{\mathcal{E}^s}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}}$ for an adversary $\mathcal{E}^s$ against the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}$ problem.

Then, we can see that the probability that $\mathcal{A}$ wins in Int_4^s can be bounded using the min-entropy of $\mathsf{HighBits}_q(\mathcal{U}(\mathcal{R}_q^k), 2\gamma_2)$. Indeed, the s-th $\mathbf{w}_i$ is sampled uniformly from $\mathcal{R}_q^k$ and ensures that each $\mathbf{w}$ that needs to be programmed follows the distribution $\mathsf{HighBits}_q(\mathcal{U}(\mathcal{R}_q^k), 2\gamma_2)$. As there are at most Q_s $\mathbf{w}$ to check, we obtain the bound $\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Int}_4^s} \leqslant Q_{\mathsf{H}}Q_s \cdot \left(\frac{2\gamma_2+1}{q}\right)^{kn}$.

Putting together the different inequalities obtained, we finally deduce the existence of an adversary $\mathcal{B}_2$ against

Int^s_3

$Q_{\mathsf{msg}}, \mathsf{UnopenedCmt}[\cdot], := \varnothing$
$\mathsf{ToProgram} := \varnothing$
$\mathsf{Cnt} := 0$
$\mathsf{wins} := 0$
$f_{\mathsf{fail}} := \bot$
$\mathsf{pp}, \mathsf{st}_{\mathcal{A}} \leftarrow \mathsf{Setup}(\kappa, N, T)$
$\mathsf{corrupt} \leftarrow \mathcal{A}(\mathsf{pp})$
assert$\{\ \mathsf{corrupt} \subseteq [N]\ \}$
assert$\{\ |\mathsf{corrupt}| < T\ \}$
$\mathsf{honest} := [N] \backslash \mathsf{corrupt}$
$(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}) \leftarrow \mathsf{Sign.Keygen}(\mathsf{pp})$
for $i \in \mathsf{honest}$ **do**
  $\mathsf{st}_i := \varnothing$
$(\mathsf{msg}, \mathsf{sig}) \leftarrow \mathcal{A}^{\mathsf{H}, (\mathsf{OSign}_i(\cdot))_{i\in[R]}}(\mathsf{st}_{\mathcal{A}}, \mathsf{vk}, (\mathsf{sk}_i)_{i\in\mathsf{corrupt}})$
return wins

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

$\mathsf{Cnt} := \mathsf{Cnt} + 1$
assert$\{\ \mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}\ \}$
assert$\{\ \mathsf{UnopenedCmt}[(i, \mathsf{pm}^i_1)] = \top\}$
$\mathsf{UnopenedCmt}[(i, \mathsf{pm}^i_1)] := \bot$
if $(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \cdot) \notin \mathsf{ToProgram}$ **then**
  $\mathsf{ToProgram} := \mathsf{ToProgram} \cup \{(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \cdot)\}$
$\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$; $\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$
$\mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, \mathbf{w}_i) := \mathsf{pm}^i_1$
for $(\mathsf{act}', (\mathsf{cmt}'_j)_{j\in\mathsf{act}'}, \mathsf{msg}', \cdot) \in \mathsf{ToProgram}$ **do**
  if $(i \in \mathsf{act}') \wedge (\mathsf{cmt}'_i = \mathsf{pm}^i_1) \wedge (\forall j \in \mathsf{act}' \backslash \{i\}, \exists \mathbf{w}_j, \mathsf{cmt}_j = \mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, j, \mathbf{w}_j))$ **then**
    $\mathsf{ToProgram} := \mathsf{ToProgram} \backslash \{(\mathsf{act}', (\mathsf{cmt}'_j)_{j\in\mathsf{act}'}, \mathsf{msg}', \tilde{c}', c')\}$
    $\mathbf{w} := \sum_{j\in\mathsf{act}'} \mathbf{w}_j$; $(\mathbf{w}_\top, \mathbf{w}_\bot) = \mathsf{HighBits}_q(\mathbf{w}, 2\gamma_2)$
    if $\mathsf{Cnt} = s \wedge \exists(\mu \| \mathbf{w}_\top) \in \mathsf{H}$ **then** ▷ H already queried on $\mathbf{w}_\top$
      $\mathsf{wins} = 1$
$\mathsf{st}_i := \mathsf{st}_i \cup \{(\mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, \mathbf{w}_i, \mathbf{r}_i)\}$
return $(\mathsf{pm}^i_2 := \mathbf{w}_i, \mathsf{st}_i)$

Int^s_4

Identical to Int^s_3

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

$\mathsf{Cnt} := \mathsf{Cnt} + 1$
assert$\{\ \mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}\ \}$
assert$\{\ \mathsf{UnopenedCmt}[(i, \mathsf{pm}^i_1)] = \top\}$
$\mathsf{UnopenedCmt}[(i, \mathsf{pm}^i_1)] := \bot$
if $(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \cdot) \notin \mathsf{ToProgram}$ **then**
  $\mathsf{ToProgram} := \mathsf{ToProgram} \cup \{(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \cdot)\}$
if $\mathsf{Cnt} = s$ **then** $\mathbf{w}_i \xleftarrow{\$} \mathcal{R}^k_q$
else $\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$; $\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$
Rest is identical

Figure 21: Intermediary games for the third hybrid of the unforgeability proof of Threshold ML-DSA. Difference with Game_2 are highlighted .

the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}$ such that:

$$\left|\mathsf{Adv}^{\mathsf{Game}_2}_{\mathcal{A}} - \mathsf{Adv}^{\mathsf{Game}_3}_{\mathcal{A}}\right| \leqslant Q_{\mathsf{H}} Q_s^2 \cdot \left(\frac{2\gamma_2 + 1}{q}\right)^{kn} + Q_{\mathsf{H}} Q_s \cdot 2^{-256} + Q_s \cdot \mathsf{Adv}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}}_{\mathcal{B}_2}$$

Game_4**.** In this hybrid, we ensure that $\mathsf{H}_{\mathsf{cmt}}$ has no collisions and that the adversary cannot find preimages of hashes cmt_i after passing them to the second signing oracle. We introduce a table Cmt that records every cmt sampled in $\mathsf{H}_{\mathsf{cmt}}$ or in OSign_1, as well as those passed to the second signing oracle. Whenever a new hash cmt is sampled, we ensure that it was not previously added to Cmt, or the challenger sets the flag f_{fail} to $\top$ in order to abort the game. This is formalized in Fig. 22.

Game_4

$Q_{\mathsf{msg}}, \mathsf{UnopenedCmt}[\cdot], \mathsf{Cmt} := \varnothing$
Rest is identical to Game_3

$\mathsf{ShareSign}_1(\mathsf{vk}, i, \mathsf{sk}_i, \mathsf{st}_i)$

$\mathsf{cmt}_i \xleftarrow{\$} \{0,1\}^{2\kappa}$
$\mathsf{UnopenedCmt}[(i, \mathsf{cmt}_i)] = \top$
if $\mathsf{cmt}_i \in \mathsf{Cmt}$ **then**
  $f_{\mathsf{fail}} := \top$
  return $\bot$
$\mathsf{Cmt} := \mathsf{Cmt} \cup \{\mathsf{cmt}_i\}$
return $(\mathsf{pm}_1^i := \mathsf{cmt}_i, \mathsf{st}_i)$

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

$\mathsf{Cmt} := \mathsf{Cmt} \cup \{\mathsf{cmt}_j := \mathsf{pm}_1^j\}_{j \in \mathsf{act}}$
The rest is identical

$\mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, \mathbf{w})$

if $Q_{\mathsf{H}_{\mathsf{cmt}}}[(\mathsf{vk}, \mathbf{w})] = \bot$ **then**
  $\mathsf{cmt} \xleftarrow{\$} \{0,1\}^{2\kappa}$
  $Q_{\mathsf{H}_{\mathsf{cmt}}}[(\mathsf{vk}, \mathbf{w})] = \mathsf{cmt}$
  if $\mathsf{cmt}_i \in \mathsf{Cmt}$ **then**
    $f_{\mathsf{fail}} := \top$
    return $\bot$
  $\mathsf{Cmt} := \mathsf{Cmt} \cup \{\mathsf{cmt}_i\}$
return $Q_{\mathsf{H}_{\mathsf{cmt}}}[(\mathsf{vk}, \mathbf{w})] = \bot$

Figure 22: Fourth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

The view of the adversary differs in Game_4 if it was previously able to (i) find a pre-image of a cmt_i after passing it to OSign_2, or (ii) if a given cmt was sampled twice.

We can bound the probability of (i) due to the pre-image resistance of the hash function $\mathsf{H}_{\mathsf{cmt}}$, and with an union bound over all the commits passed by the adversary to OSign_2 which gives a bound $Q_{\mathsf{H}_{\mathsf{cmt}}} \cdot T \cdot Q_s \cdot 2^{-2\kappa}$.

As for (ii), we rely on the collision resistance of $\mathsf{H}_{\mathsf{cmt}}$. Since the commitments are sampled uniformly from $\{0,1\}^{2\kappa}$, we can bound the probability of (ii) by $\frac{(Q_{\mathsf{H}_{\mathsf{cmt}}}+Q_s)^2}{2^{2\kappa}}$.

We conclude that,

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_3} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_4}\right| \leqslant \frac{Q_{\mathsf{H}_{\mathsf{cmt}}} \cdot T \cdot Q_s + (Q_{\mathsf{H}_{\mathsf{cmt}}} + Q_s)^2}{2^{2\kappa}}$$

Game_5. In this hybrid, we ensure that H on $tr||\mathsf{msg}$ does never return twice the same $\tilde{c}$ for two different messages. This is formalized in Fig. 23.

As for the previous hybrid, this is ensured by the collision resistance of H due to the large space of $\tilde{c}$.

We conclude that,

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_4} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_5}\right| \leqslant Q_{\mathsf{H}}^2 \cdot 2^{-512}$$

Game_6. In this hybrid, we ensure that the challenge used in round 3 is the same as the one sampled in round 2, and we precompute the responses $\mathbf{z}_i$ in advance. This is formalized in Fig. 24.

Game_5

Identical to Game_4

$\mathsf{H}(\mathsf{str})$

1: **if** $\mathsf{str} = tr||\mathsf{msg}$ and $Q_{\mathsf{H}}[tr||\mathsf{msg}] = \bot$ **then**
2: $\quad \tilde{c} \leftarrow \{0,1\}^{512}$
3: $\quad Q_{\mathsf{H}}[tr||\mathsf{msg}] = \tilde{c}$
4: $\quad$ **if** $\exists \mathsf{msg}', Q_{\mathsf{H}}[tr||\mathsf{msg}'] = \tilde{c}$ or $(\cdot, \tilde{c}, \cdot) \in \mathsf{ToProgram}$ **then**
5: $\quad\quad f_{\mathsf{fail}} := \top$
6: $\quad\quad$ **return** $\bot$

Rest is identical

Figure 23: Fifth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

We want to show that responses are identically distributed in Game_6. This is the case if programmed responses use the same c in both round 2 and round 3, and if they are used at most once.

Now, we can first observe that if the hash checks in round 3 pass, then all the $\mathsf{cmt}_i := \mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, \mathbf{w}_i)$ are correctly defined. Recall that Game_4 ensured that $\mathsf{H}_{\mathsf{cmt}}$ has no collisions, and that pre-images cannot be found after round 2 is called. Hence, if these checks pass, then it means that in round 2, all the hashes cmt_i either: (i) already had pre-images, or (ii) were produced by an honest party in OSign_1 and were waiting for programming. Eventually, all the missing cmt_i from (ii) must thus have been programmed in OSign_2 and as the challenger programs $\mathsf{SampleInBall}$ as soon as all the $\mathbf{w}_i$ are defined, then it ensures that the same c is used in round 2 and in round 3.

Additionally, responses are used at most once by a given party due to the collision resistance of H_{cmt} and Remark 4 which ensures that party i will accept cmt_i at most once.

All in all, we conclude that the adversary's view is identically distributed in Game_5 and Game_6 so,

$$\mathsf{Adv}^{\mathsf{Game}_6}_{\mathcal{A}} = \mathsf{Adv}^{\mathsf{Game}_5}_{\mathcal{A}}$$

Game_7. In this hybrid, we no longer sample $\mathbf{r}_i$ out of the rejection sampling. In case the rejection sampling aborts, we sample a fresh $\mathbf{z}_i$ from the aborting distribution $(\mathbf{z}|\mathrm{rej})_{\mathbf{v}=c\cdot\mathbf{s}_i^{\mathsf{part}}}$. Then, we compute $\mathbf{w}_i$ as $\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix}\cdot\mathbf{z}_i - \mathbf{t}_i^{\mathsf{part}}$, where $\mathbf{t}_i^{\mathsf{part}} = \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix}\cdot\mathbf{s}_i^{\mathsf{part}}$. This is formalized in Fig. 25.

We can see that the view of the adversary is identically distributed. Indeed, in Game_6 $\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix}\cdot\mathbf{z}_i = c\cdot\underbrace{\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix}\cdot\mathbf{s}_i^{\mathsf{part}}}_{\mathbf{t}_i^{\mathsf{part}}} + \underbrace{\begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix}\cdot\mathbf{r}_i}_{\mathbf{w}_i}$ for $\mathbf{z}_i = \mathbf{r}_i + c\cdot\mathbf{s}_i^{\mathsf{part}}$ (even in case it is rejected). We can thus equivalently write $\mathbf{w}_i = \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix}\cdot\mathbf{z}_i - c\cdot\mathbf{t}_i^{\mathsf{part}}$ and sample $\mathbf{z}_i$ first. The sampling of rejected $\mathbf{z}_i$ from $(\mathbf{z}|\mathrm{rej})_{\mathbf{v}=c\cdot\mathbf{s}_i^{\mathsf{part}}}$ finally ensures the same distribution of $\mathbf{z}_i$ in the above equality in case of rejections.

So,

$$\mathsf{Adv}^{\mathsf{Game}_7}_{\mathcal{A}} = \mathsf{Adv}^{\mathsf{Game}_6}_{\mathcal{A}}$$

Game_8. In this hybrid, we ensure that the values $c\cdot\mathbf{s}_i^{\mathsf{part}}$ have a twisted norm at most B. That is, whenever this is not the case we set the flag f_{fail} to $\top$ to abort the game. This is formalized in Fig. 26.

By assumption on B, for any choice of act this is true with overwhelming probability over the randomness of c and of the secrets.

Since the adversary chooses act, it may not be independent of the secrets and we cannot directly conclude. However, we can simply guess act since there are $\binom{N}{T} = \mathsf{poly}(\kappa)$ of them by assumption.

By union-bound over all the calls to a signing oracle, we conclude:

$$\left|\mathsf{Adv}^{\mathsf{Game}_7}_{\mathcal{A}} - \mathsf{Adv}^{\mathsf{Game}_8}_{\mathcal{A}}\right| \leqslant Q_s\cdot\binom{N}{T}\cdot\mathsf{negl}(\kappa)$$

Game_6

$Q_{\mathsf{msg}}, \mathsf{UnopenedCmt}[\cdot], \mathsf{Cmt}, \mathsf{Responses}[\cdot] := \varnothing$

Rest is identical to Game_4

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

assert$\{\ \mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}\ \}$
assert$\{\ \mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] = \top\ \}$
$\mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] := \bot$
if $(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \cdot) \notin \mathsf{ToProgram}$ **then**
  $\tilde{c} \xleftarrow{\$} \{0,1\}^{256}; c \leftarrow \mathcal{C}$
  $\mathsf{ToProgram} := \mathsf{ToProgram} \cup \{(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \tilde{c}, c)\}$
Pick (a_1, a_2, a_3, c) from $\mathsf{ToProgram}$ s.t. $(a_1, a_2, a_3) = (\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg})$
$\mathbf{r}_i \leftarrow \chi_{\mathbf{r}}$
$\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{r}_i$
$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I\in m_i} \mathbf{s}_I$
$\mathbf{z}_i \leftarrow \mathrm{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M; \mathbf{r}_i)$
$\mathsf{Responses}[(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, i)] := \mathbf{z}_i$

Rest is identical

$\mathsf{ShareSign}_3(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, i, \mathsf{pm}_2, i, \mathsf{sk}_i, \mathsf{st}_i)$

assert$\{\ (\mathsf{act}, \mathsf{msg}, \cdot, \mathsf{pm}_2^i, \cdot) \in \mathsf{st}_i\ \}$
Pick $(\mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, \mathbf{w}_i, \cdot)$ from st_i
  with $\mathsf{pm}_2^i = \mathbf{w}_i$
Parse $\mathsf{pm}_1 = (\mathsf{cmt}_j)_j$, $\mathsf{pm}_2 = (\mathbf{w}_j)_{j\neq i}$
for $j \in \mathsf{act}$ **do**
  if $\mathsf{cmt}_j \neq \mathsf{H}_{\mathsf{cmt}}(\mathsf{vk}, j, \mathbf{w}_j)$ **then**
    abort ▷ Check hash commit.
$\mathsf{st}_i := \mathsf{st}_i \backslash \{(\mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, \mathbf{w}_i, \cdot)\}$
if $\mathbf{z}_i^1, \mathbf{z}_i^2 := \mathsf{Responses}[(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, i)]$ **then return** $(\mathbf{z}_i^1, \mathsf{st}_i)$
else abort

Figure 24: Sixth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

Game_9. In this hybrid, we replace the aborting $\mathbf{w}_i$ by a uniform value in $\mathcal{R}_q^k$. This is formalized in Fig. 27.

There are up to Q_s $\mathbf{w}_i$ to replace. We define a family of games G^p for $p \in [Q_s + 1]$, replacing one by one the $\mathbf{w}_i$ by a uniform vector, which verifies:

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_8} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_9}\right| \leqslant \sum_{s\in[Q_s]} \left|\mathsf{Adv}_{\mathcal{A}}^{G^{p+1}} - \mathsf{Adv}_{\mathcal{A}}^{G^p}\right|$$

Let $p \in [Q_s + 1]$. We want to bound $\left|\mathsf{Adv}_{\mathcal{A}}^{G^{p+1}} - \mathsf{Adv}_{\mathcal{A}}^{G^p}\right|$.

The core idea is to condition on the value of $v := (\mathsf{act}, i, \mathbf{s}_i^{\mathsf{part}}, c)$ used to define the distribution $(\mathbf{z}|\mathrm{rej})_{\mathbf{v}=c\cdot\mathbf{s}_i^{\mathsf{part}}}$:

$$\left|\mathsf{Adv}_{\mathcal{A}}^{G^{p+1}} - \mathsf{Adv}_{\mathcal{A}}^{G^p}\right| \leqslant \sum_{\mathbf{s},c} \left|\mathsf{Adv}_{\mathcal{A}}^{F'^v} - \mathsf{Adv}_{\mathcal{A}}^{F^v}\right| \cdot \Pr\left[v \text{ used by } G^s\right] \tag{10}$$

where F'^v and F^v are respectively the games G^{p+1} and G^p where we impose the set of signers act, signer i, and challenge c for the p-th $\mathbf{w}_i$ to replace, and sample the secrets $\mathbf{s}_I$ to be consistent with the value $\mathbf{s}_i^{\mathsf{part}}$.

Game_7

Identical to Game_6

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

assert$\{\ \mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}\ \}$
assert$\{\ \mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] = \top\ \}$
$\mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] := \bot$
if $(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \cdot) \notin \mathsf{ToProgram}$ **then**
  $\tilde{c} \xleftarrow{\$} \{0,1\}^{256}; c \leftarrow \mathcal{C}$
  $\mathsf{ToProgram} := \mathsf{ToProgram} \cup \{(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \tilde{c}, c)\}$
Pick (a_1, a_2, a_3, c) from $\mathsf{ToProgram}$ s.t. $(a_1, a_2, a_3) = (\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg})$
$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I\in m_i} \mathbf{s}_I$
$\mathbf{z}_i \leftarrow \mathrm{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$
if $\mathbf{z}_i = \bot$ **then** ▷ if reject
  $\mathbf{z}_i \leftarrow (\mathbf{z}|\mathrm{rej})_{v=c\cdot\mathbf{s}_i^{\mathsf{part}}}$
$\mathbf{t}_i^{\mathsf{part}} := \sum_{I\in m_i} \mathbf{t}_I$
$\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\mathsf{part}}$
if $\mathbf{z}_i \neq \bot$ **then**
  $\mathsf{Responses}[(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, i)] := \mathbf{z}_i$

Rest is identical

Figure 25: Seventh hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

We can then define adversaries $\mathcal{E}^v$ against the $\mathsf{MLWE}_{q,k,\ell,(\mathbf{z}|\mathrm{rej})_{\mathbf{v}=c\cdot\mathbf{s}_i^{\mathsf{part}}}}$ such that

$$\left|\mathsf{Adv}_{\mathcal{A}}^{F'^v} - \mathsf{Adv}_{\mathcal{A}}^{F^v}\right| = \mathsf{Adv}_{\mathcal{E}^v}^{\mathsf{MLWE}_{q,k,\ell,(\mathbf{z}|\mathrm{rej})_{\mathbf{v}=c\cdot\mathbf{s}_i^{\mathsf{part}}}}} \tag{11}$$

Game_8 ensured that the twisted norm of $\mathbf{v} = c \cdot \mathbf{s}_i^{\mathsf{part}}$ is bounded by B which ensures that $R_\infty^\varepsilon(\chi_{\mathbf{z}}||\chi_{\mathbf{r}} + \mathbf{v}) \leqslant M$ by Lemma 2.3. We can thus apply Lemma C.2: there exists adversaries $\mathcal{E}'^v$ and $\mathcal{E}''^v$ such that

$$\mathsf{Adv}_{\mathcal{E}^v}^{\mathsf{MLWE}_{q,k,\ell,(\mathbf{z}|\mathrm{rej})_{\mathbf{v}}}} \leqslant \frac{1}{1-\frac{1}{M}} \cdot (\mathsf{Adv}_{\mathcal{E}'^v}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}} + \mathsf{Adv}_{\mathcal{E}''^v}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{z}}}} + \delta^v) \tag{12}$$

where $\mathbf{v} = c \cdot \mathbf{s}_i^{\mathsf{part}}$, δ^v is the statistical distance between $\chi_{\mathbf{z}}$ and $(\mathbf{z}|\mathrm{rej})_{\mathbf{v}=c\cdot\mathbf{s}_i^{\mathsf{part}}}$.

We can also apply Lemma C.1 to bound $\delta^v \leqslant \frac{2\varepsilon}{1-\varepsilon}$.

By combining Equations 10, 11, and 12, we obtain:

$$\begin{aligned}\left|\mathsf{Adv}_{\mathcal{A}}^{G^{p+1}} - \mathsf{Adv}_{\mathcal{A}}^{G^p}\right| \leqslant \frac{M}{M-1} \max_{\mathsf{act},i,\|c\mathbf{s}\|\leqslant B} \left(\mathsf{Adv}_{\mathcal{E}'^v}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}} + \mathsf{Adv}_{\mathcal{E}''^v}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{z}}}}\right) \\ + \frac{M}{M-1} \cdot \frac{2\varepsilon}{1-\varepsilon} + \mathsf{negl}(\kappa)\end{aligned}$$

Finally, by summing all the above inequalities, we obtain

$$\begin{aligned}\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_8} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_9}\right| \leqslant Q_s \frac{M}{M-1} \left(\mathsf{Adv}_{\mathcal{B}_3}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}} + \mathsf{Adv}_{\mathcal{B}_4}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{z}}}}\right) \\ + Q_s \frac{M}{M-1} \cdot \frac{2\varepsilon}{1-\varepsilon} + \mathsf{negl}(\kappa)\end{aligned}$$

where $\mathcal{B}_3$ is an adversary against the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{r}}}$ problem, $\mathcal{B}_4$ is an adversary against the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{z}}}$ problem.

Game_8

Identical to Game_7

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

assert$\{\ \mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}\ \}$
assert$\{\ \mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] = \top\ \}$
$\mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] := \bot$
if $(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \cdot) \notin \mathsf{ToProgram}$ **then**
  $\tilde{c} \xleftarrow{\$} \{0,1\}^{256}; c \leftarrow \mathcal{C}$
  $\mathsf{ToProgram} := \mathsf{ToProgram} \cup \{(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \tilde{c}, c)\}$
Pick (a_1, a_2, a_3, c) from $\mathsf{ToProgram}$ s.t. $(a_1, a_2, a_3) = (\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg})$
$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I\in m_i} \mathbf{s}_I$; $(\mathbf{u}_1, \mathbf{u}_2) = \mathbf{s}_i^{\mathsf{part}}$
if $\|(\nu \cdot c \cdot \mathbf{u}_1, c \cdot \mathbf{u}_2)\|_2 > B$ **then**
  $f_{\mathsf{fail}} := \top$
  return $\bot$
$\mathbf{z}_i \leftarrow \mathrm{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$
if $\mathbf{z}_i = \bot$ **then** ▷ if reject
  $\mathbf{z}_i \leftarrow (\mathbf{z}|\mathrm{rej})_{v=c\cdot\mathbf{s}_i^{\mathsf{part}}}$
$\mathbf{t}_i^{\mathsf{part}} := \sum_{I\in m_i} \mathbf{t}_I$
$\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\mathsf{part}}$
if $\mathbf{z}_i \neq \bot$ **then**
  $\mathsf{Responses}[(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, i)] := \mathbf{z}_i$

Rest is identical

Figure 26: Eighth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

Game_{10}. In this hybrid, we replace the real rejection sampling algorithm $\mathrm{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M; \mathbf{r}_i)$ by an ideal functionality $\mathrm{Ideal}(\chi_{\mathbf{z}}, M)$ which is independent of the secret. This is formalized in Fig. 28.

We rely on the Rényi divergence to prove that this only slightly increase the probability that $\mathcal{A}$ wins the game. For conciseness, we will abuse Rényi divergence notations to take a random variable as input, implicitly corresponding to the divergence between the marginal distributions from Game_9 and Game_{10}.

We wish to apply a Rényi divergence argument, and its multiplicativity, to the responses $(\mathbf{z}_j)_{j\in[Q_s]}$ produced in signing oracles. However, these responses all depend on the secrets and are not independent, which prevent a simple application of multiplicativity.

To solve that, we will instead apply the Renyi properties from Lemma A.1 to the tuple $X = ((\mathsf{sk}_i)_{i\in[N]}, \mathbf{z}_1, ..., \mathbf{z}_{Q_s})$.

By the data processing inequality of the Rényi divergence, since we can see Game_9 and Game_{10} as functions of X, their Rényi divergence is bounded by the Rényi divergence of X.

We then apply the multiplicativity of Rényi divergence Lemma A.1:

$$R_\infty(\mathsf{Game}_9 || \mathsf{Game}_{10}) \leqslant \underbrace{R_\infty((\mathsf{sk}_i)_{i\in[N]})}_{=1} \cdot \prod_j r_j$$

where $r_j = \max_{(\mathsf{sk}_i)_{i\in[N]}} R_\infty(\mathbf{z}_j \mid (\mathsf{sk}_i)_{i\in[N]})$.

We can apply again the multiplicativity of the Renyi divergence:

$$R_\infty(\mathbf{z}_j \mid (\mathsf{sk}_i)_{i\in[N]}) \leqslant \max_{\|\mathbf{v}\|\leqslant B} R_\infty(\mathbf{z}_j \mid c_j \cdot \mathbf{s}_j^{\mathsf{part}} = \mathbf{v})$$

as $\mathbf{z}_j$ only differs between the games when $\|c \cdot \mathbf{s}_j^{\mathsf{part}}\| \leqslant B$.

Game_9

Identical to Game_8

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

assert$\{\ \mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}\ \}$
assert$\{\ \mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] = \top\ \}$
$\mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] := \bot$
if $(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \cdot) \notin \mathsf{ToProgram}$ **then**
  $\tilde{c} \xleftarrow{\$} \{0,1\}^{256}; c \leftarrow \mathcal{C}$
  $\mathsf{ToProgram} := \mathsf{ToProgram} \cup \{(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \tilde{c}, c)\}$
Pick (a_1, a_2, a_3, c) from $\mathsf{ToProgram}$ s.t. $(a_1, a_2, a_3) = (\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg})$
$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I\in m_i} \mathbf{s}_I$; $(\mathbf{u}_1, \mathbf{u}_2) = \mathbf{s}_i^{\mathsf{part}}$
if $\|(\nu \cdot c \cdot \mathbf{u}_1, c \cdot \mathbf{u}_2)\|_2 > B$ **then**
  $f_{\mathsf{fail}} := \top$
  return $\bot$
$\mathbf{z}_i \leftarrow \mathrm{Rej}(c \cdot \mathbf{s}_i^{\mathsf{part}}, \chi_{\mathbf{z}}, \chi_{\mathbf{r}}, M)$
if $\mathbf{z}_i = \bot$ **then**
  $\mathbf{w}_i \xleftarrow{\$} \mathcal{R}_q^k$
else
  $\mathbf{t}_i^{\mathsf{part}} := \sum_{I\in m_i} \mathbf{t}_I$
  $\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{z}_i - c \cdot \mathbf{t}_i^{\mathsf{part}}$
  $\mathsf{Responses}[(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, i)] := \mathbf{z}_i$

Rest is identical

Figure 27: Nineth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

We finally apply Lemma C.1 to deduce that,

$$R_\infty(\mathsf{Game}_9 || \mathsf{Game}_{10}) \leqslant \left(1 + \frac{\varepsilon}{M-1}\right)^{Q_s}$$

Applying the probability preservation of the Rényi divergence, we get:

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_9} \leqslant \left(1 + \frac{\varepsilon}{M-1}\right)^{Q_s} \cdot \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_{10}}$$

Game_{11}. In this hybrid, we remove the abort condition on the norm of $c \cdot \mathbf{s}_i^{\mathsf{part}}$.

This can only increase the winning probability of the adversary, hence

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_{10}} \leqslant \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_{11}}$$

Game_{12}. In this hybrid, we replace the public key by the rounding of a uniform value, i.e. $\mathsf{Power2Round}(\mathbf{t}, d)$ where $\mathbf{t} \xleftarrow{\$} \mathcal{R}_q^k$. To preserve consistency, we fix a secret index I such that $I_0 \subseteq \mathsf{honest}$, i.e. $\mathbf{s}_{I_0}$ is not given to the adversary, and we replace $\mathbf{t}_{I_0}$ with $\mathbf{t}_{I_0} := \mathbf{t} - \sum_{I\neq I_0} \mathbf{t}_I$. This is formalized in Fig. 29.

As secrets are no longer used by honest parties in the scheme at this stage, we can replace $\mathbf{t}_{I_0}$ by a uniform value by the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{s}}}$ assumption. Equivalently, we can then sample $\mathbf{t}$ uniformly at random and define $(\mathbf{t}_\top, \mathbf{t}_\bot) := \mathsf{Power2Round}(\mathbf{t}, d)$ and $\mathbf{t}_{I_0} := \mathbf{t} - \sum_{I\neq I_0} \mathbf{t}_I$.

Formally, there exists an adversary $\mathcal{B}_5$ against the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{s}}}$ problem such that:

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_{11}} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_{12}}\right| \leqslant \mathsf{Adv}_{\mathcal{B}_5}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{s}}}}$$

Game_{10}

Identical to Game_6

$\mathsf{ShareSign}_2(\mathsf{vk}, \mathsf{act}, \mathsf{msg}, \mathsf{pm}_1, i, \mathsf{sk}_i, \mathsf{st}_i)$

assert$\{\ \mathsf{act} \subseteq [N] \wedge i \in \mathsf{act}\ \}$
assert$\{\ \mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] = \top\ \}$
$\mathsf{UnopenedCmt}[(i, \mathsf{pm}_1^i)] := \bot$
if $(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \cdot) \notin \mathsf{ToProgram}$ **then**
  $\tilde{c} \xleftarrow{\$} \{0,1\}^{256}; c \leftarrow \mathcal{C}$
  $\mathsf{ToProgram} := \mathsf{ToProgram} \cup \{(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, \tilde{c}, c)\}$
Pick (a_1, a_2, a_3, c) from $\mathsf{ToProgram}$ s.t. $(a_1, a_2, a_3) = (\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg})$
$\mathbf{m} := \mathsf{RSSRecover}(\mathsf{act})$
$\mathbf{s}_i^{\mathsf{part}} := \sum_{I\in m_i} \mathbf{s}_I$; $(\mathbf{u}_1, \mathbf{u}_2) = \mathbf{s}_i^{\mathsf{part}}$
if $\|(\nu \cdot c \cdot \mathbf{u}_1, c \cdot \mathbf{u}_2)\|_2 > B$ **then**
  $f_{\mathsf{fail}} := \top$
  return $\bot$
$\mathbf{z}_i \leftarrow \mathrm{Ideal}(\chi_{\mathbf{z}}, M)$
if $\mathbf{z}_i = \bot$ **then**
  $\mathbf{w}_i \xleftarrow{\$} \mathcal{R}_q^k$
else
  $\mathbf{t}_i^{\mathsf{part}} := \sum_{I\in m_i} \mathbf{t}_I$
  $\mathbf{w}_i := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{z}_i - c \cdot \mathbf{b}_i^{\mathsf{part}}$
  $\mathsf{Responses}[(\mathsf{act}, (\mathsf{cmt}_j)_{j\in\mathsf{act}}, \mathsf{msg}, i)] := \mathbf{z}_i$

Rest is identical

Figure 28: Tenth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

Game_{13}. In this hybrid, we first sample the high bits of the public key $\mathbf{t}_\top$ from an ML-DSA public key, then we sample the full key $\mathbf{t}$ from the distribution $\mathcal{R}_q^k$ conditioned on the value of $\mathbf{t}_\top$. This is formalized in Fig. 30.

We can equivalently sample $(\mathbf{t}, \mathbf{t}_\top)$ in Game_{12} by first sampling $(\mathbf{t}_\top, \cdot) = \mathsf{Power2Round}(\mathcal{U}(\mathcal{R}_q^k, d))$ and then sampling $\mathbf{t}$ conditioned on $\mathbf{t}_\top$. Then, we can see that Game_{13} just replaces $\mathcal{U}(\mathcal{R}_q^k)$ in this distribution by an MLWE sample with secrets sampled in $\chi_{\mathbf{s}}$, which is again indistinguishable under the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{s}}}$ assumption.

Formally, there exists an adversary $\mathcal{B}_6$ against the $\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{s}}}$ problem such that:

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_{12}} - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Game}_{13}}\right| \leqslant \mathsf{Adv}_{\mathcal{B}_6}^{\mathsf{MLWE}_{q,k,\ell,\chi_{\mathbf{s}}}}$$

Conclusion. Finally, we reduce Game_{13} to the unforgeability of ML-DSA. We design an adversary $\mathcal{B}_7$ against the unforgeability of ML-DSA, and simulate the view of the adversary in Game_{13}.

$\mathcal{B}_7$ takes as input an ML-DSA public key $\mathsf{pk} = (\rho, \mathbf{t}_\bot)$, and random oracles ExpandA, H and SampleInBall.

It does the folllowing changes:

- It programs $\mathsf{ExpandA}(\rho) = \mathbf{A}$.
- It chooses $\mathbf{t}_\top$ and ρ provided by the unforgeability challenger in the threshold Keygen.
- Queries to the random oracles H and SampleInBall are lazily passed to the random oracles provided by the unforgeability challenger instead of honestly sampling their values. Note that in case Game_{12} programs them otherwise, this behavior is not affected.

$\mathcal{B}_7$ forwards any forgery it receives from $\mathcal{A}$ to the ML-DSA unforgeability challenger.

The view of the adversary $\mathcal{A}$ is identically distributed since the ML-DSA random oracles sample images from the same distributions as Game_{13}, and the ML-DSA public key is sampled as $(\rho, \mathbf{t}_\top)$ in Game_{13}.

Then, we can see that a valid forgery in Game_{13} will also be a valid forgery for ML-DSA. Indeed, the verification procedure only depends on ρ and $\mathbf{t}_\top$ which are chosen identical for the ML-DSA signature and the threshold

Game_{12}

Identical to Game_8

$\mathsf{Keygen}(\kappa, N, T) \to (\mathsf{vk}, \mathsf{sk})$

$\rho \leftarrow \{0,1\}^{256}$
$\mathbf{A} := \mathsf{ExpandA}(\rho)$
for $I \subset [N]$ s.t. $|I| = N - T + 1$ and $I \neq I_0$ **do**
  $\mathbf{s}_I \leftarrow \chi_{\mathbf{s}}$
  $\mathbf{t}_I := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{s}_I$
  ▷ Compute partial public keys
  for $i \in I$ **do** ▷ Distribute the keys
    $\mathsf{sk}_i = \mathsf{sk}_i \cup \{\mathbf{s}_I\}$
$\mathbf{t} \xleftarrow{\$} \mathcal{R}_q^k$
$\mathbf{t}_{I_0} := \mathbf{t} - \sum_{I \neq I_0} \mathbf{t}_I$
$(\mathbf{t}_\top, \mathbf{t}_\bot) := \mathsf{Power2Round}(\mathbf{t}, d)$
$\mathsf{vk} := (\rho, \mathbf{t}_\top)$ ▷ Verification key
$\mathsf{sk} := (tr, \mathsf{sk}_i)_{i \in [N]}$
return $(\mathsf{vk}, \mathsf{sk})$

Figure 29: Twelfth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted.

signature. Also, the challenge random oracles coincide on messages for which forgeries are valid (i.e. messages for which no signing oracle was called) as a $\tilde{c}$ programmed by Game_{13} will never be used by the ML-DSA oracles thanks to the condition added in Game_5. Hence,

$$\mathsf{Adv}^{\mathsf{UF}}_{\mathcal{B}_7} = \mathsf{Adv}^{\mathsf{Game}_{13}}_{\mathcal{A}}$$

We finally collect all the bounds to obtain the theorem statement, noting that all the intermediary adversaries have an execution time roughly equal to $\mathcal{T}(\mathcal{A})$. □

Game_{13}

Identical to Game_8

$\mathsf{Keygen}(\kappa, N, T) \to (\mathsf{vk}, \mathsf{sk})$

$\rho \leftarrow \{0,1\}^{256}$
$\mathbf{A} := \mathsf{ExpandA}(\rho)$
for $I \subset [N]$ s.t. $|I| = N - T + 1$ and $I \neq I_0$ **do**
  $\mathbf{s}_I \leftarrow \chi_{\mathbf{s}}$
  $\mathbf{t}_I := \begin{bmatrix}\mathbf{A} & \mathbf{I}\end{bmatrix} \cdot \mathbf{s}_I$
▷ Compute partial public keys
  for $i \in I$ **do** ▷ Distribute the keys
    $\mathsf{sk}_i = \mathsf{sk}_i \cup \{\mathbf{s}_I\}$
$(\mathbf{s}, \mathbf{e}) \leftarrow \chi_{\mathbf{s}}$
$\widehat{\mathbf{t}} := \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$
$(\mathbf{t}_\top, \cdot) := \mathsf{Power2Round}(\widehat{\mathbf{t}}, d)$
Sample $\mathbf{t}$ from $\mathcal{U}(\mathcal{R}_q^k)$ conditioned on $(\mathbf{t}_\top, \cdot) = \mathsf{Power2Round}(\mathbf{t}, d)$
$\mathbf{t}_{I_0} := \mathbf{t} - \sum_{I \neq I_0} \mathbf{t}_I$
$\mathsf{vk} := (\rho, \mathbf{t}_\top)$ ▷ Verification key
$\mathsf{sk} := (tr, \mathsf{sk}_i)_{i \in [N]}$
return $(\mathsf{vk}, \mathsf{sk})$

Figure 30: Thirteenth hybrid of the unforgeability proof of Threshold ML-DSA. Difference with the previous hybrid are highlighted .